參考用書：Computer Organization and Design, Revised Fourth Edition
David A. Patterson, John L. Hennessy

# 計算機組織

## 完整版講義

A4 直式 · 上下兩張 · Lin Cing

---

## 目錄

一、電腦抽象、指令與算術

第 1 章　電腦抽象與效能 2

第 2 章　指令與程式執行 22

第 3 章　電腦算術 49

二、處理器與記憶體階層

第 4 章　處理器資料路徑與管線 84

第 5 章　記憶體階層 111

三、儲存、I/O 與平行系統

Chapter 6 儲存與 I/O 144

Chapter 7 多核、多處理器與叢集 165

延伸查閱：GPU 與程式工具鏈

Appendix A 圖形與運算 GPU 187

Appendix B 組譯器、連結器與執行環境 209

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 電腦抽象、ISA 與可量化效能

從一個 `sum += a[i]` 到電晶體切換，中間隔著多層契約。本章的目標是讓每一層都有可量化的邊界。

本課以 MIPS32 作為教學 ISA。MIPS 是本書選用的指令集範例，不代表所有現代處理器都直接執行 MIPS。ARM 與 x86 會在需要比較時出現。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 前置知識與學習成果

必備工具包括二進位、科學記號、比率與加權平均。課程結束時，學生應能以單位完整的式子預測程式執行時間，並說明預測在哪些條件下失效。

本課所稱「電腦效能」必須附上工作負載。沒有工作負載的「處理器很快」是測量條件不完整的句子。

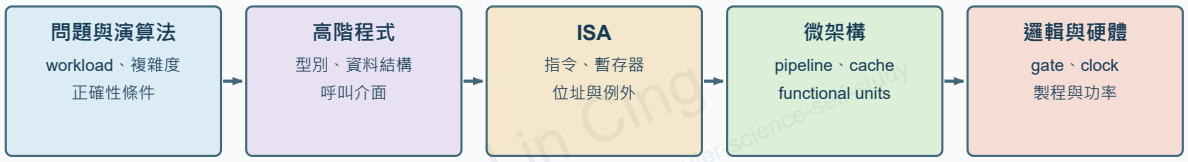
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 1.1 電腦抽象層

電腦系統用分層管理複雜度。應用軟體使用系統呼叫與函式庫，編譯器產生 ISA 指令，微架構把指令展開為資料路徑操作，邏輯與實體層實現暫存器、加法器與記憶體。

相鄰層透過介面交換可觀察行為。介面讓上層不必知道下層每個電晶體，但最佳化會跨層傳遞資訊，例如資料局部性影響編譯器排程與快取命中。

抽象堆疊以介面承接上下層契約




上層提出可觀察契約，下層提供實作；最佳化資訊可跨層傳遞，但不可破壞介面。

抽象不是隱藏一切。軟體可透過指令、位址、例外與效能計數器觀察下層的部分行為。



## ISA 與 microarchitecture

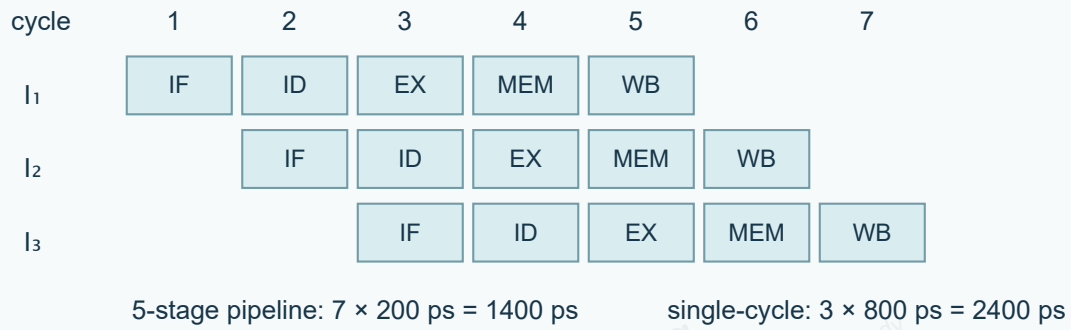
Instruction set architecture，中文為指令集架構，定義軟體可見的指令、暫存器、資料型別、定址、權限與例外。兩臺處理器可以有相同 ISA，卻有不同 pipeline 深度、cache 大小與執行單元。

Microarchitecture，中文為微架構，定義如何實作 ISA。它決定某條指令在各時鐘週期走到哪個階段，以及處理器能否同時維持多個 in-flight instructions。

若程式結果與 ISA 契約一致，微架構可以自由重排、預測與快取。若重排讓程式觀察到不同記憶體結果，必須由同步規則約束。

ISA 像函式介面，微架構像函式內部的資料結構與演算法。API 相同，成本與並行度可以不同。

## 同一 ISA，不同實作：三條指令的時間




假設同一執行檔、相同輸入、三條互不相依的指令，兩台機器都正確執行 ISA，且沒有停滯或例外；非記憶體指令也占用圖中的 MEM 階段。單週期機每條指令用 1 個 800 ps cycle，完成時間 $3 \times 800 = 2400$ ps。

五階管線每階 1 個 200 ps cycle，從第一個 IF 到最後一個 WB 共 7 cycles，完成時間 $7 \times 200 = 1400$ ps；加速比為 $2400/1400 = 12/7$。指令結果相同，不代表執行時間相同。



## 應用類別與設計取捨

個人電腦、伺服器、嵌入式與 mobile 系統有不同的限制。伺服器最佳化總吞吐量與 availability，mobile 系統受到 energy 與 thermal envelope 限制，嵌入式控制則重視 worst-case response。

同一項技術在不同市場的價值不同。有大容量 cache 可改善一般負載，但會占面積與增加 static power。低功耗休眠可延長電池壽命，卻可能提高喚醒延遲。

設計取捨至少包含效能、成本、功率、可靠度與可程式性五個量，不能只報告其中一項。

## 課程地圖

第 2 章建立 ISA，第 3 章處理整數與浮點算術，第 4 章把 ISA 變成 datapath 與 pipeline。第 5 章處理 cache 與 virtual memory，第 6 章處理 storage 與 I/O，第 7 章討論 multicore、multiprocessor 與 cluster。

後面的章節反覆使用本章三個工具：單位分析、局部化比較、以 workload 驗證。看到新的架構名詞時，先問它改變了哪個資源與哪個時間分量。

| 主線 | 本章建立的模型 | 後續機制 |
| --- | --- | --- |
| 資料 | operands、結果與 IC | 第2章暫存器/位址 → 第3章算術 → 第4章 datapath → 第5章 cache |
| 控制 | ISA 的可觀察狀態與時間 | 第2章 branch/call → 第4章 hazard/exception → 第6章 I/O completion |
| 通訊 | 搬移 bytes 與共享資源 | 第5章 coherence → 第6章 DMA/links → 第7章 synchronization/networks |



## 1.2 程式如何落到硬體

高階語言先經 compiler 產生 assembly，assembler 再產生 machine instructions。Static linker 合併 object files，loader 建立 process 可用的位址空間，處理器最後依 ISA 解碼並執行。

每一步都產生可檢查內容。編譯器可改變指令數與暫存器使用；assembler 決定 encoding；linker 解決符號與 relocation；loader 建立初始記憶體映射。

效能分析不能只比較 C 原始碼行數，因為最佳化層級、資料型別與編譯器輸出會改變 instruction count。

請區分 source code、assembly 與 machine code。本課的 ISA 圖都畫在 assembly 與機器碼之間。

## 抽象介面與可攜性

ISA 讓同一個 executable 能在實作相同 ISA 的處理器上執行。作業系統還要提供 system call 與 binary interface，程式才完整可攜。

應用 binary interface，簡稱 ABI，規定暫存器保存方式、呼叫慣例、資料對齊與 executable 格式。它比 ISA 更接近程式工具鏈。

一個處理器可能維持舊 ISA 以執行既有程式，同時在內部把舊指令解譯成新的 micro-operations。這是實作選擇，不表示 ISA 消失。

區分 compatibility、portability 與 equivalence。能執行不代表得到相同效能。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 1.3 開啟電腦外殼

系統由 processor、volatile memory、nonvolatile storage、I/O 與 interconnect 構成。輸入裝置把外部狀態轉成資料，輸出裝置把資料轉成可觀察效果。

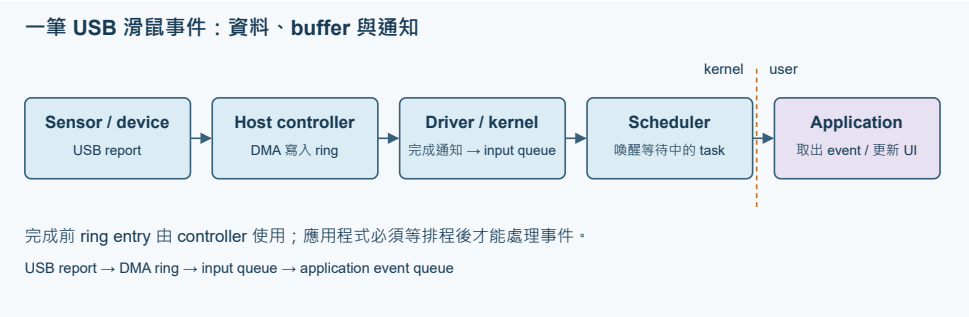
處理器內有 datapath 與 control，外部有 DRAM 與 persistent storage。匯流排或點對點 interconnect 傳遞位址、資料與控制訊息。

周邊裝置不是「電腦之外」的附屬品。鍵盤、滑鼠、顯示器與網路都會透過 interrupt、DMA 或 memory-mapped I/O 影響系統效能。

以下固定 USB 滑鼠、host-controller DMA ring 與一個等待輸入的應用程式。圖中的 buffer 是不同責任邊界，不假設每個箭頭都複製整份資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## USB 輸入事件的路徑




Controller 先將 report 寫入 driver 配置的 ring，完成通知讓 driver 取得已完成項，再交給 input queue；scheduler 喚醒應用後，應用才取走事件。DMA 完成前不能回收 ring slot；應用醒來時間也不等於封包抵達時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從滑鼠看周邊介面

滑鼠感測器產生位移資訊。裝置控制器把它整理成封包，透過 USB 或 Bluetooth 等介面傳給 host controller。驅動程式與作業系統再交給 input subsystem。

每次移動不一定立即喚醒應用程式。系統會批次處理事件以降低 CPU 喚醒成本，這形成 latency 與 energy 的取捨。

測量「滑鼠反應慢」時要分清楚裝置取樣率、匯流排傳輸、作業系統排程、應用程式 frame rate 與顯示器更新率。

這個例子用來練習端到端延遲分解，數字未指定時不可假設由單一元件的速度決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 從平板看系統組成

Tablet 使用 system on chip，簡稱 SoC，把多個 CPU cores、GPU、media processor、memory controller 與 I/O controller 放在同一晶片。整合降低封裝與通訊成本，也提高 power density 與 thermal 管理難度。

使用者感受的 response time 包含 touch sampling、GUI composition、computation 與 display scan-out。處理器 burst 很快完成時，仍需等待顯示 pipeline。

Mobile workload 常見短暫 burst 與長時間 idle。平均值會掩蓋 tail latency 與 energy state transition。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 主機板與資料路徑

主機板提供芯片與周邊的實體連接。現代系統多採 point-to-point link，不再把所有裝置接在同一條共享 bus，因為共享 bus 的 contention 與電氣負載會限制頻寬。

晶片內的 interconnect 連接 cores、last-level cache 與 memory controller。Cache coherence 維持共享資料的可觀察一致性。

拓樸決定 traffic hop count 與 contention。頻寬峰值不能直接當成應用程式可得到的持續 bandwidth。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 資料保存層級

Register 與 cache 屬 volatile storage，斷電後資料消失。DRAM 也是 volatile。Flash、magnetic disk 與 optical media 屬 nonvolatile storage。

可靠性與備份是不同問題。RAID 可容忍某些裝置故障，但無法取代異地備份。檔案系統 crash consistency 也不能只靠平均值中的 MTBF 保證。

Temporary copy 必須和 authoritative copy 區分。Cache line 與 write-back dirty bit 改變資料在哪裡最新，但不改變程式的 logical data model。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 網路為何屬於系統組織

現代電腦的效能與可用性受 network latency、bandwidth 與 protocol overhead 影響。機架內與跨資料中心的距離不同，光速、交換器佇列與 congestion control 共同決定端到端時間。

資料可能先從 storage 進入 memory，再由 DMA engine 送到 network interface。CPU 不必逐 byte 搬移，但仍需設定 descriptor 與處理完成事件。

系統設計需一起考慮 computation、storage 與 network，否則算得再快的服務可能等待 I/O。

在無重疊、連線已建立的教學模型，固定 request encoding 0.2 ms、等待 0.3 ms、去程傳輸與傳播 2.0 ms、server computation 1.5 ms、回程 2.0 ms。總 latency 為 0.2+0.3+2.0+1.5+2.0=6.0 ms。只把 server computation 加速3倍，總時間為5.0 ms，整體 speedup=6/5=1.2。各數字是明定模型輸入，若網路傳輸與計算重疊，必須改以 dependency critical path 計算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 製程與成本

IC 技術以 transistor integration 與 feature size 描述。較小元件通常降低每單位功能的面積與動態能量，但 leakage、變異與製造成本會使成本下降不再自動持續。

晶圓良率使每顆可用晶片成本不是裸晶面積的線性函數。大晶片能整合更多功能，也承受較高 defective die 比率。

成本要指定範圍。單顆晶片製造、封裝、電源、冷卻、伺服器機架與營運 energy 的成本不同。

不要把「電晶體更多」當成「每個程式更快」。這個結論需要 parallel 或 architectural 利用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Die 面積、良率與成本

固定wafer diameter $D = 200$ mm、每片成本1000成本單位、每die面積 $A$（mm²）。邊緣損失的幾何近似採

$$N_{\text{die}} = \left\lfloor \frac{\pi (D/2)^2}{A} - \frac{\pi D}{\sqrt{2A}} \right\rfloor .$$

另明定負二項良率模型 $Y = (1 + dA_c/\alpha)^{-\alpha}$，$d = 0.5$ defects/cm²、clustering參數 $\alpha = 2$、$A_c = A/100$ cm²。此處100 mm²=1 cm²，不能直接把mm²代入defects/cm²。忽略封裝、測試、備援及其他成本，平均good-die成本為 $1000/(N_{\text{die}}Y)$。

| Die面積 | 幾何項與邊緣項 | Gross dies | Yield | 預期good dies | 每good die成本 |
|---|---|---|---|---|---|
| 100 mm² | floor(314.159−44.429) | 269 | (1+0.5/2)⁻²=0.64 | 172.16 | 5.8086 |
| 200 mm² | floor(157.080−31.416) | 125 | (1+1/2)⁻²=4/9 | 55.5556 | 18.0000 |

面積加倍同時降低gross dies及yield，本模型成本比為18/5.8086≈3.10，並非2。Good dies為期望值，實際產出為整數；本例的幾何/良率與成本數字皆為教學假設，非製造報價。

## 1.4 效能的定義

Response time，中文為回應時間，是單一工作從開始到完成經過的時間。Throughput，中文為吞吐量，是單位時間完成的工作數。

若最佳化縮短 critical path，可能改善 response time。若增加流水線、batch 或多核心，可能主要改善 throughput。

任何延遲改善百分比都要用同一 baseline。$T_{\text{new}} = T_{\text{old}}/S$，其中 speedup $S > 0$。若時間減半，$S = 2$，時間節省比例是 $50\%$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 延遲與吞吐量

以輸送帶為例，單筆 work 經過五個 stations 的總時間是 latency。穩定狀態下每週期離開一筆，是 throughput。Pipelining 通常降低平均完成間隔，舊工作的 latency 未必等比例降低。

Batch processing 可攤薄 setup 成本，提高 throughput，但第一筆結果可能更晚。多核心能並行不同工作，卻受 shared resources 與 synchronization 限制。

公平比較要固定 workload、輸入大小、warm-up、軟體版本與量測視窗。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 基準、比率與幾何平均

Benchmark 必須指定 workload、輸入大小、系統配置、compiler flags、warm-up 與量測視窗。這些條件不同時，兩個「SPEC 分數」不是同一個實驗。

以 reference machine 的程式 $i$ 執行時間 $T_{\text{ref},i}$ 除以受測機器的 $T_{X,i}$，得到

$$r_{X,i} = \frac{T_{\text{ref},i}}{T_{X,i}}.$$

分數愈大表示受測時間愈短。單一程式可以直接比較；多個程式若要形成摘要，通常取幾何平均

$$GM_X = \left(\prod_{i=1}^{n} r_{X,i}\right)^{1/n}.$$

幾何平均適合比率的原因可以直接證明。兩台機器 X、Y 的幾何平均比值為

$$\frac{GM_X}{GM_Y} = \left(\prod_{i=1}^{n} \frac{r_{X,i}}{r_{Y,i}}\right)^{1/n} = \left(\prod_{i=1}^{n} \frac{T_{Y,i}}{T_{X,i}}\right)^{1/n}.$$

每個程式的 $T_{\text{ref},i}$ 都在 X/Y 的比值中成對抵銷。因此 reference machine 換成另一台、各程式 reference time 各自不同時，X 對 Y 的幾何平均比值仍不變；算術平均沒有這個性質。

摘要分數仍不代表所有 workload。若某使用者的工作有 90% 時間落在單一 kernel，應以該 workload 重新加權或直接比較那支程式；不能只用幾何平均宣告普遍最快。

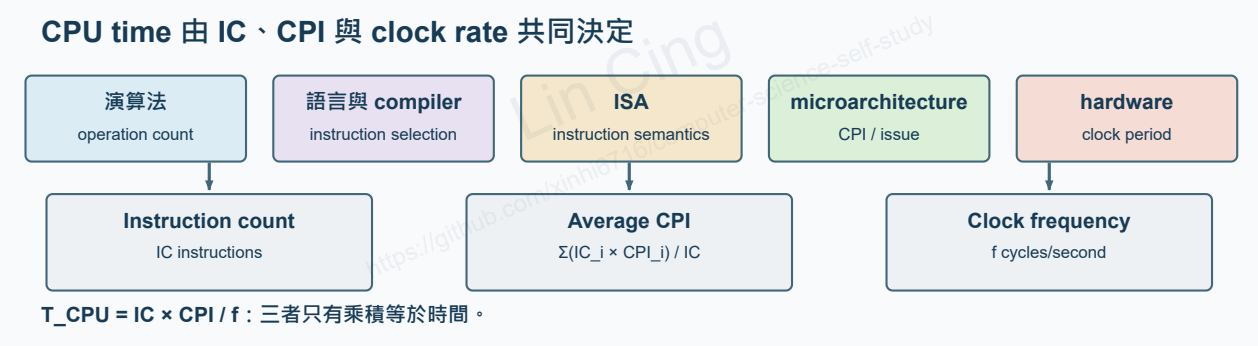


## CPU time 的可重建模型

CPU time 計算CPU替程式執行的時間；以下不等式限定單執行緒、同一量測視窗，且 user/system accounting不重複：

$$T_{\text{elapsed}} \geq T_{\text{CPU}} = T_{\text{user}} + T_{\text{system}}.$$

I/O等待、被排程器換出與OS替其他工作執行仍消耗elapsed time。多執行緒process若將各 thread CPU time相加，總CPU-seconds可以超過elapsed seconds：4cores各工作1秒，總CPU time為4 CPU-s，elapsed為1 s。比較效能時必須明列thread/process accounting與使用的時間分母。




先從 cycle 數開始：

$$C = IC \times CPI.$$

## CPU time 的可重建模型 （續）

每個 cycle 長 $T_c$ 秒，所以

$$T_{\text{CPU}} = C\,T_c = IC \times CPI \times T_c = \frac{IC \times CPI}{f}.$$

量綱是

$$\text{instructions} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}} = \text{seconds}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 完整比較例

同一程式在 A、B 上都執行 $I = 10^9$ 條 ISA instructions。A 的 CPI=2.0、clock period=250 ps；B 的 CPI=1.2、clock period=500 ps。

$$C_A = 10^9(2.0) = 2.0 \times 10^9, \qquad T_A = 2.0 \times 10^9(250\,\text{ps}) = 0.500\ \text{s}.$$
$$C_B = 10^9(1.2) = 1.2 \times 10^9, \qquad T_B = 1.2 \times 10^9(500\,\text{ps}) = 0.600\ \text{s}.$$

因此

$$S_{A/B} = \frac{T_B}{T_A} = \frac{0.600}{0.500} = 1.2.$$

A 比 B 快 1.2 倍。B 的 CPI 較低，但每個 cycle 較長，所以仍較慢。只比較 CPI、頻率或 instruction count 都會漏掉另外兩項。

本例直接假設兩者實際動態 $IC$ 相同。同一執行檔與輸入若產生相同指令追蹤，就有相同指令數；同一 ISA 與高階程式本身不保證如此，不同 compiler、設定或執行環境都可能改變指令數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 加權 CPI 與「少一條指令未必較快」

若類別 $i$ 有 $IC_i$ 條 instruction，並採用可加總且不重複計算的 cycles 攤計 $CPI_i$，總 cycles 與 CPI 是

$$C = \sum_i IC_i\,CPI_i, \qquad CPI_{\text{avg}} = \frac{\sum_i IC_i\,CPI_i}{\sum_i IC_i}.$$

分母必須是全部執行過的 instructions，不是類別數，也不是 source line 數。流水線或多發射處理器可重疊指令執行：單條延遲不能直接相加。下例的類別 CPI 是已依題設加總的週期成本；若頻率隨時間變動，則依區間累加時間，或用同一活躍區間的有效平均頻率 $C/T$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 完整反例

硬體有三類 instruction：

| 類別 | CPI |
|---|---|
| A | 1 |
| B | 2 |
| C | 3 |

兩個 compiler 對同一高階語句產生：

| 序列 | A 條數 | B 條數 | C 條數 | 總 instruction |
|---|---|---|---|---|
| 1 | 2 | 1 | 2 | 5 |
| 2 | 4 | 1 | 1 | 6 |

序列 1：

$$C_1 = 2(1) + 1(2) + 2(3) = 10, \qquad CPI_1 = 10/5 = 2.0.$$

序列 2：

$$C_2 = 4(1) + 1(2) + 1(3) = 9, \qquad CPI_2 = 9/6 = 1.5.$$

在相同 frequency 下，

$$T_1 = 10T_c, \quad T_2 = 9T_c, \quad S_{2/1} = 10/9 \approx 1.111.$$

## 完整反例 （續）

序列 2 多一條 instruction，卻少一個 cycle。原因不是「instruction 較少一定快」，而是總 cycle 數才決定 CPU time。這個結果也提醒我們：instruction mix 改變時，不能把單一 CPI 當常數。

量測時可用 profiler、ISA simulator 或 hardware performance counters 取得 $IC_i$ 與 cycle 來源；三種方法的覆蓋範圍與額外 overhead 必須另外記錄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 經典效能方程式的邊界

效能方程式是 accounting identity，不是保證任何局部改動會改善總時間。它假設 $IC$、CPI 與 $f$ 能被合理量測且 workload 固定。

CPU time 也不等於 wall-clock response time。若程式等待 I/O、作業系統排程或網路，必須把等待時間納入。

對多核平行程式，還需考慮使用 core 數 $p$、平行比例、負載不平衡與 synchronization overhead。第 7 章會用 Amdahl 與 Roofline 延伸本章。

若處理器頻率不變、CPI 降低，但 compiler 產生更多指令，總結果取決於兩個乘數的競賽。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 完整效能預算例

甲程式執行 $2.0 \times 10^8$ 條 instructions，CPI=1.25，頻率 2.0 GHz。

1. cycles：$2.0 \times 10^8 \cdot 1.25 = 2.5 \times 10^8$。
2. cycle time：$1/(2.0 \times 10^9) = 0.5$ ns。
3. CPU time：$2.5 \times 10^8 \cdot 0.5\ \text{ns} = 0.125$ s。

$$T_{\text{A}} = \frac{(2.0 \times 10^8)(1.25)}{2.0 \times 10^9} = 0.125\ \text{s}.$$

新 compiler 把 instruction count 降至 $1.6 \times 10^8$，但 CPI 升為 1.40，frequency 仍為 2.0 GHz：

$$C_{\text{B}} = 1.6 \times 10^8 (1.40) = 2.24 \times 10^8,$$

$$T_{\text{B}} = \frac{2.24 \times 10^8}{2.0 \times 10^9} = 0.112\ \text{s}, \qquad S = \frac{0.125}{0.112} = 1.1161.$$

Instruction count 降低 $20\%$，但 CPI 上升 $12\%$，所以時間只降低

$$(0.125 - 0.112)/0.125 = 10.4\%.$$

如果把 B 的 CPI 誤用成 A 的 1.25，會得到 $T = 0.100$ s、speedup 1.25；這個答案忽略 instruction mix 已改變。



## 1.5 功耗牆

Dynamic power 的簡化式是

$$P_{\text{dyn}} = C V^2 f\, \alpha,$$

其中 $C$ 為被切換電容，$V$ 為電壓，$f$ 為頻率，$\alpha$ 為 activity factor。提高頻率通常也要提高 voltage，因此 voltage-frequency scaling 會使 power 成長快於線性。

晶片還有 leakage power。降低供應電壓本身通常會降低漏電，但也會降低速度；若為了在低電壓維持速度而同時降低臨界電壓，次臨界漏電可能增加，製程變異的相對影響也可能變大。

散熱限制決定可持續的 power density。接近或超過平台的溫度、電流或功率限制時，時脈可能降低或部分資源停用；TDP是散熱設計參數，不是所有處理器固定且唯一的throttling門檻。

## 單核到多核與 Amdahl 的完整推導

功率密度、clock distribution、可用的 instruction-level parallelism 與記憶體延遲共同使提高單核頻率不再是永續的效能來源。多核提高 aggregate throughput，並可能縮短能把工作切成獨立部分的大型問題 elapsed time。

令固定工作的 baseline time 為 $T_0$，其中 $fT_0$ 可由 $N$ 個 worker 分擔，$(1-f)T_0$ 必須序列執行。假設工作均分且沒有新增成本：

$$T_N = (1-f)T_0 + \frac{fT_0}{N} = T_0\left(1 - f + \frac{f}{N}\right).$$

$$S_N = \frac{T_0}{T_N} = \frac{1}{1 - f + f/N}.$$

當 $N \to \infty$，平行部分趨近 0，但序列部分仍在：

$$S_\infty = \frac{1}{1-f}.$$



## 反解所需平行比例

若要達到指定的 finite-$N$ speedup $S$，必須

$$\frac{1}{S} = 1 - f + \frac{f}{N} = 1 - f\left(1 - \frac{1}{N}\right),$$

因此

$$f \geq \frac{N(S-1)}{S(N-1)}, \qquad 1 < S < N.$$

例如 $N = 100, S = 90$：

$$f \geq \frac{100(89)}{90(99)} \approx 0.998878,$$

序列比例只能約 $0.1122\%$。若 $S > N$，此理想模型不可達；$S = N$ 在 $f = 1$ 且新增成本為零時可達。只要 $f < 1$，有限 $N > 1$ 才有嚴格的 $S_N < N$。

## 原書型反例

程式 baseline 為 100 s，其中乘法占 80 s、其他運算占 20 s。若目標是總時間 20 s，即 5 倍速度，則

$$20 = \frac{80}{s_{\text{mul}}} + 20.$$

移項後 $80/s_{\text{mul}} = 0$，只有 $s_{\text{mul}} \to \infty$ 才趨近，因此有限加速不可能達到 5 倍。若只要求 2 倍，$T_{\text{new}} = 50$ s：

$$50 = \frac{80}{s_{\text{mul}}} + 20 \Rightarrow s_{\text{mul}} = 80/30 = 2.667.$$

這就是 Amdahl 的實作意義：先量「受改善部分占 baseline elapsed time 多少」，再問局部加速能否覆蓋其餘固定成本；不能把 operation count 或 source-line 比例直接當時間比例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 製造與 AMD Opteron 案例

製造流程包含 wafer fabrication、測試、封裝與 system integration。Yield learning 與 design-for-test 會影響可用晶片數。效能 benchmark 只描述已配置的機器，不能單獨代表製造成本。

本章案例 AMD Opteron X4 是 2000 年代末的案例。閱讀時要保留年代，不能把它與今天的 core、cache、memory bandwidth 直接比較。

技術案例的用途是理解設計取捨。設計細節應回到當時的 ISA、製程、workload 與 server configuration。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## SPEC CPU 基準

SPEC CPU 的指標包含 single-thread 與 throughput 類。Single-thread 指標描述一個程式執行速度，throughput 指標描述指定並行配置下完成的 work。

Reference score 不是秒數。它是相對於 reference machine 的標準化結果，所以不能把兩個 ratio 直接相加成「總效能」。

最佳實務是保留每個 benchmark 的 ratio、硬體配置、compiler flags 與輸入。幾何平均可作摘要，但不取代個別結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## SPEC power 基準

Power benchmark 把 performance 與 active-idle energy 一起衡量。常見思維是指定工作負載下，每秒工作的總能源成本。

尖峰效能與 idle power 都會影響資料中心。若伺服器大部分時間低負載，idle 與 power-state transition 的成本會顯著。

Energy 與 power 要分清楚。$E = \int P(t)\,dt$。Power 的單位是 watt，energy 的單位是 joule。完成相同工作時比較 energy 才有意義。

不要用瓦數直接除以秒。先確認資料是平均功率還是累積能量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 1.8 謬誤與陷阱

第一個謬誤是局部加速比例等於整體加速比例。若該部分只占 baseline 的 10%，即使無限加速，總時間最多只降低 10%。

第二個陷阱是不管 workload。處理器在某個 benchmark 強，不代表在 database、compiler 或數值程式也強。

第三個陷阱是忽略利用率與成本。購買更多資源若不能提高完成工作數，可能只增加 energy 與管理成本。

三個可核對反例分別是：baseline 100 ms 中10 ms加速無限倍，仍需90 ms，speedup上限100/90；系統X在R1花1 ms、R2花10 ms，Y分別花2 ms、5 ms，所以優勝者隨workload改變；若單一link最多100 requests/s，增加CPU而不改link不能把持續完成率推過100。三例分別違反「局部等於整體」、「單一benchmark代表所有負載」及「資源數等於可交付吞吐」的假設。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 抽象與介面小結

抽象將複雜度分層，ISA 是最重要的硬軟體介面之一。效能來自 workload、ISA、compiler、microarchitecture、memory 與 I/O 的共同結果。

本章的四個檢查問題是：

1. 比較的是 latency 還是 throughput？
2. 指令數、CPI、frequency 哪一項改變？
3. 哪些資源是共享的，是否已飽和？
4. benchmark 與系統配置能否重現？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 歷史視角

電晶體整合提升可放在晶片上的功能，也讓設計從手工邏輯轉向 HDL 與合成。Pipeline、cache 與 out-of-order execution 逐步提高單核 ILP，但 power 與 complexity 設下限制。

2000 年代後，多核、hardware multithreading、SIMD 與 GPU 把平行度帶入主流。Compiler、runtime 與作業系統必須共同利用這些資源。

歷史比較要固定 ISA 與 workload，否則容易把世代、實作與應用變化混在一起。

電晶體與架構提供可用資源；只有 workload 的 critical path、指令工作量或資料搬移成本實際下降，才形成可量到的 application speedup。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 本章重點

- ISA 規定軟硬體介面；不同微架構可有相同可觀察結果與不同時間。
- CPU 時間以動態指令數、平均 CPI 與同區間有效頻率共同決定。
- 比較效能必須明定基準、單位與工作負載；延遲與吞吐量分開量。
- Amdahl 定律以基準時間比例計算，固定部分決定整體加速上限。

### 參考資料與署名

Lin Cing，《計算機組織》手寫筆記。 Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 1. CSO course slides, 2026, Chapter 1.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 從程式語意到 MIPS 指令

Compiler 不必直接知道電晶體如何切換，它只需產生符合 ISA 契約的指令。Microarchitecture 不必理解 `if` 或函式，它只需正確完成 commands、位址計算、例外與記憶體效果。

本章用 MIPS32 建立完整的軟硬體介面，之後再以 ARM 與 x86 對照。學習重點是可追蹤的資料流與位址，不是背誦所有指令。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 學習路線與必要符號

本章使用下列符號。

- $R[x]$ 表示 index 為 $x$ 的 32-bit register。
- $M[a]$ 表示 byte-addressable memory 中由位址 $a$ 開始的資料。
- $PC$ 是目前討論指令的位址；定址公式中的 $PC_{\text{next}} = PC + 4$。
- $I[k:j]$ 是 instruction $I$ 的第 $k$ 至第 $j$ 個 bit。

本章採傳統 pre-Release-6 MIPS32，保留一個 branch/jump delay slot；所有控制轉移範例以 `nop` 明示延遲槽。`jal` 寫入 `$ra=PC+8`，不是 `PC+4`。本章不模擬早期 R2000 的 load delay。所有 MIPS core instructions 在案例中長度固定為 32 bits。Alignment 假設是 instruction address 為 4-byte aligned，word load/store 位址也為 4-byte aligned。

若實際 assembler 會重排指令，須使用其對應的順序控制（例如支援時使用 `.set noreorder`）並檢查 pseudo-instruction 展開，才能保證本文的 delay-slot 追蹤。

先固定 alignment 與 endianness 假設，否則同樣位址會得到不同 byte interpretation。

## 2.1 ISA 是軟硬體契約

ISA 描述 programmer-visible state 與 operations。MIPS32 的可見狀態包括 32 個 general-purpose registers、program counter、floating-point registers、HI/LO、status 與 exception-related registers。

程式可依 instruction semantics 依賴某些結果，但不能依賴 pipeline 有幾階或 cache 放哪裡。Timing 可以由 microarchitecture 改變，architectural result 必須維持。

ISA 也定義哪些 situations 必須產生 exception，例如 arithmetic overflow 或 address fault。精確例外的條件由 architecture 與特權規格確定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 2.2 MIPS 運算指令

Arithmetic instruction 採三-operand形式。例如

```
add  $t0, $t1, $t2
```

語意是

$$R[t0] \leftarrow R[t1] + R[t2].$$

`sub` 做二補數減法。`add` 與 `addi` 在 signed overflow 時依規格產生 exception，`addu` 與 `addiu` 則不因 signed overflow 產生同樣例外。選擇哪個 instruction 會改變 architecturally visible trap。

`addiu` 的名稱不代表只有 unsigned operands。它是 immediate arithmetic without signed-overflow trap，負 immediate 仍經 sign extension。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 指令與語言的表達力

ISA 的核心運算數量可以很少，compiler 用組合完成高階語言 semantics。乘法可由 iteration、shift-and-add 或乘法單元實作，但 ISA 是否提供 `mult` 會影響 instruction count。

固定長度 encoding 使 decode 位置穩定，卻限制單一 instruction 能容納的 immediate bits。複雜 instruction 可減少 instruction count，也可能拉長 critical path。

比較 ISA 時要同時看 instruction count、每 instruction work、encoding 複雜度與 compiler 能否有效使用。

不要用 RISC 或 CISC 標籤直接預測效能。第 4 章會看到 microarchitecture 可以縮小兩者差異。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 2.3 運算元與暫存器

MIPS32 有 32 個 32-bit general-purpose registers。C 的 scalar 變數通常先放入 registers，load/store instructions 才處理 memory。

這形成 load-store architecture。Arithmetic 指令的 operands 是 registers 或 encoded immediate，不直接指定兩個 memory locations。

Register file 的讀取埠、write port 與 bypass 路徑屬 microarchitecture。ISA 只保證寫入後的 architectural value，不保證在 pipeline 哪一 cycle 可見。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 記憶體運算元

典型 MIPS load/store 的 effective address 是

$$EA = GPR[base] + sign_extend(imm16).$$

`lw $t0, 12($s0)` 把 $M[EA]$ 起算的 32-bit word 載入 register。`sw` 則把 register 值寫到 memory。

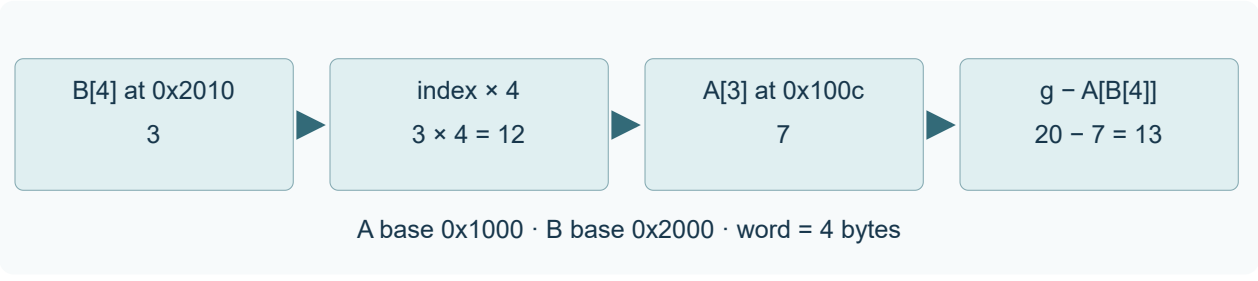
Immediate 是有號 16-bit displacement，範圍 $-32768$ 到 $32767$ bytes。大型 offset 或不規則位址需先計算 base register。

若 base 是 32-bit，加法在 modulo $2^{32}$ 位址空間中發生。Instruction memory alignment 另行限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 間接索引：逐步追蹤兩次讀取




假設兩個陣列的 word 均為 4 bytes，base $A = 0x1000$、$B = 0x2000$，$B[4] = 3$、$A[3] = 7$、$g = 20$。所有索引、位址與讀取均有效且對齊；範例不發生算術例外。

```
lw   $t0, 16($s7)   # B[4] = 3
sll  $t0, $t0, 2    # 3 × 4 = 12 bytes
addu $t0, $t0, $s6  # address A[3] = 0x100c
lw   $s0, 0($t0)    # A[3] = 7
sub  $s0, $s1, $s0  # f = 20 − 7 = 13
```

其中 `$s6=0x1000`、`$s7=0x2000`、`$s1=20`；$B[4]$ 的 16 是 byte offset，不是元素索引本身。

Cing

## 立即值

`addi $t0, $t0, -7` 把 encoded 16-bit 欄位 sign-extend 後加到 register。`lui` 把 16-bit immediate 放入 upper half，常用兩條 instruction 合成 32-bit constant。

對 $0x12345678$，可概念化為 `lui $t0, 0x1234` 後再接 `ori $t0, $t0, 0x5678`。`lui` 不用 sign extension 的方式形成常數，而是把 lower 16 bits 清零。

ISA 是否提供 single-instruction 32-bit literal 是 encoding 取捨。固定 32-bit instruction 不可能同時保留五個 register fields 與任意 32-bit immediate。

`lui` 加 `ori` 組合立即值；load 則讀取當時的記憶體，可能遭遇位址、權限或頁面例外。只有常數內容固定、位址有效且沒有額外可觀察副作用時，兩者的載入值才可相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 2.4 整數表示與溢位

Unsigned $n$-bit integer 的範圍是 $0$ 到 $2^n - 1$。Two's complement 將最高位權重設為 $-2^{n-1}$，範圍是

$$-2^{n-1} \le x \le 2^{n-1} - 1.$$

對 32 bits，範圍是 $-2147483648$ 到 $2147483647$。負數的 bit pattern 可由 complement 加 1 得到。

Signed operation 與 unsigned operation 可使用相同加法器，差別在 overflow interpretation、comparison 與乘除法處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 溢位與比較

Signed addition overflow 不是 carry-out 的別名。兩個同號 operands 相加得到異號結果時，結果越界。

溢位判斷可用 sign bits。令 operands 的 sign 為 $s_a, s_b$，result 為 $s_r$：

$$overflow = \overline{s_a \oplus s_b} \wedge (s_a \oplus s_r).$$

Unsigned addition 的 overflow 看 carry-out。比較也有 signed 與 unsigned 版本，所以 MIPS 提供 `slt` 與 `sltu`。

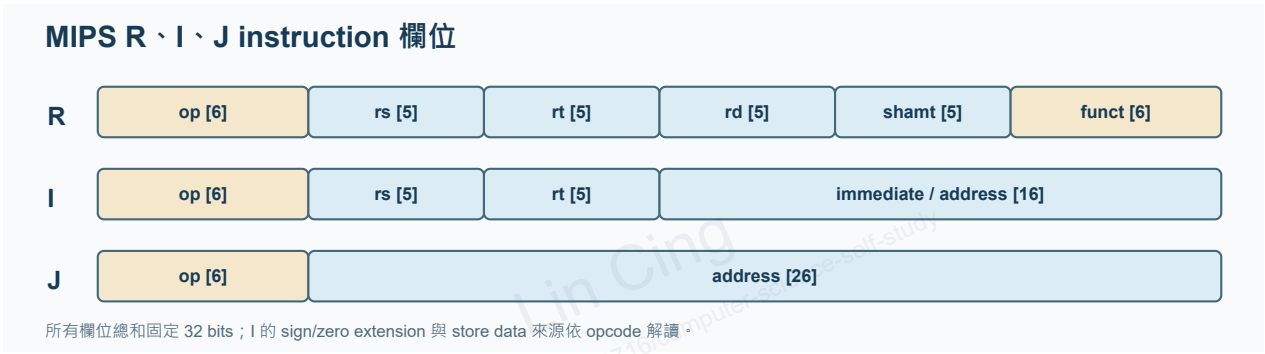
`0xffffffff + 1` 在 unsigned 是 overflow，在 two's complement 是 0 且無 signed overflow。必須先指定 interpretation。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 2.5 MIPS 指令格式

MIPS core instruction 一律 32 bits。R-type 用 opcode 000000 加 `funct` 區分 ALU operation；I-type 用 opcode 選擇 operation 與 16-bit immediate；算術及位址 displacement 做 sign extension，邏輯立即值做 zero extension；J-type 提供 26-bit jump index。




固定欄位位置使 fetch 後可並行讀取 opcode、registers 與 immediate。Encoding 的精簡會把更多 decision 推到 compiler 與多 instruction sequence。

欄位圖是概念示意，不代表實際晶片佈局。Bit order 從 MSB 31 往 LSB 0。

## R、I、J 三種格式

R-type 的 MSB 到 LSB 欄位是 `op[31:26]`、`rs[25:21]`、`rt[20:16]`、`rd[15:11]`、`shamt[10:6]`、`funct[5:0]`。

I-type 是 `op`、`rs`、`rt`、`imm16`。Branch 使用 immediate 計算 offset；load/store 使用 immediate 計算有效位址；store data 來自 `rt`，不是 immediate。

J-type 是 `op` 與 `target26`。它不能任意指定 32-bit absolute address，因此要和程式所在 region 的 upper PC bits 合併。

I-type 的 `rt` 在 `addi` 常作 destination，在 store 作 source。欄位名稱相同，角色由 opcode 決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 2.6 邏輯與移位

Bitwise operations 是 `and`、`or`、`nor` 與 `xor`。Immediate forms 用 zero-extended 16-bit immediate，因為遮罩通常是 bit pattern。

Logical shift 補 zero。Arithmetic right shift 補 sign bit。MIPS 以 `srl`、`sra`、`sll` 操作 registers，shift amount 可為 `shamt`。

例如要取出 word 的 bits 11 到 8：

```
srl $t1, $t0, 8
andi $t1, $t1, 0x000f
```

`and` 常用於清零不需要的 bit；`or` 用於設定 bit；`xor` 可切換 bit。Mask 必須配合 shift amount。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 2.7 條件分支與迴圈

`beq rs, rt, label` 在 equality 成立時改變 control flow。若不成立，architecture-visible next PC 是 sequential instruction。

本章的傳統 MIPS32 模型有一個 branch delay slot，緊接 branch 的指令無論分支是否成立都會執行。例子以 `nop` 避免把不安全操作放入延遲槽；Release 6 的無延遲槽控制轉移不在本章模型內。

Branch 型態通常包括 equal、not equal、less-than 等比較。組合語言 pseudo-instruction 可由 branch 與 `slt` 展開。

必須宣告使用 classic MIPS 還是 MIPS32 release 的 ISA。不同版本對 branch delay、pseudo-instructions 與 instruction availability 可能不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 將迴圈翻成指令

C 片段：

```
int sum = 0;
for (int i = 0; i < n; ++i)
    sum += a[i];
```

令 `$a0` 指向 `a`、`$a1` 為 `n`、`$v0` 為 sum：

```
move $v0, $zero
move $t0, $zero
beq  $t0, $a1, done
nop
loop:
sll  $t1, $t0, 2
addu $t1, $t1, $a0
lw   $t2, 0($t1)
addu $v0, $v0, $t2
addiu $t0, $t0, 1
bne  $t0, $a1, loop
nop
done:
```

## 將迴圈翻成指令 （續）

前提是signed `n >= 0`、array位址有效且不wrap，並限定signed C sum不溢位，才與上面C語意等價；MIPS `addu`本身仍採modulo 2^32。`sll` 將 index 乘 4，因為 word 是 4 bytes。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Switch 的策略

密而集中的 case labels 可用 jump table。先檢查 index 是否在 table 範圍，再以 `sll` 與 `lw` 取得 target address，最後 `jr`。

稀疏或數目很少的 labels 可編成 compare-and-branch chain。Jump table 常降低決策鏈長度，但增加 table 空間、indirect jump 與 cache footprint。

編譯器還可能反向排列 cases、用 binary search tree 或 range checks。策略依 label distribution、安全性與 microarchitecture 選擇。

Direct branch的target由PC與immediate固定；jump table的同一個indirect-jump PC可隨index指向多個targets，需預測的不只有taken方向。Table若不在cache，還會先等target-address load；因此決策鏈較短不保證elapsed time較短。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 從 bit field 到 control flow 的完整追蹤

解碼不能只查 mnemonic。以下三個 word 分別覆蓋 I、R 與 J 格式的關鍵差異。

| 32-bit word | 欄位拆解 | 執行語意 |
|---|---|---|
| `0x8fa8000c` | `op=0x23`、`rs=29`、`rt=8`、`imm16=12` | `lw $t0, 12($sp)`；`EA=R[sp]+12` |
| `0x012a4020` | `op=0`、`rs=9`、`rt=10`、`rd=8`、`shamt=0`、`funct=0x20` | `add $t0,$t1,$t2` |
| `0x0c100140` | `op=3`、`target26=0x100140` | `jal` 至同一個 256-MiB region 的 `0x00400500` |

I-format 的 `imm16` 並非一律 signed。`lw` 與 `addi/addiu` 做 sign extension；`andi/ori/xori` 做 zero extension；store 的 `rt` 是資料來源。opcode 先決定格式與欄位角色，不能先假定 `rt` 一定是 destination。

ISA 結果與高階語言語意也要分開。MIPS `addu/addiu` 的 32-bit result 是 modulo $2^{32}$；C 的 unsigned arithmetic 也是 modulo $2^N$，但 signed overflow 在 C 標準中屬於 undefined behavior。Compiler 不能只因兩個 operands 是 signed 就把任意越界程式翻譯成有保證的 modulo 結果；原始程式需先證明不越界，或改用 unsigned／更寬型別。

令 `beq` word 是 `0x1109fffc`，且 instruction 位於 `0x00400020`。`imm16=0xfffc=-4`，因此

$$PC_{\text{next}} = 0x00400020 + 4 = 0x00400024,$$

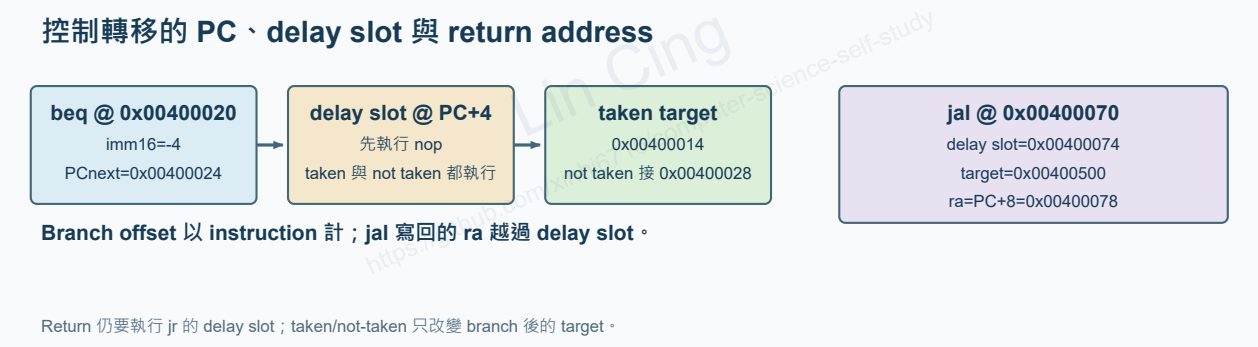


# 從 bit field 到 control flow 的完整追蹤 （續）

$$target = PC_{\text{next}} + (\text{sign_extend}(0xfffc) \ll 2) = 0x00400024 - 16 = 0x00400014.$$

在 classic MIPS32 的一個 delay slot 模型下，branch taken 的 execution sequence 是 `0x00400020 → 0x00400024 → 0x00400014`；not taken 是 `0x00400020 → 0x00400024 → 0x00400028`。`0x00400024` 無論條件為何都執行，因此不能把它誤當成只在 taken 時執行。

對 `jal` 位於 `0x00400070` 的例子，delay slot 在 `0x00400074`，target 是 `0x00400500`，`$ra=0x00400070+8=0x00400078`。Return sequence `jr $ra; nop` 會先執行 return 的 delay slot，才到 `0x00400078`。




三個必要邊界是 instruction address 必須 word aligned、branch offset 以 instruction 為單位、J-type target 只能沿用 PC+4 的高四位。任一把 byte offset、word offset 或目前 PC 的高四位混用，換一組輸入就會得到錯的 target。

## 2.8 程序呼叫契約

`jal target` 把 return address 放入 `$ra`，再跳到 target。本章傳統 MIPS32 明定 `$ra = PC+8`，返回位置越過 call 的一個 delay-slot instruction。

`jr $ra` 跳回 caller。被呼叫程序若要呼叫其他程序，必須先保存 caller 的 `$ra`，否則 nested call 會覆寫。

Calling convention 是 ABI 的一部分，不是 processor 硬體自動完成的全部工作。Register allocation、stack layout 與保存順序由規格約定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 暫存器分工

教材案例中的 MIPS convention 通常是：

- `$zero` 固定為 0。
- `$a0` 至 `$a3` 傳 arguments。
- `$v0`、`$v1` 傳 return values。
- `$t0` 至 `$t9` 由 caller 保存。
- `$s0` 至 `$s7` 由 callee 保存。
- `$sp` 是 stack pointer，`$ra` 是 return address。

Caller-saved 的意思是 caller 若要在 call 後繼續依賴該 register，就要自己保存。Callee-saved 則表示 callee 有義務在 return 前恢復原值。

這些名稱是 convention 與 ABI 的責任，硬體不會依 `$t` 或 `$s` 名稱自動保存 register。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 巢狀呼叫與 $ra

假設 `main` 呼叫 `f`，`f` 再呼叫 `g`。`jal g` 會覆寫 `f` return 到 `main` 所需的 `$ra`。

因此 `f` 在第一次 call 之前保存 `$ra` 到自己的 frame，等 `g` 返回後再 restore。

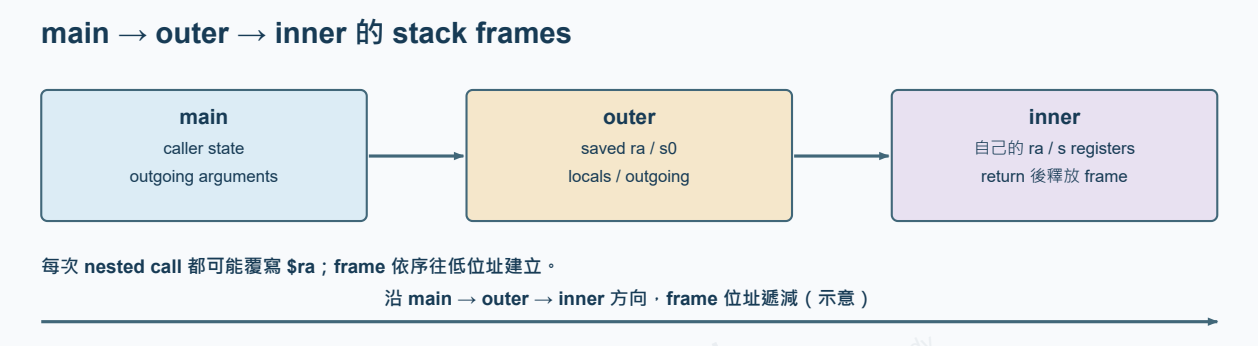
以下8-byte frame只示範return-address保存，不宣稱完整o32 outgoing-argument配置。最小sequence可概念化為：

```
addiu $sp, $sp, -8
sw    $ra, 4($sp)
jal   g
nop
lw    $ra, 4($sp)
addiu $sp, $sp, 8
jr    $ra
nop
```

Frame size 保持 alignment 是 ABI 要求，本例以 8 bytes 為例。實際 frame 還要保存 arguments、locals 與 callee-saved registers。



## 堆疊框




Call frame 通常包含 saved `$ra`、saved `$s` registers、local arrays、spill registers 與 outgoing argument area。Frame size必須容納同時存活的slots，offset需落在frame內且滿足各slot alignment，不能以offset大小代替所需空間。

Stack pointer 在 entry 減去 frame size，在 exit 前恢復。每個 stack slot 的 alignment 與 argument area 由 ABI 決定。

若 pointer 指向已釋放 frame，後面 call 可能覆寫它。Lifetime 錯誤是語意問題，編譯器未必能自動修正。

## Heap 記憶體

Stack 隨 procedure lifetime 成長與縮回。Heap 用 `malloc` 或 `new` 配置，lifetime 由 program 控制，直到 `free` 或 garbage collection。

Allocator 管理 free blocks、metadata 與 fragmentation。Allocation 可能失敗，也可能觸發 page fault 與作業系統擴張 virtual address space。

Heap 的 logical allocation 與實際 physical residency 不同。成功取得 virtual memory 不代表每頁已載入 DRAM。

若函式回傳指向local int的pointer，return後該物件生命期已結束，後續dereference不是合法存取；stack bytes暫未覆寫也不延長其生命期。Heap leak則是在配置後失去最後一個可釋放的pointer，例如反覆malloc再覆寫唯一pointer；物件仍占配置空間，與dangling pointer的「存取已失效物件」不同。



## 2.9 字元、字串與位元組

ASCII 定義 7-bit 碼位，通常存於一個 8-bit byte；Unicode 則透過 UTF-8、UTF-16 等編碼表示。C 字串以終止 byte 表示長度；其他語言可能同時保存 length metadata。

MIPS 以 `lb`、`lbu`、`sb` 存取 byte。`lb` 做 sign extension，`lbu` 做 zero extension。String copy 要維持 destination capacity、alignment policy 與終止符。

Byte order 會影響 multi-byte word 在 memory 中的排列。Endianness 必須在 data exchange 與 binary format 邊界明確。

同一組 bytes `41 42 43 00` 作 NUL-terminated ASCII string 是 "ABC"；作32-bit integer，big-endian 是 `0x41424300`，little-endian 是 `0x00434241`。讀到同樣 bits 並未決定其型別語意。

## 2.10 大型立即值

Fix-width instruction 的 immediate field 有限。要把 arbitrary 32-bit constant 放入 register，需要多 instruction。

`lui rt, imm16` 將 `imm16` 放到 bits 31 至 16，low half 清零。接著 `ori rt, rt, imm16` 填 low half。

若 compiler 能在 symbol relocation 時直接填入 address，`lui` 與 `addiu` 也可能形成 address sequence。Effective address 最終仍是 32-bit modulo address space。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 分支與跳躍定址

PC-relative branch 的 target 為

$$target = (PC_{\text{next}}) + (sign_extend(imm16) \ll 2).$$

Offset 的單位是 instructions，不是 bytes。16-bit signed offset 可表示的 byte 範圍約為 $\pm 2^{17}$，且 instruction target 必須 4-byte aligned。

J-type 的 target 為

$$target = (PC_{\text{next}}[31:28] \parallel target26 \parallel 00_2).$$

它保留 PC+4 的高四位，所以目標限於 PC+4 所屬的 256-MiB region；當指令位於 region 末尾時，不應直接改用目前 PC 的高四位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 定址模式總覽

教材包含 register、immediate、base-plus-displacement 與 PC-relative addressing。`lw $t0, 0($sp)` 使用執行該指令時的 architectural base value 加 displacement；資料相依由實作的前送或停頓處理，branch delay slot 並不是保存 base 的機制。

Addressing mode 不是越多越好。每增加一個模式，就增加 decode、exception priority 與編碼複雜度。

Pseudo-direct jump 用小 target field 換取固定 encoding，代價是 region 限制。Compiler 與 linker 若把 target 放錯 region，可能需要 trampoline 或 indirect jump。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 解碼機器碼

以 `0x012a4020` 為例，依序拆欄位：

```
op       = 000000
rs       = 01001  ($t1)
rt       = 01010  ($t2)
rd       = 01000  ($t0)
shamt    = 00000
funct    = 100000 (add)
```

所以它是 `add $t0, $t1, $t2`。decoder 先看 opcode，再依 format 選 `funct` 或 immediate。

解碼必須先確認 ISA、register naming 與 instruction length。若把 x86 variable-length bytes 當成 MIPS 固定欄位，會得到沒有意義的結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 2.11 同步與原子操作

多個 cores 共享 memory 時，單純 `lw`、`addiu`、`sw` 會形成 read-modify-write race。兩個 cores 可能都讀到舊值，再各自寫入新值。

MIPS 的 `ll` 與 `sc` 提供 atomic pair。`ll` 讀取並建立 reservation；`sc rt, offset(base)` 以 `rt` 的舊值作 store data，成功則原子寫入並把 `rt` 改為 1，失敗則不寫入並把 `rt` 改為 0。

失敗時常見策略是 retry loop：

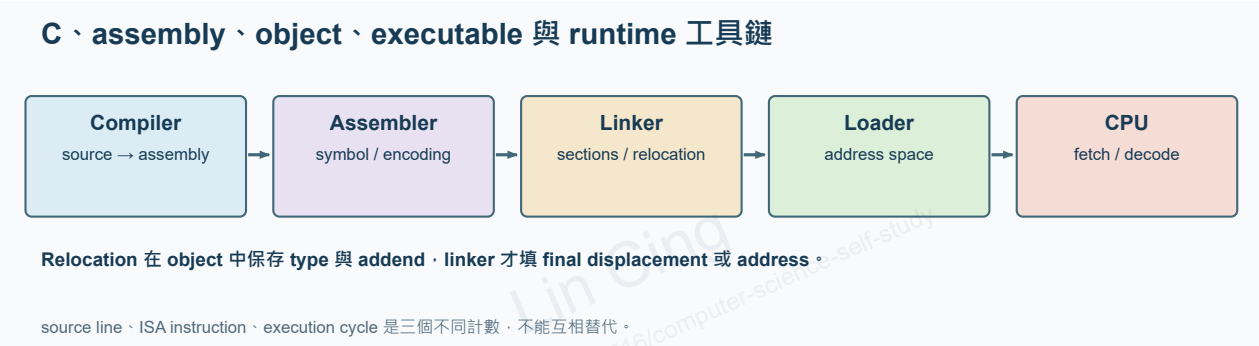
```
loop:
ll   $t0, 0($s0)
addiu $t0, $t0, 1
sc   $t0, 0($s0)
beq  $t0, $zero, loop
nop
```

具體 retry 次數與 fairness 不是 ISA 保證。若 reservation 被另一個 store 或例外中斷，`sc` 可能失敗。



## 2.12 從 C 到執行檔

Compiler 做型別檢查、最佳化與 instruction selection。Assembler 把 symbols 與 pseudo-instructions 轉成 machine instructions 與 relocation records。Linker 合併 object files、解析 external references、配置 sections。Loader 建立 process address space 與初始 registers。




每個階段都可能失敗。Compiler 的 undefined behavior、assembler 的 invalid operand、linker 的 undefined symbol、loader 的 permission 或 mapping 問題必須分開診斷。

以外部 `printf` 為例，object 留下 undefined symbol 與 call relocation；靜態連結器產生可執行檔；選擇動態連結時，可依 ABI 建立 stub、間接表或重定位，再由 dynamic loader 映入提供者並解析 runtime address。完全靜態連結則不需要動態載入器解析該符號。若採 lazy binding，首次經 resolver 更新表項，後續走已解析的間接路徑。實際 stub 與暫存器規則依 ABI，不能把此跨階段流程當作固定 MIPS 指令序列。

## 工具鏈的地址語意

Assembler 第一次看到 local label 時可計算 section-relative address，但 external symbol 尚未確定。Object file 保存 relocation type 與 addend，讓 linker 填入最終 displacement 或 address。

Static linking 在 executable 內解析 references。Dynamic shared library 可能用 position-independent code 與 runtime resolver，降低 multi-process physical memory duplication。

Debug 時要分清楚 virtual address、file offset、load segment offset 與 physical page。它們不是同一個數字。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 呼叫慣例案例

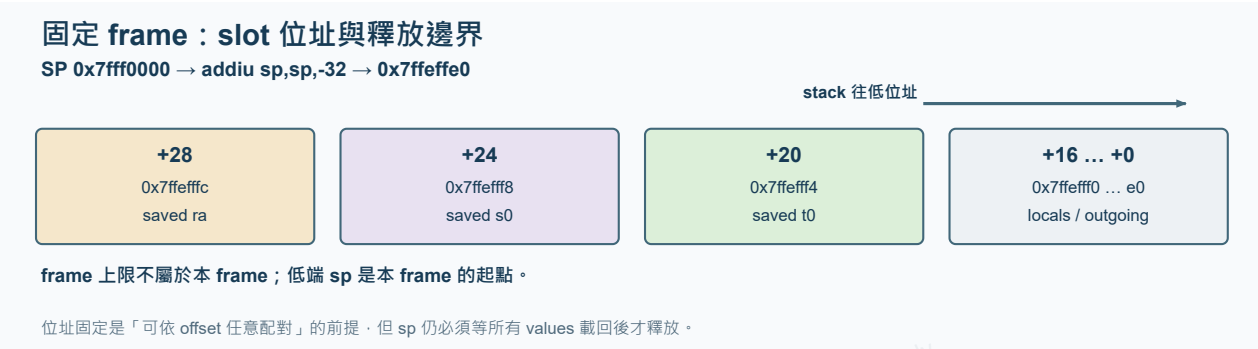
```
int twice_sum(int x, int y) {
    int s = x + y;
    return s * 2;
}
```

假設 signed 加法與乘 2 的 mathematical result 都可表示，概念序列如下：

```
twice_sum:
addu $v0, $a0, $a1
sll  $v0, $v0, 1
jr   $ra
nop
```

這個 leaf function 沒有 call，不必保存 `$ra`；它只寫 `$v0`，也不破壞 caller 依賴的 `$s` registers。

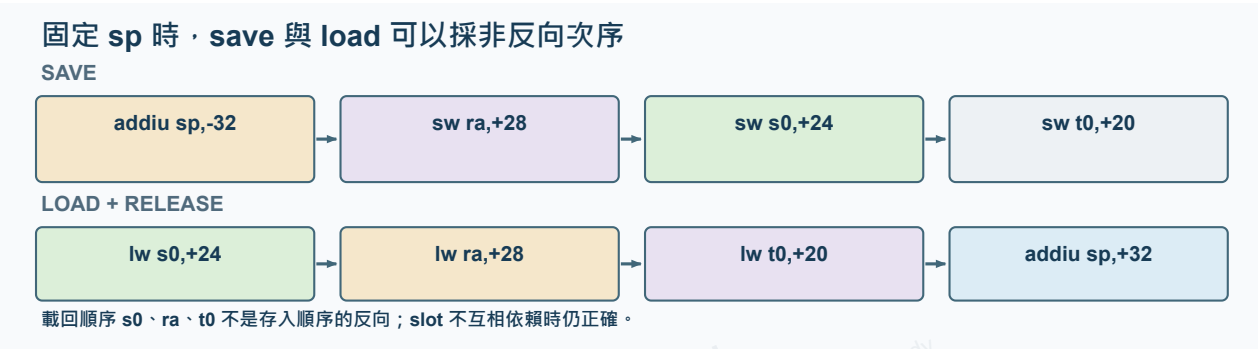
# 固定 frame 的 slot 位址




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 固定 frame 的 save、load 與 release 時序




令進入函式前 `sp=0x7fff0000`，frame 為 32 bytes。`addiu sp,sp,-32` 後 `sp=0x7ffeffe0`，三個獨立 slot 的位址固定如下。

| Slot | 位址 | 存入 | 載回 |
|---|---|---|---|
| `28(sp)` | `0x7ffefffc` | `ra` | `ra` |
| `24(sp)` | `0x7ffefff8` | `s0` | `s0` |
| `20(sp)` | `0x7ffefff4` | `t0` | `t0` |

一個有效序列是依序存 `ra`、`s0`、`t0`，再依序載回 `s0`、`ra`、`t0`，最後才 `addiu sp,sp,32`。載回次序並非存入次序的反向，結果仍正確，因為 `sp` 在載回期間不變，三個 offset 指向不同 slots，且 load destinations 不互相依賴。

## 固定 frame 的 save、load 與 release 時序 （續）

「save 與 restore 必須反向」只適用於 push/pop 會改變 `sp` 的 LIFO 序列，或 slot/register 之間存在依賴。固定 frame 的釋放時機則是硬性條件：所有需要的 values 都載回後，才可增加 `sp`；先用新的 `sp` 讀取舊 frame 會錯位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 遞迴呼叫的 frame trace

令 `main` 在 `0x00400010` 執行 `jal f`，`f` 的回傳位址是 `0x00400018`。`f` 進入時先令 `$sp=0x7fff0000`，配置 32-byte frame 後 `$sp=0x7ffeffe0`：

| 寫入 | 位址 | 值 |
|---|---|---|
| `sw $ra,28($sp)` | `0x7ffefffc` | `0x00400018` |
| `sw $s0,24($sp)` | `0x7ffefff8` | caller 的 `$s0` |

`f` 在 `0x00400120` 再執行 `jal f`，新的 `$ra` 是 `0x00400128`。inner activation 令 `$sp=0x7ffeffc0`，並把 `0x00400128` 存到 `0x7ffeffdc`。此時兩個 frames 同時存活，任何 `$sp+28` 都指向目前 activation 的 slot，不能拿 inner 的 `$sp` 讀 outer frame。

Base case 返回時，inner 依序載回自己的 `$ra` 與 `$s0`，把 `$sp` 加回 `0x7ffeffe0`，再 `jr` 到 `0x00400128`。Outer activation 繼續執行，最後從 `0x7ffefffc` 載回 `0x00400018`、釋放 32 bytes，再回到 `main`。每個 `jal` 與 `jr` 後都保留 `nop` 作為本章模型的 delay slot；移除它會改變實際執行的 instruction。

這個 trace 同時驗證三件事。第一，`$ra` 必須在 nested call 前保存；第二，frame 的 slot 位址由目前 `$sp` 與固定 offset 決定；第三，release 必須等所有需要的 loads 完成後才增加 `$sp`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 陣列與指標的指令差異

Array indexing 的位址通常是

$$address = base + i \times element_size.$$

Pointer traversal 每次增加 `element_size`，迴圈中少一次 index scaling，但 compiler 可能在 array 版本做相同 strength reduction。

指標本身可以是 64-bit，也可以指向 invalid 或 one-past-end 之外的位址。C/C++ 的 pointer arithmetic 有 language-level 邊界，超過有效物件範圍即使 machine 能形成 bit pattern，也不保證合法。

索引版本每次可包含 `sll index,2` 與 base addition；pointer版本把位址保存在暫存器、每次加4。下方 clear 範例將兩種穩態序列完整展開為7與5個指令位置；此差距包含 loop test 與 delay slot，但不含 prologue、exit 或 memory stalls。Compiler strength reduction 可把前者改成後者。



## C 排序的完整資料流與成本

以下用插入式相鄰交換排序，展示 C 控制流、陣列定址與程序呼叫：

```
for (int i = 0; i < n; i += 1)
  for (int j = i - 1; j >= 0 && v[j] > v[j + 1]; j -= 1)
    swap(v, j);
```

取 `v=[3,1,2]`、`n=3`。`i=0` 時 inner loop 不安裝；`i=1` 時 `j=0`，因 `v[0]=3>v[1]=1` 呼叫 `swap(v,0)`，陣列變成 `[1,3,2]`；`i=2` 時先比較 `v[1]=3>v[2]=2`，交換成 `[1,2,3]`，再比較 `v[0]=1≤v[1]=2` 而停止。此時 sorted invariant 成立。

以下把完整控制流寫成 classic MIPS32；每個 branch 與 `jal` 後的 `nop` 都是本章模型的 delay slot。 此處sort與所附leaf swap採精簡教學呼叫契約；24-byte frame不保留o32要求的16-byte outgoing argument home area，因此只保證與本例leaf callee搭配，不能直接與任意o32編譯的callee互換。若要求完整o32互通，需另為outgoing area保留0..15 offsets並移動saved slots。前置條件是 `n>=0`、`v` 指向 `n` 個 aligned words，且 index calculation 不讓有效位址越界。

## C 排序的完整資料流與成本 （續）

```
sort:
addiu $sp, $sp, -24
sw    $ra, 20($sp)
sw    $s3, 16($sp)
sw    $s2, 12($sp)
sw    $s1, 8($sp)
sw    $s0, 4($sp)
addu  $s2, $a0, $zero
addu  $s3, $a1, $zero
addu  $s0, $zero, $zero
for1:
slt   $t0, $s0, $s3
```

https://github.com/xinhi6716/com



## C 排序的完整資料流與成本 （續）

```
beq   $t0, $zero, exit1
nop
addiu $s1, $s0, -1
for2:
slti  $t0, $s1, 0
bne   $t0, $zero, exit2
nop
sll   $t1, $s1, 2
addu  $t2, $s2, $t1
lw    $t3, 0($t2)
lw    $t4, 4($t2)
slt   $t0, $t4, $t3
```

https://github.com/xinhi6716/com

## C 排序的完整資料流與成本 （續）

```
beq   $t0, $zero, exit2
nop
addu  $a0, $s2, $zero
addu  $a1, $s1, $zero
jal   swap
nop
addiu $s1, $s1, -1
j     for2
nop
exit2:
addiu $s0, $s0, 1
j     for1
```



## C 排序的完整資料流與成本 （續）

```
nop
exit1:
lw    $s0, 4($sp)
lw    $s1, 8($sp)
lw    $s2, 12($sp)
lw    $s3, 16($sp)
lw    $ra, 20($sp)
addiu $sp, $sp, 24
jr    $ra
nop
```

令 `v=0x10010000`、`k=1`。`swap` 先算 `k*4=4`，兩元素位址是 `0x10010004` 與 `0x10010008`；它載入兩個 word，再以一個暫存 register 完成 store。Leaf procedure 只使用 caller-saved temporaries，不改用 `$ra` 以外的 saved state，因此不必配置 frame，但 `jr` 在本章 classic MIPS 模型仍需要 delay slot：

## C 排序的完整資料流與成本 （續）

```
swap:
sll  $t1, $a1, 2
addu $t1, $t1, $a0
lw   $t0, 0($t1)
lw   $t2, 4($t1)
sw   $t2, 0($t1)
sw   $t0, 4($t1)
jr   $ra
nop
```

只用一個迴圈 iteration 的穩態成本比較 array index 與 pointer clear。Array 版是 `sll+addu+sw+addiu+slt+bne+nop` 共 7 個指令位置；pointer 版是 `sw+addiu+slt+bne+nop` 共 5 個。省下的 2 個是 index scaling 與 base addition，不包括 prologue、初始 guard 或 exit。現代 compiler 可用 strength reduction 與 induction-variable elimination 把 index 版轉成 pointer-like code，所以不能從原始碼寫法直接斷言效能。

這個成本模型要求兩版使用相同 word size、相同 memory traffic、沒有 page fault 或 trap、branch delay slot 固定存在，且比較範圍只含穩態迴圈。對 `n=100` 的指定模型，array body 是 `7×100=700` 個指令位置，optimized pointer body 是 `5×100=500` 個；兩者的初始化與最後 exit check 另計。



## 編譯與解譯路徑

C compiler 通常先產生 target-specific assembly。Java bytecode 不直接等於 MIPS machine code，JVM 依平台實作可 interpretation、JIT compile 或二者混合。

Interpretation 每次執行 bytecode 都付出 dispatch overhead。JIT 累積 profile 後生成 native code，改善 hot path；warm-up 與 code generation 會增加 startup latency。

ISA 相似不代表語意相同。Integer overflow、unaligned access、atomic operations 與 memory model 都可能不同。

比較 C 與 Java 時要把 type semantics、runtime 與 engine 分開，不能只看一道算術 instruction。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## ARM 比較

ARM 是 load/store RISC 家族，透過 conditional execution、operand shifts 與多種 addressing 設計減少 branches。不同 ARM architecture version 的功能差異很大，不能把一個版本的 instruction 套用到全部 ARM 裝置。

與 MIPS 相同的是 registers、load/store 邊界與 procedure convention 的重要性。不同之處在 encoding、conditional flags 與 ISA extensions。

本章只讀教材版本的 ARM 案例，不把它當成所有現代 ARM64 的規格。

指出 chapter 的 ARM 例子接近 32-bit ARM 時代，與 AArch64 不可混用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## x86 比較

x86 採 variable-length instructions，encoding 可包含 prefixes、ModR/M、SIB 與 displacement。相同 instruction 可由一至多 bytes 表示。

有利於 code density 與 backward compatibility，代價是 fetch/decode alignment 複雜、instruction boundary 不一定可在固定位置找到。

Intel 與 AMD 實作會把 x86 instructions 解成 internal micro-operations，再用 superscalar pipeline 執行。ISA instruction 數不等於 microarchitecture 的 µops 數。

解釋為什麼 micro-op cache 可隱藏 variable-length decode 的成本，但不能讓 ISA 變成 MIPS。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## x86-64 的 LEA、struct layout 與可變指令長度

採 System V AMD64 ABI 的預設對齊、沒有 packing，且存取的 row 存在、$0 \le j < 4$。考慮 `struct row { int64_t id; int32_t vals[4]; int64_t sum; };`。`id` 在 offset 0，`vals` 在 offset 8、佔 16 bytes，`sum` 在 offset 24，整個 struct 對齊 8 bytes，因此 `sizeof(struct row)=32`。

對 `rows[i].vals[j]`，byte address 是

$$address = base + 32i + 8 + 4j.$$

若 `base=0x3000`、`i=5`、`j=3`，地址是 `0x3000+160+8+12=0x30b4`。假設 `i` 已擴成64-bit並放在 `%rdi`、有號 `int j` 在 `%esi`，而 `base` 在 `%rdx`，可先 sign-extend `j`、把 `i` 乘 32，再使用 SIB 的 scale 4：

```
movslq %esi, %rsi
salq   $5, %rdi
leaq   8(%rdi,%rsi,4), %rax
leaq   (%rdx,%rax), %rax
```

`lea` 只計算 address，不讀 memory；`movslq` 則把 32-bit signed index 擴成 64-bit address arithmetic。x86 instruction 是 variable length，branch displacement 以 byte 計，instruction boundary 也不固定在相同 modulo；這些規則與 fixed-width MIPS 不可互換。



## ISA 常見誤解

第一個誤解是「more powerful instruction 一定更快」。一個 instruction 可能只用 decoder 較少精力，但執行 latency 很長，或阻擋 parallel issue。

第二個誤解是「register 多，編譯器一定用得更好」。Register pressure、calling convention 與 spilling 仍決定實際效果。

第三個誤解是「pseudo-instruction 是硬體指令」。Assembler expansion 的 sequence 可能因 register 或 immediate 條件不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## ISA 設計小結

良好 ISA 要在 encoding、compiler、例外、virtual memory、同步與 backward compatibility 之間取捨。固定格式降低 decode 成本，但需要 multi-instruction constants；variable format 增加密度，代價是複雜 decode。

Program correctness 要求 architecture-visible result 一致。Performance 則依 instruction mix、CPI、microarchitecture 與 memory path。

本章所有 address 與 condition 都要交代位元寬、alignment 與 sign/zero extension。

本章lw/sw的base及displacement以bytes計；branch immediate則以4-byte instruction為單位。ADDIU會sign-extend而ORI會zero-extend；相同16-bit欄位必須先依opcode決定解釋。



## 指令集歷史

早期處理器為少量、固定欄位與簡單指令，便於實現與 compiler 產生程式。之後 CISC 加入複雜 addressing 與較高 code density；RISC 把操作規則化，將複雜工作交給 compiler。

現代處理器界線模糊。RISC instruction 可以 decode 成 µops，CISC instruction 也可以在內部拆成規則化操作。

歷史閱讀重點是設計限制，而不是把某個時代的口號當成永恆定理。

| 面向 | 機制與必須一併檢查的成本 |
| --- | --- |
| Code size | 密集encoding可減fetch bytes；複雜指令仍可能需多個內部operations |
| Decode | 固定欄位可平行讀取；變長encoding需先辨識邊界 |
| Memory traffic | Load/store數、寬度、locality與cache miss決定bytes，不能只看mnemonic |
| Parallel issue | 真資料相依及functional-unit可用性限制重疊；單條做更多工作未必更易並行 |
| Compatibility | 保留舊binary需維持ISA/ABI語意；內部decode與execution可重新實作 |

## 本章重點

- ISA 版本、delay slot 與 assembler 指令順序決定控制流程。
- 位址由 base 加 byte offset；載入前先檢查元素大小、範圍與對齊。
- signed overflow 會否 trap 取決於指令；ABI 規定保存義務與 layout。
- LL/SC 的成功、記憶體排序與公平性是不同保證。

### 參考資料與署名

Lin Cing，《計算機組織》手寫筆記。 Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 2. CSO course slides, Chapter 2.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

## 同一串位元，先問它代表什麼

硬體只收到有限長度的位元；**位元如何解讀、結果保留幾位、超出範圍如何處理**，才決定運算的意義。

| 問題 | 必須先固定 | 本章追蹤的結果 |
| --- | --- | --- |
| 整數加減 | 位寬、有號或無號、溢位政策 | 結果位元、進位與溢位 |
| 整數乘除 | 暫存器寬度、每輪操作、商的取整方式 | 逐輪狀態、最後乘積或商餘數 |
| 浮點運算 | 格式、有效精度、捨入模式、例外環境 | 編碼、捨入結果與旗標 |

例如 `11111111` 可代表無號的 255，也可代表八位元二補數的 −1。位元沒有改變，解讀不同；後續的比較、除法與高位擴充也就不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 三種結果規則：截低位、飽和、捨入

固定寬度無法表示所有整數與實數。先算出數學上的精確值，再看指定的格式如何保存它。

| 八位元輸入 100+100 | 保存的結果 | 解讀 |
| --- | --- | --- |
| 保留低八位 | `11001000` | 無號為 200；二補數為 −56 |
| 有號飽和 | `01111111` | 夾到有號上界 127 |
| 有號且檢查溢位 | 依指令處理例外 | 數學和 200 超出 [−128,127] |

浮點數則依捨入模式選可表示的候選值，不是把整個實數當成整數直接截低位。格式、運算規則與時序是三個問題：**得到正確結果，不代表只需一個週期。**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.1 先寫清楚運算模型

無號 n 位元數的範圍是 $[0, 2^n - 1]$；n 位元二補數的範圍是 $[-2^{n-1}, 2^{n-1} - 1]$。加減若只保留低 n 位，位元結果等於模 $2^n$ 運算；有號的數學結果是否可表示，仍要另外判斷。

有限浮點輸入的基本運算則先求精確值，再依目標格式與捨入模式映回可表示集合：

$$y = \mathrm{round}(x \circ z).$$

此式描述一般有限值路徑；NaN、無窮大、除以零與超出範圍要套用另外的規則。整數 `0/0` 的行為依指令或語言而定；IEEE 浮點 `0/0` 屬 invalid，停用 trap 時通常回傳 quiet NaN。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 3.2 二補數：相同加法器，不同範圍判斷

n 位元二補數中，最高位帶負權重：

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

加法器逐位相加的規則與無號加法相同。減法利用 $a - b = a + \bar{b} + 1$，把 b 的每位反相、最低位帶入進位 1，就能共用加法器。

例如四位元 `0101+0011=1000`。無號解讀是 5+3=8；二補數解讀時，5 和 3 均合法，但數學和 8 超出上界 7，`1000` 反而表示 −8。**低位結果正確與有號結果可表示，必須分開檢查。**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 有號溢位：為什麼要比較符號？

一正一負相加，和介於兩個輸入之間，因此不會超出有號範圍。同號相加才可能超界；超界後保留低 n 位，結果符號翻轉。令輸入與結果的符號位為 $s_a, s_b, s_r$，則

$$ov = \overline{s_a \oplus s_b} \wedge (s_a \oplus s_r).$$

減法的對應條件為 $ov = (s_a \oplus s_b) \wedge (s_a \oplus s_r)$：異號相減才可能超界，且結果符號與 a 相反。

八位元 `0x7f+1` 有有號溢位卻沒有最高位進位；`0xff+1` 有最高位進位卻沒有有號溢位。進位離開最高位判斷的是**無號加法是否超界**，不能直接當成有號溢位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 逐位核對：進入與離開符號位的進位

對八位元加法，令 $c_7$ 進入符號位、$c_8$ 離開符號位。有號溢位等價於 $ov = c_7 \oplus c_8$。

| 運算 | 低八位結果 | $c_7$ | $c_8$ | 有號溢位 |
|---|---|---|---|---|
| `0x7f+0x01` | `10000000` | 1 | 0 | 1 |
| `0x80+0xff` | `01111111` | 0 | 1 | 1 |
| `0x80-0x01` | `01111111` | 0 | 1 | 1 |
| `0xff+0x01` | `00000000` | 1 | 1 | 0 |

第三列實作為 `10000000+11111110+1`，最低位輸入進位為 1。符號位滿足 $s_r = s_a \oplus s_b \oplus c_7$，而其進位輸出為三輸入的多數決；代入同號與異號兩種情況，即得上述 XOR 判定。減法表中的 $c_8$ 是加法器輸出，**不是一律等於 borrow（借位）**。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 加法器的等待時間與指令例外是兩件事

漣波進位加法器（ripple-carry adder）的高位結果要等低位進位，最長路徑隨位寬成長。前瞻進位或平行前綴結構，用更多邏輯與連線縮短依賴鏈；實際時脈仍須容納最慢可成立路徑及暫存器成本。

在本章傳統 MIPS32 案例，`add` 檢查有號溢位；發生例外時不能把錯誤結果當成正常寫回。`addu` 則保留模 $2^{32}$ 的結果，不因有號超界而 trap。名稱中的 u 不表示加法器只能接收正數。

因此要分別回答：位元結果是什麼、指令允許它寫回嗎、硬體多久能產生結果。只看運算元的平均情況，不能證明最壞時序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 飽和與 SIMD：每個 lane 都有自己的邊界

飽和運算（saturating arithmetic）超界時夾到端點。例如有號八位元 100+100 得 127；保留低八位則得到 −56。音訊振幅可能需要飽和，雜湊或校驗碼的模運算則不能任意改用它。

SIMD 將一個暫存器切成多個 lane，分別做短整數運算。**進位不跨 lane 傳遞。** 例如兩個八位元無號 lane 各自計算 `(255,1)+(1,2)`，截低位結果是 `(0,3)`，不是把整串十六位元當成一個整數相加。

飽和會失去可逆性：127 可由許多超界和得到，無法只靠結果還原輸入。選指令前要確認 lane 寬度、有號解讀與飽和政策。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.3 乘法：把乘數的每一位變成一列

兩個 n 位元無號數的完整乘積最多需要 2n 位，因為 $(2^n-1)^2 < 2^{2n}$。32 位元輸入的完整乘積最多需 64 位；若只需要低 32 位，也可直接計算模 $2^{32}$ 的結果，硬體不必先形成高半部。

把乘數展開為 $Q=\sum_{k=0}^{n-1} q_k 2^k$，便得到

$$MQ=\sum_{k=0}^{n-1} q_k (M \ll k).$$

$q_k$=1 就加上左移 k 位的 M，$q_k$=0 就加零。這些列稱為部分乘積（partial products）。循序硬體逐輪累加，平行硬體先同時生成再壓縮；兩者都要保留各列的位權。

先固定無號或二補數解讀。相同位元在有號乘法中的高位權重不同，不能直接照搬無號部分乘積。



## 循序乘法：13×11 的四輪狀態

M=`1101`、Q=`1011`、A=0。每輪讀 $Q_0$，為 1 就做 $C:A=A+M$，否則 C=0、A 不變；再將完整的 (C,A,Q) 邏輯右移一位，C 進入 A 的最高位。

| 輪次 | $Q_0$ | 加法結果 $C:A$ | 右移後 A | 右移後 Q |
|---|---|---|---|---|
| 0 | | | `0000` | `1011` |
| 1 | 1 | `0:1101` | `0110` | `1101` |
| 2 | 1 | `1:0011` | `1001` | `1110` |
| 3 | 0 | `0:1001` | `0100` | `1111` |
| 4 | 1 | `1:0001` | `1000` | `1111` |

最終 `(A,Q)=10001111`$_2$`=143=13×11`。C 保存當輪加法的第五位；不能在右移之前把它丟掉。

## 循序乘法：用不變量連起每輪狀態

固定初始乘數為 $q$，完成 $k$ 回合的暫存器為 $A_k, Q_k$，定義 $R_k = 2^n A_k + Q_k$。精確不變量是

$$R_k = \left\lfloor \frac{q}{2^k} \right\rfloor + M(q \bmod 2^k)\, 2^{n-k}, \qquad 0 \le k \le n.$$

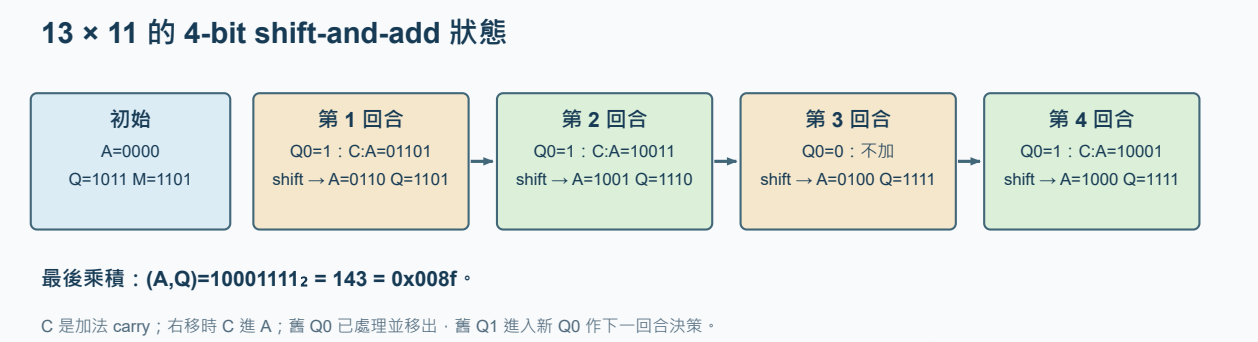
$k = 0$ 時 $R_0 = q$，成立。若 $k < n$，右項第二項為偶數，所以當前 $Q_k$ 的最低位恰為原乘數第 $k$ 位 $q_k$。依此位決定加法後，完整 $C : A : Q$ 的值為 $R_k + q_k M 2^n$；右移得到

$$R_{k+1} = \left\lfloor \frac{R_k + q_k M 2^n}{2} \right\rfloor = \left\lfloor \frac{q}{2^{k+1}} \right\rfloor + M(q \bmod 2^{k+1}) 2^{n-k-1}.$$

這使用 $q \bmod 2^{k+1} = (q \bmod 2^k) + q_k 2^k$，因此歸納延續。$k = n$ 時因 $q < 2^n$，有 $R_n = Mq$；A 為乘積高半部，Q 為低半部。進位 C 保留本回合加法的溢位並隨右移進入 A，沒有遺失該位。13×11的實際 $R_k$ 依序為11、109、158、79、143。



## 循序乘法：跨暫存器的位元如何移動？




成本模型先寫清楚才可比較：

$$T_{\text{serial}} = t_{\text{setup}} + n\, t_{\text{iter}} + t_{\text{finalize}}.$$

每輪加法與移位若可同週期完成，32 輪才對應 32 個運算週期；若分別占一週期，就需另算。管線化可縮短啟動間隔，單筆延遲仍須依階段數與時脈週期計算。

## 有號乘法：先決定位元寬度

對兩個 $n$-bit 二補數輸入，完整乘積放在 $2n$ 個有號位元中。可以先處理大小、最後套用符號，也可以直接用 Booth 重編碼產生帶正負號的部分乘積。

**不能在原本的有號寬度內，直接對最小負數取絕對值。** 例如四位元二補數的範圍是 $[-8, 7]$，$-8$ 的正大小 8 需要五個有號位元；若改用四位元無號容器，`1000` 才能代表大小 8。

以下先固定 radix-2 Booth：輸入 $M, Q$ 各為 $n$-bit 有號數；$M$ 稱被乘數，$Q$ 稱乘數。內部將 $M$ 符號擴充至 $n+1$ 位，累加器 $A$ 也用 $n+1$ 位，並額外保存一位 $q_{-1}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Booth 的兩個位元在判斷什麼？

把原乘數的位元寫成 $q_{n-1} \cdots q_0$，補 $q_{-1} = 0$。第 $i$ 個重編碼係數為 $d_i = q_{i-1} - q_i$。本輪讀取順序固定是 **目前最低位在前、上一位在後**：

| $Q_0\, q_{-1}$ | d | 右移前的操作 |
|---|---|---|
| `00` | 0 | A 不變 |
| `01` | +1 | A ← A + M |
| `10` | −1 | A ← A − M |
| `11` | 0 | A 不變 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 為什麼重編碼保留二補數的值？

把每個原位元分別收集。最低位之外的正項來自下一個係數，負項來自目前係數；最高位沒有下一項抵消，留下二補數的負權重。

重編碼確實保留二補數的值，因為

$$\begin{aligned}\sum_{i=0}^{n-1}(q_{i-1}-q_i)2^i &= \sum_{i=0}^{n-2} q_i 2^i - q_{n-1}2^{n-1} \\ &= Q_{\text{signed}}.\end{aligned}$$

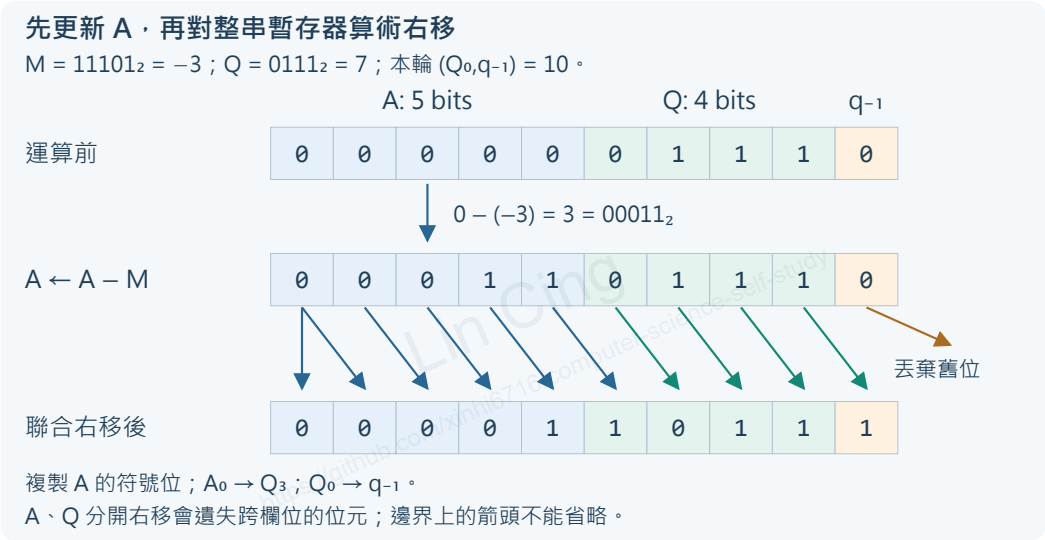
中間位元的兩項合成 $(2^{i+1}-2^i)q_i = 2^i q_i$；最高位只留下負權重。這是規則對負乘數也成立的理由。

例如 `0111` 得到低位至高位的係數 −1、0、0、+1，因此值是 −1+8=7；`1101` 得到 −1、+1、−1、0，因此值是 −1+2−4=−3。係數可為負，總和仍是原乘數。



## 一輪先加減，再聯合算術右移

初始化 $A=0$、$Q=$ 輸入乘數位元、$q_{-1}=0$。每輪先依表更新 $A$，再將串接的 $(A,Q,q_{-1})$ **整串算術右移一位**，共做 $n$ 輪。




符號擴充只複製最左端 $A$ 的符號位；$A_0$ 進入 $Q$ 的最高位，舊 $Q_0$ 進入 $q_{-1}$。結束後丟掉 $q_{-1}$，$(A,Q)$ 多出的一個最左符號位也是冗餘位；剩下 $2n$ 位就是完整乘積。

## −3 乘 7：四輪暫存器追蹤

取 $n = 4$，$M = -3$ 擴位為 `11101`；起始 $(A, Q, q_{-1}) =$`00000 0111 0`。每列最後才右移，不能用右移後的 pair 決定同一列操作。

| 輪次 | 舊 $Q_0q_{-1}$ | A 的操作 | 加減後 A Q $q_{-1}$ | 右移後 A Q $q_{-1}$ |
|---|---|---|---|---|
| 1 | `10` | −M | `00011 0111 0` | `00001 1011 1` |
| 2 | `11` | 0 | `00001 1011 1` | `00000 1101 1` |
| 3 | `11` | 0 | `00000 1101 1` | `00000 0110 1` |
| 4 | `01` | +M | `11101 0110 1` | `11110 1011 0` |

最後 $(A, Q) =$`11110 1011`，是九位元二補數的 $-21$；移除冗餘符號位得到 `11101011`，八位元值仍為 $-21$。

原乘數 `0111` 的係數由低位到高位是 $(-1, 0, 0, +1)$，所以 $7 = -1 + 8$，$(-3) \cdot 7 = 3 - 24 = -21$。四輪中只有兩次加減，另外兩輪仍須移位。



## Booth 數例：正被乘數與負乘數

M = 00010 (+2) Q = 1101 (−3) A = 00000 $q_{-1}$ = 0

| 輪次 | 舊 $Q_0q_{-1}$ | 操作 | 算術右移後：A Q $q_{-1}$ |
|---|---|---|---|
| 1 | 10 | A − M | 11111 0110 1 |
| 2 | 01 | A + M | 00000 1011 0 |
| 3 | 10 | A − M | 11111 0101 1 |
| 4 | 11 | 不變 | 11111 1010 1 |

令 $n = 4$、$M = 2$、$Q = -3$。圖中 $A, M$ 均擴為 5 bits，$Q$ 保持 4 bits；每輪先依舊的 $Q_0q_{-1}$ 選操作，再將整個 $(A, Q, q_{-1})$ 算術右移。

四輪後 $(A, Q) =$ `11111 1010` 是 9-bit 的 −6；移除冗餘符號位得 8-bit `11111010`，與 $2 \times (-3) = -6$ 一致。

## 多留一個符號位，才能涵蓋最小負數

對 $M=-8, Q=-3$，第一個 pair 是 `10`，必須計算 $A-M=+8$。若 $A, M$ 都只用四位元，`1000` 被解讀成 $-8$，第一輪便錯了。

### −8 × (−3)：取負數之前，先擴充符號位

只用 4-bit A、M：失敗

M = $1000_2$ = −8

−M = $1000_2$，仍被解讀為 −8

第一次 A−M：1000，不是 +8

右移後：1100 0110 1

四輪結果：$11101000_2$ = −24

使用 5-bit A、M：正確

M = $11000_2$ = −8

−M = $01000_2$ = +8

第一次 A−M：01000 = +8

右移後：00100 0110 1

四輪結果：$00011000_2$ = +24

兩邊都從 A=0、Q=$1101_2$、$q_{-1}$=0 開始；差異是加減器與 A 的寬度。

先用五位元精確保存 +8，右移才會得到正確的 +4；四位元路徑卻從 −8 移到 −4。下一頁證明多留一位對所有輸入都足夠。



## 加減與右移的範圍證明

五位元不是任意補救：每次移位後，$A$ 都落在 $[-2^{n-1}, 2^{n-1}-1]$。加減 $n$-bit 的 $M$ 後，範圍包含於 $[-2^n, 2^n-1]$，可由 $n+1$ 位精確表示；再右移，便回到前述範圍。因此本模型不依賴中間溢位碰巧抵消。

起始 A=0 符合上述範圍。若某輪開始時 A 已在此範圍，A+M 的最小值是 $-2^n$，最大值是 $2^n-2$；A−M 的範圍則是 $[-2^n+1, 2^n-1]$。不操作時也包含於同一個 n+1 位範圍。

算術右移後的新 A，是加減結果除以 2 並向下取整，因此又落回 $[-2^{n-1}, 2^{n-1}-1]$。由歸納法，每輪的加減都能精確執行，包含被乘數等於最小負數的情況。

## 把暫存器狀態寫成可追蹤的不變量

令 $u$ 是原乘數位元的**無號值**，$d_i = q_{i-1} - q_i$ 仍從原乘數位元定義。做完 $k$ 輪後，令

$$R_k = 2^n \,\mathrm{signed}(A_k) + \mathrm{unsigned}(Q_k), \quad D_k = \sum_{i=0}^{k-1} d_i 2^i.$$

不變量是

$$R_k = \left\lfloor \frac{u}{2^k} \right\rfloor + M 2^{n-k} D_k, \qquad 0 \le k \le n.$$

起始 $R_0 = u$、$D_0 = 0$。對 $k < n$，$Q_k$ 的低 $n-k$ 位尚未被來自 $A$ 的位元覆蓋，故 $Q_0$ 正是原來的 $q_k$，歷史位則是 $q_{k-1}$。因此硬體讀到的 pair 確實選出 $d_k$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 一輪更新保留不變量，最後就是乘積

更新 $A$ 相當於把 $d_k M 2^n$ 加進 $R_k$；整串算術右移使

$$R_{k+1} = \left\lfloor \frac{R_k + d_k M 2^n}{2} \right\rfloor.$$

代入不變量。當 $k < n$，含 $M$ 的項可被 2 整除，即使它是負數也一樣，因此

$$R_{k+1} = \left\lfloor \frac{u}{2^{k+1}} \right\rfloor + M 2^{n-k-1}(D_k + d_k 2^k).$$

這就是下一輪的形式。做完 $n$ 輪，$0 \le u < 2^n$ 使第一項為零；由重編碼恆等式，

$$R_n = M D_n = M Q_{\mathrm{signed}}.$$

算術右移對負整數相當於向負無窮方向取整，不是向零截斷；這個差別已包含在上面的 floor 中。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 減少加減次數，不等於一定減少週期

`0111` 只在 1 的連續區段兩端運算；`0101` 則四個 pair 都是 `10` 或 `01`，需要四次加減。後者的無號 shift-add 只需兩次加法，故 radix-2 Booth 並不對每個輸入都較省。

在本文每輪固定一次控制與移位的模型，兩者都執行 $n$ 輪。是否略過零區段、加減與移位是否同週期、是否管線化，必須另給硬體時序才能換算 latency（單次延遲）或 throughput（吞吐量）。不能直接把「少兩次加法」寫成「快兩倍」。

Radix-4 是另一種重編碼：每次處理兩個新位元，搭配一個重疊的歷史位。它改變部分乘積數量與選取電路，不能只把上圖的右移改成兩位就當作完整規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Radix-4：三位元決定一個帶符號的係數

對偶數 $n$，第 $j$ 組讀 $(q_{2j+1}, q_{2j}, q_{2j-1})$，其中 $q_{-1} = 0$。相鄰組重疊一位；奇數寬度須先補足符號位。

| 三位元 | 係數 $c_j$ | 部分乘積的倍數 |
|---|---|---|
| `000` | 0 | 0 |
| `001` | +1 | +M |
| `010` | +1 | +M |
| `011` | +2 | +2M |
| `100` | −2 | −2M |
| `101` | −1 | −M |
| `110` | −1 | −M |
| `111` | 0 | 0 |

## Radix-4 的係數從哪裡來？

把相鄰兩個 radix-2 係數併為一組，保留低位的權重：

$$\begin{aligned} c_j &= d_{2j} + 2d_{2j+1} \\ &= (q_{2j-1} - q_{2j}) + 2(q_{2j} - q_{2j+1}) \\ &= -2q_{2j+1} + q_{2j} + q_{2j-1}. \end{aligned}$$

所以 $Q = \sum_j c_j 4^j$。三個位元都取 0 或 1，係數只能是 −2、−1、0、1、2。例如 `011` 給 0+1+1=2；`100` 給 −2+0+0=−2；`111` 給 −2+1+1=0。

每往高位移一組，基礎權重乘 4，而非乘 2。最低組補歷史位 0；之後每組共用前一組的最高位。例如 `001111` 由低至高讀成 `110`、`111`、`001`，得到 −1、0、1，因此 15=−1+0×4+1×16。



## Radix-4 數例：−7 乘 15

用六位元輸入 $M = -7$、$Q =$`001111`。本頁生成完整十二位元部分乘積；先符號擴充，再依 $4^j$ 左移。

| j | 位元位置 | 讀到的三位元 | $c_j$ | $c_j \cdot 4^j$ | 十二位元部分乘積 |
|---|---|---|---|---|---|
| 0 | $q_1\ q_0\ q_{-1}$ | `110` | −1 | +7 | `000000000111` |
| 1 | $q_3\ q_2\ q_1$ | `111` | 0 | 0 | `000000000000` |
| 2 | $q_5\ q_4\ q_3$ | `001` | +1 | −112 | `111110010000` |

$15 = -1 + 0 \cdot 4 + 1 \cdot 16$，所以乘積為 $7 - 112 = -105$，十二位元結果是 `111110010111`。加法在十二位元容器中進行，最高位外的 carry 不屬於結果；其有號值仍是 $-105$。

**Radix-4 的內部寬度要重新分析。** 若被乘數是最小負數，$-2M = 2^n$ 需要 $n+2$ 個有號位元。本文先在 $2n$ 位形成部分乘積；沒有宣稱 radix-2 的 $n+1$-bit 累加器可直接套用於所有 radix-4 設計。

## 乘法加速：先把部分乘積排成同一欄位

以下只處理四位元**無號乘法**：M=13、Q=15。先產生四列部分乘積，每列已依乘數位元的位置左移，全部放在八位元容器中。

| 列 | 來源 | 八位元內容 | 無號值 |
|---|---|---|---|
| $P_0$ | $M \cdot q_0$ | 00001101 | 13 |
| $P_1$ | $M \cdot q_1 \cdot 2$ | 00011010 | 26 |
| $P_2$ | $M \cdot q_2 \cdot 4$ | 00110100 | 52 |
| $P_3$ | $M \cdot q_3 \cdot 8$ | 01101000 | 104 |

目標是把四列的總和 195 算出來。若逐次把兩列加成一個普通二進位數，每次加法都可能等待跨欄進位。Carry-save 的做法是：**中途先保留兩列，最後才把它們加成單一結果**。



## 三個同權重位元，改存成兩個位元

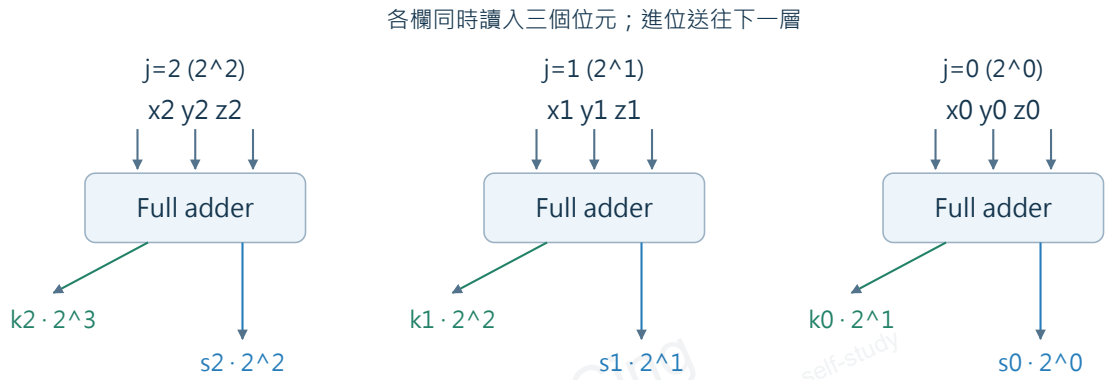
第 j 欄的 $x_j$、$y_j$、$z_j$ 都帶權重 $2^j$。全加器（full adder）只看這三個輸入，輸出同欄的 $s_j$ 與下一欄的 $k_j$：

$$s_j = x_j \oplus y_j \oplus z_j, \qquad k_j = (x_j y_j) \vee (y_j z_j) \vee (z_j x_j).$$

| $x_j+y_j+z_j$ | 對應輸入 | $s_j$ | $k_j$ | 恆等式 |
|---|---|---|---|---|
| 0 | 000 | 0 | 0 | 0=0+2・0 |
| 1 | 001、010、100 | 1 | 0 | 1=1+2・0 |
| 2 | 011、101、110 | 0 | 1 | 2=0+2・1 |
| 3 | 111 | 1 | 1 | 3=1+2・1 |

將各欄恆等式乘上 $2^j$ 再相加，得到 X+Y+Z=S+C，其中 $S = \sum_j s_j 2^j$、$C = \sum_j k_j 2^{j+1}$。**C 已經包含左移一位**，後面不可再移一次。此處的乘法 $x_j y_j$ 是位元 AND，$\vee$ 是 OR。

## Carry-save 省下的是「同一層等待進位」

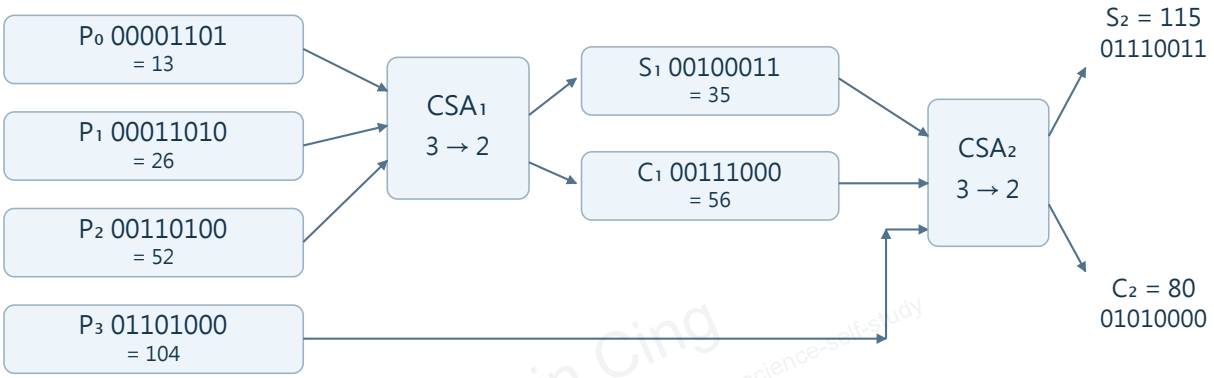



每個全加器都只讀**本層輸入的三列**；它產生的進位寫入下一層的 C 列，不拿來驅動本層左鄰的全加器。圖中 x、y、z 的三個位元可同時備妥，三欄也可並行計算。

這不代表進位消失：它成了另一列資料。S 與 C 各自都還不是總和；只有 S+C 才代表原三列。若每層都立即把 S+C 算完，就重新引入了完整進位傳播。



## 13×15：四列經兩層壓縮，再相加




第一層壓縮 $P_0$、$P_1$、$P_2$，得到 $S_1$=35、$C_1$=56，並直接保留 $P_3$=104；總和仍為 35+56+104=195。第二層得到 $S_2$=115、$C_2$=80；兩列相加才得到 195。

圖中的 $C_1$、$C_2$ 都已按實際權重左移。每一層只壓縮列數，沒有把數值除以 2，也沒有把尚未計入的進位丟掉。

## 最後的加法，確實還要處理進位

現在做普通加法 $01110011_2 + 01010000_2$。令 $c_j$ 是進入第 j 欄的進位，初始 $c_0=0$；由最低位開始：

| j，由低到高 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| $S_2[j]$ | 1 | 1 | 0 | 0 | 1 | 1 | 1 | 0 |
| $C_2[j]$ | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 |
| 輸入進位 $c_j$ | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| 結果位 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 |
| 輸出進位 $c_{j+1}$ | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 |

反向讀結果欄得到 $11000011_2 = 195$。第 4 欄產生的進位一路影響到第 7 欄；這就是最後的 carry-propagate adder（CPA）需要解決的相依。CPA 可用 ripple-carry 或更快的 prefix 結構，不能視為零延遲。



## 何時可以丟掉最高位外的進位？

對任意三個 W 位元無號數，精確 S 需要 W 位，C 可能需要 W+1 位，而精確總和可能需要 W+2 位。例：W=4、X=Y=Z=15，S=15、C=30，所以總和為 45，不能放進四位元。

若明定只取低 W 位，則可計算

$$X + Y + Z \equiv S + (C \bmod 2^W) \pmod{2^W}.$$

上例四位元 C 只剩 14；15+14=29，再取低四位得到 13，恰等於 45 mod 16。**模數結果正確，不等於精確總和沒有溢位**。

本節無號乘法不同：輸入小於 $2^n$，所有非負部分乘積的總和小於 $2^{2n}$。因此每層保存的非負列也不會超過此總和，2n 位足以容納。若使用 Booth 負部分乘積，則須先在指定寬度做二補數與符號擴充；例如前節十二位元的 7+0+3984=3991，低十二位解讀為 −105。此時不能把各列當成原本的無號部分乘積來證明範圍。

## 列數變少，不等於每一層都是一次完整加法

採用「每層將三列分組壓成兩列，剩餘列直接往下傳」的模型，列數滿足

$$R_{\ell+1} = 2\lfloor R_\ell/3 \rfloor + (R_\ell \bmod 3).$$

32 列依序變成 22、15、10、7、5、4、3、2，共八層壓縮。這是指定分組模型的層數；實際 Wallace／Dadda 位元堆可逐欄安排全加器與半加器，不能只憑名稱套用此精確數字。

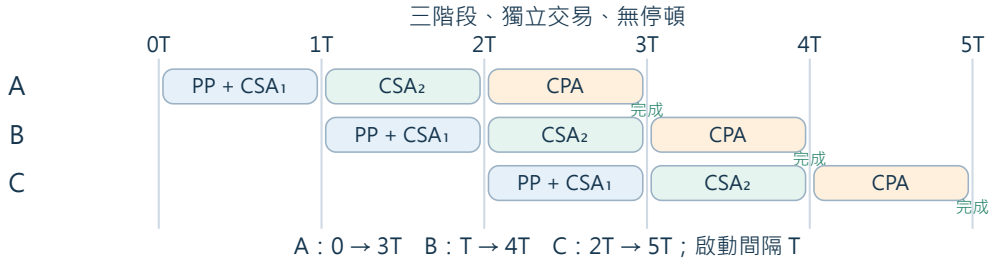
對每個 CSA 層的延遲上界 t_CSA，最長路徑可估為

$$T \le t_{\mathrm{PP}} + 8t_{\mathrm{CSA}} + t_{\mathrm{CPA}} + t_{\mathrm{wire}}.$$

其中 t_PP 包含部分乘積生成；t_wire 明列額外布線成本。八層 CSA 不是八次 W 位 CPA。平衡的二輸入普通加法樹，32 列只需五層，卻每層都可能有進位傳播。兩者延遲要代入電路與負載資料，不能用 8 和 5 直接比快慢。



## Pipeline：較常接受新輸入，不代表較早得到同一筆結果




假設把前述四列數例切成三個暫存器階段：①生成部分乘積並完成 CSA₁，②完成 CSA₂，③完成 CPA。每階段都能在共同時脈週期 T 內完成，沒有停頓，且相鄰交易彼此獨立。

交易 A 在邊緣 0 接受、邊緣 3 完成；B 在邊緣 1 接受、邊緣 4 完成。單筆延遲是 3T，啟動間隔（initiation interval）是 T。N≥1 筆獨立交易的最後一筆於 $(N+2)T$ 完成。

若 B 必須使用 A 的乘積，就不能在邊緣 1 當作已取得結果。T 還須容納最慢階段、暫存器與時脈相關成本；未切 pipeline 的電路沒有相同的 T 定義，不能由這張表推論固定三倍加速。

## MIPS 乘法：HI 與 LO 分別保存哪一半？

傳統 MIPS32 的 `mult` 以有號方式解讀兩個輸入，`multu` 以無號方式解讀；兩者都產生完整 64 位元乘積，HI 保存高 32 位、LO 保存低 32 位。`mfhi`、`mflo` 再將指定半部搬到一般暫存器。

例如 `0xffffffff×2`：有號解讀為 −1×2，完整結果為 `0xfffffffffffffffe`；無號解讀為 4294967295×2，結果為 `0x00000001fffffffe`。**低半部相同，高半部不同。**

這兩條指令不因乘積超出 32 位元而自動產生有號溢位 trap。若程式要求結果可放入某種 32 位元型別，還須檢查高半部是否符合該型別的延伸規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 3.4 除法：先固定商與餘數的規則

無號被除數 D、非零除數 M 要求

$$D = QM + R, \qquad 0 \le R < M.$$

等式與餘數範圍一起決定唯一的 Q、R。只有等式仍不夠，例如 13=2・5+3，也等於 1・5+8；後者不符合餘數範圍。

本節先用恢復餘數除法（restoring division）處理無號輸入，再處理有號的取整規則。除數為零須先排除；有號最小負數除以 −1 則另有可表示範圍問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 每輪先帶下一位進來，再試減一次

每輪處理一個被除數位元 d。假設原餘數為 A，先形成 $T = 2A + d$，試算 $T - M$。

| 試減結果 | 新商位 b | 新餘數 A′ |
|---|---|---|
| $T - M \geq 0$ | 1 | $T - M$ |
| $T - M < 0$ | 0 | T；若已做減法，就加回 M |

已知 $0 \leq A < M$、$d \in \{0, 1\}$，所以 $0 \leq T < 2M$。一次減法就足夠：減或不減之後，兩個分支都回到 $0 \leq A' < M$。這正是每輪只產生一個商位的原因。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 13÷5：分開看已讀前綴、商與餘數

由左至右讀取 D=1101。表中的商只包含已產生的位元，不是後面硬體圖的整個 Q 暫存器。

| k | 新位 d | 已讀前綴 $P_k$ | 試算 T | T−5 | 已產生商 | 新餘數 |
|---|---|---|---|---|---|---|
| 0 | — | 0 | — | — | 空 | 0 |
| 1 | 1 | 1 | 1 | −4 | `0` | 1 |
| 2 | 1 | 3 | 3 | −2 | `00` | 3 |
| 3 | 0 | 6 | 6 | 1 | `001` | 1 |
| 4 | 1 | 13 | 3 | −2 | `0010` | 3 |

每列都可核對 $P_k = q_k M + A_k$。例如第三列是 6=1・5+1；最後得到 13=2・5+3，且餘數 3 小於除數 5。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Q 暫存器同時裝著未讀輸入與已產生的商

硬體初值是 A=`00000`、Q=`1101`、M=`00101`。每輪先將 A、Q 合併左移；舊 Q 的最高位進入 A，Q 的最低位空出來存新商位。

| 輪次 k | 餘數 A（5 位） | | Q：未讀輸入 → 已產生商 | 商 $q_k$ | $P_k=q_k\cdot5+A$ |
|---|---|---|---|---|---|
| 0 | 00000 | = 0 | 1 1 0 1 | 0 | 0 |
| 1 | 00001 | = 1 | 1 0 1 \| 0 | 0 | 1 |
| 2 | 00011 | = 3 | 0 1 \| 0 0 | 0 | 3 |
| 3 | 00001 | = 1 | 1 \| 0 0 1 | 1 | 6 |
| 4 | 00011 | = 3 | 0 0 1 0 | 2 | 13 |

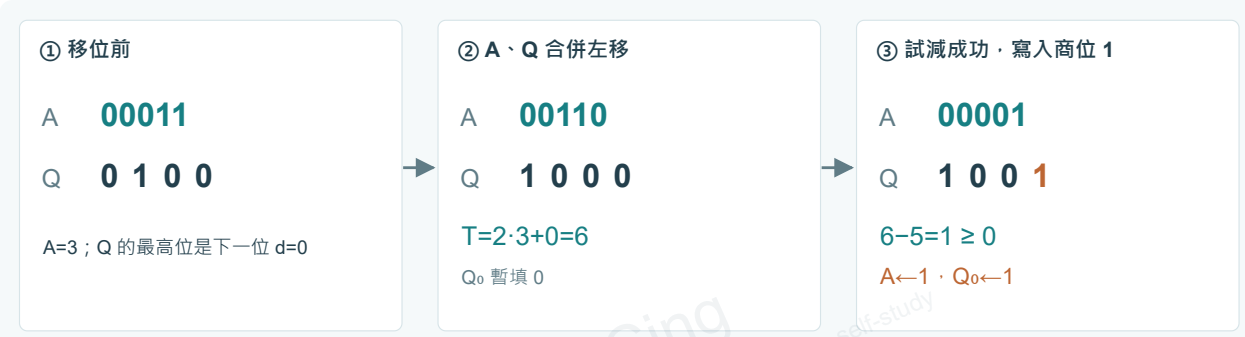
綠：未讀的被除數位元　橘：商位　|　只有 k=4 時，整個 Q 才是商 2。

**前 k 輪的數學商 $q_k$，不等於同一時刻整個 Q 的無號值。** 前者只含圖中的商區；後者還混有未讀的被除數位元。到第 n 輪，未讀區才完全消失，Q 才是最終商。



## 第三輪：跨暫存器移位後，才決定新商位

第二輪結束時 A=3、Q=`0100`。接下來讀的是原被除數第三位 0，故左移形成 T=6，再試減 M=5。




舊 $Q_3$ 進入 $A_0$；其餘 Q 位元左移。最後一個空位保存本輪商位，不是另一個輸入位元。

圖中試減結果為 +1，寫回 A=`00001`，並把 $Q_0$ 設為 1，得到 Q=`1001`。這個 Q 尚不是最終商 9；最高位的 1 仍是最後一個未讀被除數位元。

## 一般證明：前綴不變量與餘數範圍

起始 $P_0 = q_0 = A_0 = 0$。假設 $P_k = q_k M + A_k$ 且 $0 \le A_k < M$。下一位為 d，令 $T = 2A_k + d$，選 b=1 當且僅當 T≥M，則

$$P_{k+1} = 2P_k + d = (2q_k + b)M + (T - bM).$$

因此 $q_{k+1} = 2q_k + b$、$A_{k+1} = T - bM$，等式持續成立；由 T<2M，餘數也保持在 [0,M)。做完 n 輪後 $P_n$=D，恰得到要求的商餘數。

本教學實作以 n+1 位保存 T 與試減差：T≤$2^n$−1，且 −($2^n$−1)≤T−M≤$2^n$−1，皆可精確表示。T 的上界也可由「已讀前綴最多 n 位」與餘數不大於此前綴推出，不能只靠 T<2M 就把任意 T 當成 n+1 位有號數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 有號除法：向零截斷與餘數符號

本節固定商向零截斷（truncate toward zero）。先以無號容器計算兩輸入的大小，再依符號還原；非零餘數的符號與被除數相同。

| D | M | 商 Q | 餘數 R | 驗算 D=QM+R |
|---|---|---|---|---|
| 7 | 2 | 3 | 1 | 7=3・2+1 |
| −7 | 2 | −3 | −1 | −7=(−3)・2−1 |
| 7 | −2 | −3 | 1 | 7=(−3)・(−2)+1 |
| −7 | −2 | 3 | −1 | −7=3・(−2)−1 |

此時 $|R| < |M|$，但 R 不必非負。若規定商向下取整，−7÷2 改得商 −4、餘數 +1；等式仍成立，規則已不同。零餘數不需要附加正負號條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 邊界與時序：不能只把每輪位數換成倍速

n 位元最小負數的大小 $2^{n-1}$ 可放入 n 位元無號容器，卻不能放入 n 位元正有號數。最小負數除以 −1 的正商超出有號上界，還原符號前就要檢查。

傳統 MIPS32 的 `div`、`divu` 以 LO 保存商、HI 保存餘數；本章版本不因除以零或有號除法溢位自動產生算術 trap，這些輸入下的 HI/LO 結果不可依賴，程式須先檢查。

本節組合暫存器演算法每輪讀一位，共 n 輪；原書先移動寬除數的初版演算法使用 n+1 次，不能混用迴圈數。SRT 等設計每輪可選多位商，但選位、正規化與修正都有成本；每輪兩位不直接證明快兩倍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 算術右移不一定等於有號除法

二補數算術右移一位等於除以 2 後向下取整；本節有號除法則向零截斷。以 −7 為例：

| 操作 | 結果 | 原因 |
|---|---|---|
| `sra` 一位 | −4 | $\lfloor -7/2 \rfloor$ |
| 向零截斷的除以 2 | −3 | 丟棄小數部分 |
| 負數先加 1，再算術右移一位 | −3 | $\lfloor (-7+1)/2 \rfloor$ |

一般除以 $2^k$ 可對負數先加 $2^k - 1$ 再算術右移，但必須固定 $1 \le k < n$、輸入型別與不溢位的運算方式。編譯器用移位或倒數乘法替換除法，也必須維持原本的取整規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.5 浮點格式：範圍與精度分開看

IEEE 二進位浮點格式包含符號、偏移指數與 fraction 欄位。正常值（normal）的有效數首位 1 不存入欄位，因此有效精度比 fraction 多一位。

| 格式 | 符號位 | 指數位 | fraction 位 | 有效精度 p | 指數偏移 bias |
|---|---|---|---|---|---|
| binary32 | 1 | 8 | 23 | 24 | 127 |
| binary64 | 1 | 11 | 52 | 53 | 1023 |

增加指數位主要擴大動態範圍；增加 fraction 位才增加有效精度。欄位指數全零或全一時須改用特殊規則，不能一律補上隱含的 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 浮點欄位：先找到符號、指數與 fraction

圖中的 fraction 不包含正常值隱含的首位 1；欄位位數與有效精度不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 欄位解碼：先分類，再代公式

對 binary32，令 E 是指數欄的無號整數、F 是 23 位 fraction 的無號整數：

| E | F | 數值或類型 |
|---|---|---|
| 1⋯254 | 任意 | $(-1)^s(1+F/2^{23})2^{E-127}$ |
| 0 | 非零 | subnormal：$(-1)^s F 2^{-149}$ |
| 0 | 0 | 帶符號的零 |
| 255 | 0／非零 | 無窮大／NaN |

subnormal 把正常值的首位 1 改成 0，靠近零的間距固定為 $2^{-149}$。這是 gradual underflow（漸進下溢），不是把所有過小數直接變成零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 浮點例外：旗標記錄了什麼？

| 旗標 | 典型觸發條件 | 停用 trap 時的例子 |
|---|---|---|
| invalid | 沒有定義的數值操作 | 0/0、0 • ∞ → quiet NaN |
| divide-by-zero | 有限非零數除以零 | 1/(+0) → +∞ |
| overflow | 以目標精度、無界指數先捨入後超出最大指數 | RNE 下最大有限值加自身 → +∞ |
| underflow | 本節環境中 tiny 且 inexact | 最小正 subnormal 除以 2 → +0 |
| inexact | 結果不等於精確值 | $1+2^{-24}$ 捨成 binary32 的 1 |

旗標與程式語言拋出的 exception 不是同一層。是否啟用 trap，以及如何讀取或清除旗標，要由浮點環境指定。後面的數例固定 RNE、停用 trap 與漸進下溢，並另說明 tininess 的判定時點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 浮點加法：先對齊，再決定捨入

有限正常值相加時，先比較指數，把指數較小的有效數（significand）右移到共同尺度；異號輸入改作大小相減，絕對值不同時由較大絕對值的輸入決定符號；大小相等而精確相消時，零的符號依捨入方向決定，例如 RNE 得 +0、朝負無窮方向捨入得 −0。移出的低位仍須保留資訊，不能先截斷。

完成加減後，先正規化，再依目標精度捨入；若捨入產生新的進位，還要再正規化並調整指數。特殊值、零、溢位與 subnormal 路徑須另外處理。

以下使用 **p=4 個有效位（含首位 1）、RNE、指數範圍不設限** 的縮小模型，讓所有中間位元可見；它不是 binary32 格式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 對齊數例：sticky bit 如何改變結果

令 a=1、b=9/128=0.0703125。b 原為 $1.001_2 \times 2^{-4}$，對齊到 $2^0$ 後為 $0.0001001_2$；兩個輸入在 p=4 都是精確值。

**p=4（含首位 1），RNE；指數範圍不設限**

| ×$2^0$ | 保留 4 bits | G | R | 其餘 bits |
|---|---|---|---|---|
| a = 1 | 1.000 | 0 | 0 | 00 |
| b = 9/128 | 0.000 | 1 | 0 | 01 |
| **a + b** | **1.000** | **1** | **0** | **01** |

$b = 1.001_2 \times 2^{-4} = 0.0001001_2 \times 2^0$

**S = OR(0,1) = 1 → GRS = 101 → $1.001_2$ = 1.125**

若把 tail 當 00，會誤判為 tie，得到 $1.000_2$。

精確和為 137/128；相鄰候選是 1 與 9/8，中點為 17/16=136/128。精確和大於中點，故選 9/8，誤差 7/128。若先丟棄低位而回傳 1，誤差是 9/128，已大於半個 ULP=1/16。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 相消後的正規化：先找回首位 1

同一個 p=4 模型計算 0.75−0.6875。輸入指數相同，不必先移位；相減後卻要左移三位，才能恢復 normal 的首位 1。

**p=4、RNE：0.75 − 0.6875，兩個輸入皆可精確表示**

| | | |
|---|---|---|
| a | **1.100 000** | × $2^{-1}$ |
| − b | **1.011 000** | × $2^{-1}$ |
| 相減 | **0.001 000** | × $2^{-1}$ |
| 左移 3 位 | **1.000 000** | × $2^{-4}$ |

每左移一位，指數減一；數值維持不變。

GRS=000，結果 1/16=0.0625 是精確值，inexact=0。

本例的輸入與差都精確，沒有捨入誤差。相消可能放大**輸入先前已有的相對誤差**，不代表每一次相減本身都不精確。若左移時已碰到最小 normal 指數，必須改走 subnormal 路徑，不能無限降低指數。



## Guard、round、sticky：從距離推導 RNE

先看結果的大小。最低保留位記為 L；其後第一位為 guard G，第二位為 round R，更右所有位元取 OR 得 sticky S。S 是「後方是否有非零位」的摘要，不是單一位元的位置。

相對一個保留單位 ULP，G 的權重是 1/2，R 是 1/4，其餘尾端小於 1/4。故 G=0 時低於中點；G=1、R=S=0 時恰在中點；G=1 且 R 或 S 非零時高於中點。中點選最低保留位為 0 的候選，於是

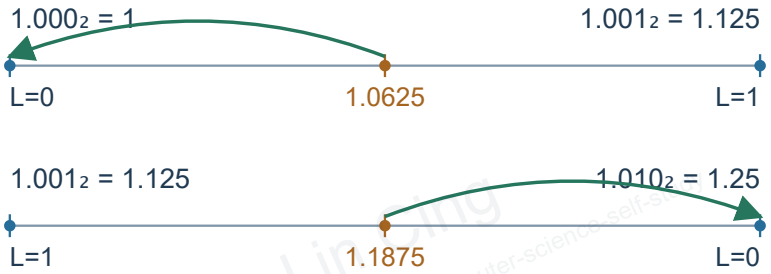
$$\mathrm{inc} = G \wedge (R \vee S \vee L).$$

inc=1 時對**保留的大小**加一個 ULP，之後加回符號。這對負數可能使數值更負，因此不能一律稱為「向上捨入」。只要丟棄部分非零，結果就不等於精確值，須記錄 inexact。

## 同樣是中點，為何有時保留、有時加一？

下圖仍用 p=4。第一個區間左端為偶，第二個區間右端為偶；even 指有效數整數編碼的最低位是 0，不是十進位值是否為偶數。

**p=4、RNE；每條數線只畫一個相鄰區間**




**inc = G ∧ (R ∨ S ∨ L)**

負數先捨入大小，再加回符號：−1.1875 → −1.25。

捨入也可能產生進位。例如 $1.1111_2$ 截成 p=4 後 L=1、GRS=100，須加一：$1.111_2+0.001_2=10.000_2$，再正規化為 $1.000_2\times2^1$。不能只修改 fraction 欄而漏掉指數增加。



## Binary32 的兩個捨入檢查點

binary32 在 [1,2) 的間距為 $2^{-23}$，精度 p=24。對 $1+2^{-24}$，L=0、GRS=100，故選 1（0x3f800000）；對 $1+3\cdot2^{-25}$，GRS=110，故選 $1+2^{-23}$（0x3f800001）。兩次皆不精確，inexact=1。

**Binary32 round-to-nearest-even 與五種 FP 例外**

| Tie：$1+2^{-24}$ | Above half：$1+3\cdot2^{-25}$ | Underflow tie：$2^{-150}$ | Overflow：max+max |
|---|---|---|---|
| GRS=100，保留 LSB=0 | GRS=110 | 0 與 $2^{-149}$ 的中點 | exact result > max finite |
| candidate：1.0 | round up：$1+2^{-23}$ | even→+0 | rounded→+∞ |
| inexact | inexact | underflow + inexact | overflow + inexact |

**Invalid：0×∞、∞−∞、0/0 → quiet NaN；1/±0 → signed infinity。**

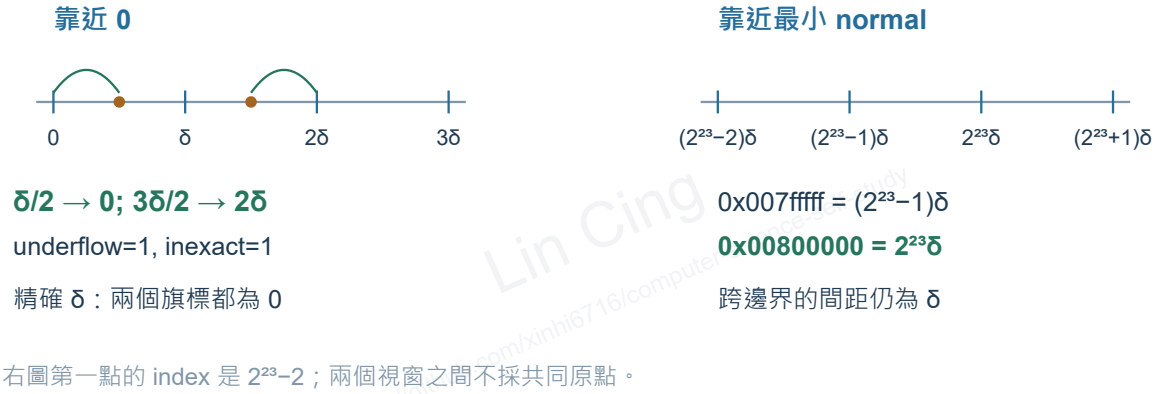
G/R/S 是 guard、round、sticky；tie 選最低保留 bit 為 0，NaN 依 payload 與 comparison 規則傳播。

這裡的 $2^{-24}$ 是相鄰值間距的一半；不是「精確和的一半」。FMA 對精確 ab+c 只捨入一次，分開的乘法與加法各自捨入，結果可能不同；本章稍後給出可核對的位元數例。

## Subnormal：範圍延伸與旗標要分開判斷

令 $\delta=2^{-149}$。binary32 的正 subnormal 為 $F\delta$，其中 fraction 整數 $1\le F\le 2^{23}-1$；最小 normal 為 $2^{23}\delta=2^{-126}$。接近零的間距固定為 $\delta$，跨到最小 normal 時仍連續。

**binary32: $\delta = 2^{-149}$; RNE; gradual underflow**




**$\delta/2 \to 0$; $3\delta/2 \to 2\delta$**

underflow=1, inexact=1

精確 δ：兩個旗標都為 0

0x007fffff = $(2^{23}-1)\delta$

**0x00800000 = $2^{23}\delta$**

跨邊界的間距仍為 δ

右圖第一點的 index 是 $2^{23}-2$；兩個視窗之間不採共同原點。

**Subnormal 不等於一定設定 underflow 旗標。** 精確的 $2^{-126}/2$ 得 0x00400000，沒有誤差；圖中兩個中點則因 tiny 且 inexact，同時設定兩個旗標。下一頁明定浮點環境並列出比較。



## 用同一張表核對 subnormal 的精確性

本表固定 RNE、gradual underflow、停用 trap、旗標初值為 0，並在捨入後判定 tininess。所有輸入皆能以 binary32 精確表示；UF、NX 分別代表 underflow 與 inexact 旗標。

| 運算 | 精確值 | RNE 結果 bits | UF | NX |
|---|---|---|---|---|
| $2^{-126}/2$ | $2^{-127}$ | 0x00400000 | 0 | 0 |
| $\delta/2$ | $\delta/2$ | 0x00000000 | 1 | 1 |
| $(3\delta)/2$ | $3\delta/2$ | 0x00000002 | 1 | 1 |
| 最小 normal − 最大 subnormal | $\delta$ | 0x00000001 | 0 | 0 |

前兩個中點以 fraction 整數的奇偶選擇：0 與 1 之間選 0；1 與 2 之間選 2。旗標不儲存誤差大小；通常新觸發的旗標會累積，直到明確清除。若要判斷單次運算是否觸發，應先清除舊旗標。若開啟 flush-to-zero（FTZ）或把 subnormal 輸入當成 0，已改變本節模型，不能直接套用此表。

表示法與 gradual underflow 可參照 Oracle Numerical Computation Guide § 2.3。

## 特殊值：同時寫出結果與例外

最大有限值加自身在 RNE 下得 +∞，設定 overflow 與 inexact。0×∞、∞−∞、0/0 為 invalid，停用 trap 時回傳 quiet NaN。

有限非零數除以 ±0 得帶符號的 infinity，符號由兩輸入的 sign bits XOR 決定，設定 divide-by-zero；∞/+0 則仍為 infinity，不因這次運算設定 divide-by-zero。詳見 Oracle Numerical Computation Guide 的例外表。

NaN 的比較須依 predicate 判斷：普通相等比較即使兩邊都是同一個 NaN 也為 false，不等比較為 true；ordered 與 unordered 不能混用。+0 與 −0 比較相等，但倒數與 signbit 可區分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 浮點乘法：符號、指數、有效數各做什麼？

先限於有限正常輸入 $a = (-1)^{s_a} m_a 2^{e_a}$、$b = (-1)^{s_b} m_b 2^{e_b}$，其中 $1 \le m_a, m_b < 2$。符號為 $s_a \oplus s_b$，精確有效數乘積在 [1,4)，未正規化指數為 $e_a + e_\beta$。

若乘積≥2，右移一位並把指數加一；再依目標精度捨入，必要時再次處理進位。例 $1.5 \times 1.5 = 10.01_2 = 1.001_2 \times 2^1$，正規化需要右移一位，不能寫成固定左移。

儲存的指數有 bias，兩個指數欄相加時要扣回一次 bias，再加入正規化調整。subnormal、零與無窮大另外分類；0 • ∞ 屬 invalid，不能走一般「零乘任何數得零」路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## MIPS 浮點：資料格式也寫在指令中

本章傳統 MIPS 的浮點暫存器位於 coprocessor 1，與一般整數暫存器分開。`lwc1`、`swc1` 搬移一個浮點 word 的位元；`add.s` 與 `add.d` 則選擇不同浮點格式的加法。

浮點比較設定條件碼，再由 `bc1t`、`bc1f` 分支。遇到 NaN 時，必須確認使用的是哪種 ordered／unordered 判斷，不能把它當成普通整數比較。

同一串位元可被搬移而不做數值轉換；要改變格式或轉成整數，須使用相應轉換指令。暫存器配置與保存區大小也要配合格式及 ABI，不能只看變數名稱。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 捨入方向與誤差尺度

RNE（round to nearest, ties to even）選最近值，中點才看最低保留位。方向性捨入則可朝 $+\infty$、$-\infty$ 或零；它們回答不同需求，例如區間上下界不能任意用 RNE 代替。

ULP 描述某個尺度附近相鄰可表示值的間距。binary32 在 [1,2) 的間距為 $2^{-23}$，在 [2,4) 則為 $2^{-22}$。同一個絕對誤差，在接近零與遠離零時的相對大小不同。

有效精度 p 控制可保留的有效二進位位數，指數範圍控制能表示多大或多小。兩者分別限制細節與尺度，不能用「位元更多」概括。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## NaN 與帶符號的零：不能照整數排序

普通相等比較中，NaN 與任何值都不相等，包括自身；不等比較則為真。要辨識 NaN，使用 `isnan` 或指定的 unordered 判斷，並確認比較指令的例外規則。

IEEE 相等比較中 +0=−0，但符號位不同。倒數可分成 +∞、−∞，`signbit` 也能區分，因此相等不代表編碼相同。

浮點負數的編碼順序、帶符號的零與 NaN 都使「把位元當有號整數直接比較」失去一般正確性。應使用對應格式的浮點比較規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 3.6 平行求和：結果為什麼會隨順序改變？

精確整數加法，以及固定模數的加法，都有結合律；逐步捨入的浮點加法則可能使 $(a+b)+c \neq a+(b+c)$。平行歸約（parallel reduction）改變求和樹，也就可能改變中間捨入。

若要求位元完全重現，須固定求和次序，以及格式、捨入模式、中間精度與 FMA 收縮等環境；或採用有明確順序無關保證的累加器。補償求和與提高精度主要改善誤差，本身不保證換序後位元相同。

因此結果略有差異時，先核對運算順序與浮點環境；不能只憑差異就判定有資料競爭，也不能因此排除真的競爭問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 補償求和也不是順序無關的保證

固定標準 Kahan 的每次基本運算均以 binary32、RNE 執行。其更新為 y=a−c、t=s+y、c=(t−s)−y、s=t；起始 s=c=0。

| 讀入順序 | 關鍵中間步驟 | 最後 s |
|---|---|---|
| $[2^{25}, 1, -2^{25}]$ | 加 1 時 s 不變，c=−1；下一步 $-2^{25}+1$ 又在中點捨回 $-2^{25}$ | 0 |
| $[2^{25}, -2^{25}, 1]$ | 前兩項先精確抵銷，再加 1 | 1 |

補償變數能保存某些丟失資訊，但後續計算它或使用它時仍會捨入。這個數例否定「補償求和一律能保證換序結果相同」，不否定它在許多資料上改善誤差的價值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 換結合順序，就換了捨入時點

運算順序的影響可用確切位元驗證。以 binary32 $x = 2^{24}$ 為例，$x$ 的 ULP 是 2。x+1 恰好落在 x 與 x+2 的中點，最低保留 bit 要求選 even，所以先 round 成 x：

$$((x + 1) - x) = 0, \qquad (x - x) + 1 = 1.$$

兩式在實數算術中相等，逐次 binary32 捨入卻不同；差異來自捨入時點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## FMA：精確乘加後只捨入一次

取 $a = b = 1 + 2^{-23}$（`0x3f800001`）、$c = -(1 + 2^{-22})$（`0xbf800002`），皆能以 binary32 精確表示。

$$ab = 1 + 2^{-22} + 2^{-46}, \qquad ab + c = 2^{-46}.$$

| 計算路徑 | 中間捨入 | 最終結果 |
| --- | --- | --- |
| 先乘再加 | 乘積先捨成 $1 + 2^{-22}$，小項消失 | +0 |
| FMA | 精確形成 ab+c 後只捨入一次 | $2^{-46}$，`0x28800000` |

差別來自捨入時點，不是實數乘法規則改變。本章傳統 MIPS 核心不保證提供 FMA；使用相應擴充時，仍要指定運算元格式與捨入模式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 3.7 中間精度：暫存器裡算完，存回後可能不同

本章歷史案例比較 x87 的延伸精度堆疊暫存器與 SSE2 的 XMM 暫存器。若中間值先留在較寬格式，與每步都捨入到 binary32，最後結果可能不同。

例如 binary32 的 $1 + 2^{-24}$ 恰為中點，RNE 先捨成 1，再減 1 得 0；若中間格式能精確保留該和，再減 1，可得 $2^{-24}$。若較早存回 binary32，差異又會消失。

重現數值結果須記錄格式、捨入模式、中間精度、存回時點與編譯選項。此歷史比較不代表所有 x86 程式都固定使用同一種中間精度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.8 三個常見推論，如何用反例檢查？

| 常見推論 | 本章反例或缺少的條件 |
| --- | --- |
| 有進位就有有號溢位 | `0xff+1` 有進位，二補數 −1+1 卻精確為 0 |
| 每輪處理兩位就一定快兩倍 | 每輪延遲、前置處理及修正次數可能增加 |
| 浮點是近似，所以順序不重要 | binary32 的 $(2^{24}+1)-2^{24}=0$，換序可得 1 |

編譯器的 fast-math 選項可能允許重排、改變結合順序或假設沒有特殊值。若程式依賴特定旗標、NaN 或逐步捨入結果，就要先確認允許的轉換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 3.9 一個結果，至少要留下三種證據

| 檢查層次 | 應留下的證據 | 常見漏項 |
| --- | --- | --- |
| 表示 | 輸入寬度、型別與解碼值 | 把二補數當無號數 |
| 運算 | 中間狀態、範圍與一般不變量 | 只有最後答案，沒有進位或餘數 |
| 交付結果 | 截取／捨入、例外、寫回政策 | 精確數學值不在輸出格式中 |

乘除法另須區分回合數、單筆延遲與啟動間隔。浮點數另須檢查正常值、零、subnormal、無窮大、NaN 及中點；只測普通正數不足以驗收。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 格式標準化之後，仍有哪些問題？

IEEE 浮點格式讓位元解碼、捨入與特殊值有共同規則；乘法器的重編碼、部分乘積壓縮、管線化與 SIMD 則改善不同層次的硬體成本。

格式標準不能替代演算法的數值分析。病態問題可能放大輸入誤差，求和樹改變可能改變捨入結果，多執行緒也可能改變運算順序。

因此閱讀硬體效能宣稱時，分別問：使用什麼格式、每次運算遵守什麼規則、單筆與連續多筆花多少時間、整個演算法的誤差如何累積。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 讀完後，試著完成這些檢查

1. 從輸入符號與進位，分別判斷有號及無號加法溢位。
2. 逐輪追蹤乘除法，說明暫存器寬度與一般不變量。
3. 編碼、解碼 binary32，區分正常值、subnormal、無窮大、NaN 與帶符號的零。
4. 從對齊後的 guard、round、sticky 位元推導 RNE 結果。
5. 對五種浮點例外分別寫出結果與旗標，包含精確 subnormal 的反例。
6. 用確切數值說明換結合順序及 FMA 如何改變逐步捨入結果。
7. 在指定時序模型下，分別計算單筆延遲與穩定吞吐量。

### 參考資料與署名

Lin Cing，《計算機組織》手寫筆記。 Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 3. CSO course slides, arithmetic.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

# 從 ISA 契約到資料路徑與控制

Processor implementation 把 ISA 的抽象契約變成 bit-level state transition。ISA 規定指令看到什麼，microarchitecture（微架構）決定用多少 cycles、多少 functional units 與多少 power 達成它。

本章依序建立單週期 MIPS subset、五階 pipeline、hazard control、exception，最後擴到 multiple issue 與 speculative execution。每一步都要能說出資料由哪個 register 送到哪個 unit，以及錯誤的代價是什麼。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



# 4.1 設計問題與範圍

`datapath` 是搬運與運算資料的元件集合，`control` 依 instruction、ALU flags 與外部 request 產生 selector 和 enable signals。state 由 PC（program counter，目前指令位址）、register file 與 memory 等 clocked elements 保存。

本章教學 subset 包含：

1. memory reference：`lw`、`sw`。
2. arithmetic/logical：`add`、`sub`、`and`、`or`、`slt`。
3. control transfer：`beq`、`j`。

一般五階時序題採沒有 architectural delay slot 的簡化 subset，taken/misprediction 直接 squash 所有較年輕錯路徑指令；control hazard 小節另以 classic pre-Release-6 MIPS 的 one-delay-slot ISA 作對照。兩者的 flush 數、`$ra=PC+8` 與 wrong-path instruction count 不可互套。本章仍不是完整 MIPS；`addi`、byte load/store、multiply、floating point 與更完整 exception support 需要更多欄位、單元或狀態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 從功能性到可實作契約

一個 instruction 的實作必須指定：

- source operands 與 destination architectural state。
- instruction 執行時 memory read/write 的數量。
- branch/jump target 的計算方式。
- exception 發生時哪些 register 或 memory write 可以保留。
- back-to-back instructions 對同一 register 的 read/write 關係。

例如 `lw rt, imm(rs)` 先以 `rs + sign_extend(imm)` 算 address，再於 MEM 取得 data，最後在 WB 寫 `rt`。若後一條指令立即讀 `rt`，硬體必須等 data 回來或使用正確的 forwarding path。

把 correctness 寫成 architectural state 的最終可見結果，而不是只看某張 waveform 是否看起來合理。



## 4.2 Clocking 與 edge-triggered state

Combinational logic 沒有內部 clocked state；它的 output 會隨 input 傳播。Edge-triggered register 只在指定 clock edge 取樣，所以一個 cycle 的邏輯不得靠 feedback 穿過多個 state updates。

簡化週期約束可寫成

$$T_{\text{clk}} \geq T_{\text{cq}} + T_{\text{logic,max}} + T_{\text{setup}} + T_{\text{skew}}.$$

`T_cq` 是 clock edge 到 register output 有效的延遲，`T_logic,max` 是最長 combinational path，`T_setup` 是下一 register 的要求，`T_skew` 捕捉 clock 到達差異。hold constraint 由最快路徑限制，不能只用最慢路徑證明正確。

教材若寫「一個 gate 等於 1 cycle」是把時間抽象掉；實體電路驗證要看 process、voltage、temperature corner。

## 4.3 Datapath 的積木

基本 datapath elements 包括 instruction memory、data memory、PC、adder、ALU、register file 與 multiplexor（多工選擇器）。

Register file 可同時讀兩個 registers、寫一個 register。Read 是 combinational；write 在 clock edge 發生。Separate instruction/data memories 讓 IF 與 MEM 在同一個 cycle 發生，但這是教學上的 Harvard-style model。真實 unified cache 可用更多 ports 或拆分 pipeline stages。

32-bit register file 有 32 個 architectural registers。`$zero` 的 read value 固定為 0，對它的 write 被忽略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## ALU 與控制的分層

Main control 解讀 opcode，產生 `RegDst`、`ALUSrc`、`MemtoReg`、`RegWrite`、`MemRead`、`MemWrite`、`Branch`、`Jump` 等 signals。ALU control 再依 instruction class 與 R-type `funct` 欄位選 ALU operation。

以簡化 encoding 為例：

| ALUOp | instruction class | ALU action |
|---|---|---|
| `00` | `lw/sw` | address addition |
| `01` | `beq` | subtraction |
| `10` | R-type | decode `funct` |

控制輸出可由 truth table 或 ROM 實作。Decoder 的 propagation delay 也是 critical path 的一部分。

例如 R-type 使用 MemtoReg=0、RegDst=1、RegWrite=1，將 ALU 結果寫入 rd；sw 也用 ALU 算位址，但 RegWrite=0，結果送到資料記憶體位址端，由 MemWrite 啟用寫入。ALU operation 相同不代表目的地相同。

Lin Cing

# Single-cycle 的資料流

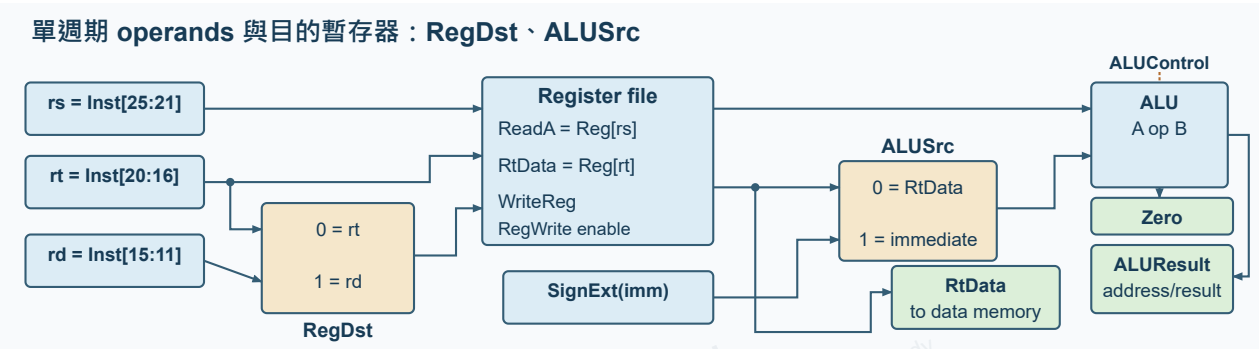
本節是無architectural delay slot的教學subset，支援R-type ALU、lw、sw、beq、j，獨立instruction memory與data memory；讀取為cycle內組合運算，PC與register writes、store在時脈邊緣提交。它沒有多週期的IR/A/B/MDR state，也沒有共用memory所需的IorD selector。

Cycle開始由PC.Q提供instruction address，instruction memory輸出opcode、rs、rt、rd、immediate與jump target fields。Control依opcode與funct產生select/enables；多條路徑同時運算，最後只有被enable的state更新，不是所有instruction都沿PC→IM→Reg→ALU→DM→WB一條直鏈走到底。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



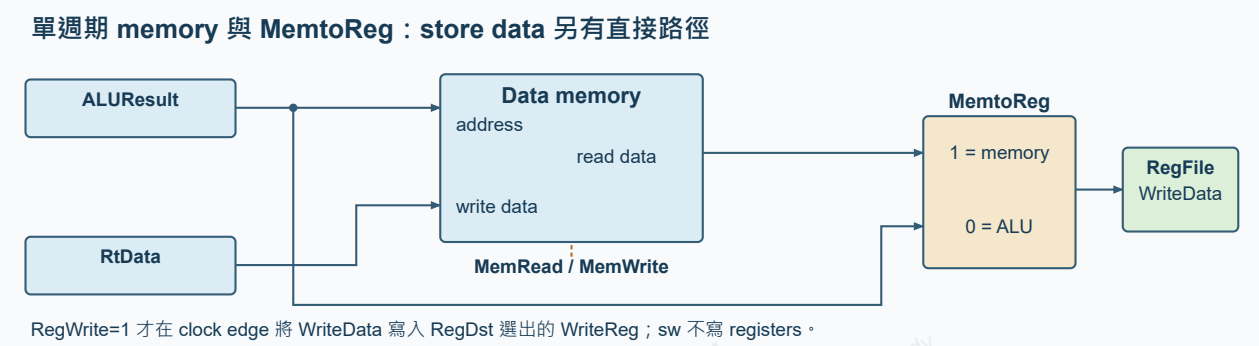
# Single-cycle 的資料流 （續）




RegDst=0選Instruction[20:16]的rt作WriteReg，RegDst=1選Instruction[15:11]的rd。ReadA=Reg[rs]、RtData=Reg[rt]。ALUSrc=0把RtData送進ALU第二輸入，=1改送sign-extended immediate；第一輸入是ReadA。RtData另外直接供sw的memory write data，不能因ALUSrc=1就把immediate當作要儲存的資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 單週期 memory 與 write-back 的分支



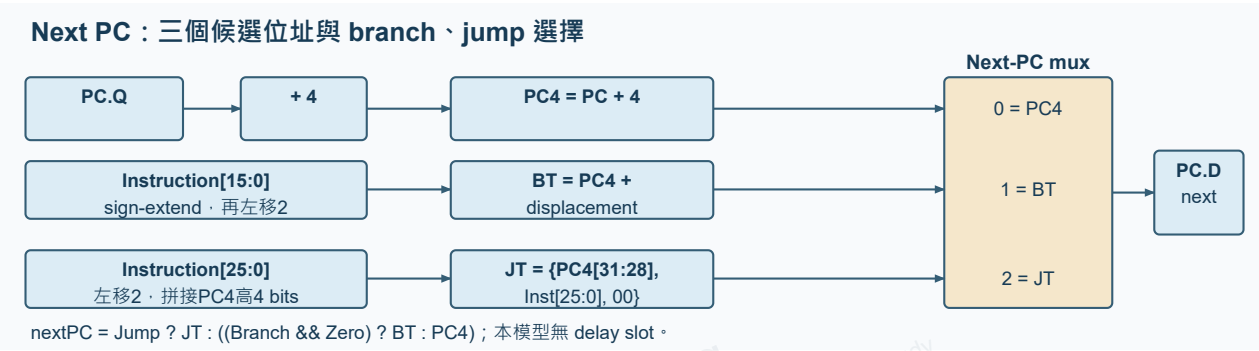

ALUResult對lw/sw是有效位址，對R-type是運算結果。Data memory的write data來自RtData，只有MemWrite=1才在本cycle的提交邊緣寫入。MemRead=1時提供該位址資料給MemtoReg的memory輸入。

MemtoReg=0選ALUResult，=1選memory read data，形成WriteData。只有RegWrite=1，才把WriteData寫入上一圖RegDst選出的WriteReg；目的為$zero時依ISA不改其固定值。圖中相同名稱是同一組資料wire的跨圖端點，不代表額外pipeline registers。

例如 `sw $t2,-4($s0)`，Reg[s0]=0x1000、Reg[t2]=0x12345678，immediate=-4。ALUSrc=1得到address0x0ffc，但store data仍為0x12345678，MemWrite=1、RegWrite=0。若誤把ALU的第二輸入當store data，就會把−4寫入memory。



# 單週期 next-PC 的三個候選




PC4=(PC+4) mod2^32。Branch displacement=(signext(imm)<<2) mod2^32，BT=PC4+displacement；beq以ALU做ReadA−RtData，Zero表示兩個32-bit值相等。Jump的JT拼接PC4[31:28]、Instruction[25:0]、00，不使用PC原值的高4 bits。

選擇函數為 `nextPC = Jump ? JT : ((Branch && Zero) ? BT : PC4)`，可實作成先branch/seq、再jump的兩個2-input mux。下個edge把nextPC提交到PC；本模型不先執行delay slot。圖把相同PC4用名稱接到各計算式，所有計算都屬同一cycle的組合邏輯。

PC=0x00400020、beq immediate=-2時，PC4=0x00400024、BT=0x0040001c；相等則選BT，否則PC4。PC=0x0ffffffc且jump field=0時，PC4=0x10000000，所以JT=0x10000000；若錯用PC高bits會跳到0。

## Control table 與每個 mux 的選擇

| Signal | lw | sw | R-type | beq | j |
|---|---|---|---|---|---|
| RegDst | 0 | X | 1 | X | X |
| ALUSrc | 1 | 1 | 0 | 0 | X |
| MemtoReg | 1 | X | 0 | X | X |
| RegWrite | 1 | 0 | 1 | 0 | 0 |
| MemRead | 1 | 0 | 0 | 0 | 0 |
| MemWrite | 0 | 1 | 0 | 0 | 0 |
| Branch | 0 | 0 | 0 | 1 | 0 |
| Jump | 0 | 0 | 0 | 0 | 1 |
| ALUOp | 00 | 00 | 10 | 01 | X |

ALUOp=00選add用於address，01選subtract以產生Zero，10再由R-type funct選add/sub/and/or/slt等。X是don't-care，只在結果不會被提交的欄位使用；RegWrite/MemWrite不能因其他欄位不用而隨意assert。非法opcode與例外還需額外控制，本subset不假裝已涵蓋完整MIPS。



## 五類 instruction 的完整 state effects

取每一列獨立的cycle，PC=0x00400020、Reg[s0]=0x1000、Reg[s1]=7、Reg[s2]=5、Reg[t2]=0x12345678，Mem[0x100c]=0xabcdef01。Word accesses對齊，運算不產生本subset未處理的overflow。

| Instruction | 關鍵資料路徑 | 本cycle提交 |
|---|---|---|
| addu t0,s1,s2 | ReadA7、RtData5；ALUSrc0；ALU12；MemtoReg0；RegDst1 | t0=12，PC=0x00400024 |
| lw t1,12(s0) | ReadA0x1000、imm12；ALU0x100c；MemtoReg1；RegDst0 | t1=0xabcdef01，PC=0x00400024 |
| sw t2,-4(s0) | ALU0x0ffc；RtData0x12345678直入memory | Mem[0x0ffc]=0x12345678，PC=0x00400024 |
| beq s1,s2,-2 | ALU7−5=2，Zero0；Branch1，Jump0 | 無register/memory writes；PC=0x00400024 |
| j 0x00401000 | field0x00100400；Jump1 | 無register/memory writes；PC=0x00401000 |

若只把beq輸入改成兩者皆5，Zero=1，PC改為0x0040001c。Lw的rt欄位是目的暫存器，sw/beq的rt則是來源；不能把相同欄位名稱當作永遠同一讀寫用途。

## 單週期最慢路徑決定共同時脈

lw通常走PC clock-to-Q、instruction read、register read、ALU address、data-memory read、MemtoReg及register setup，因此

$$T_{clk} \geq T_{PCQ} + T_{IM} + T_{regread} + T_{ALU} + T_{DM} + T_{mux} + T_{setup}.$$

明定這些已合併的delay項為30+200+120+180+250+20+40 ps，得到840 ps。R-type的此模型路徑30+200+120+180+40=570 ps，仍需等待同一840 ps clock。這個簡化delay預算忽略的wire/control/mux成本必須已可被所用項目涵蓋，不能再聲稱是某個實際晶片的完整timing closure。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 單週期效能與限制

以下是與前頁 840 ps 延遲預算不同的獨立模型，假設 clock period 為 800 ps，程式有 60% R-type、25% load/store、15% branch/jump，且 single-cycle CPI 對每條 instruction 都是 1。CPU time 為

$$T = IC \times CPI \times T_{\text{clk}} = IC \times 1 \times 800\,\text{ps}.$$

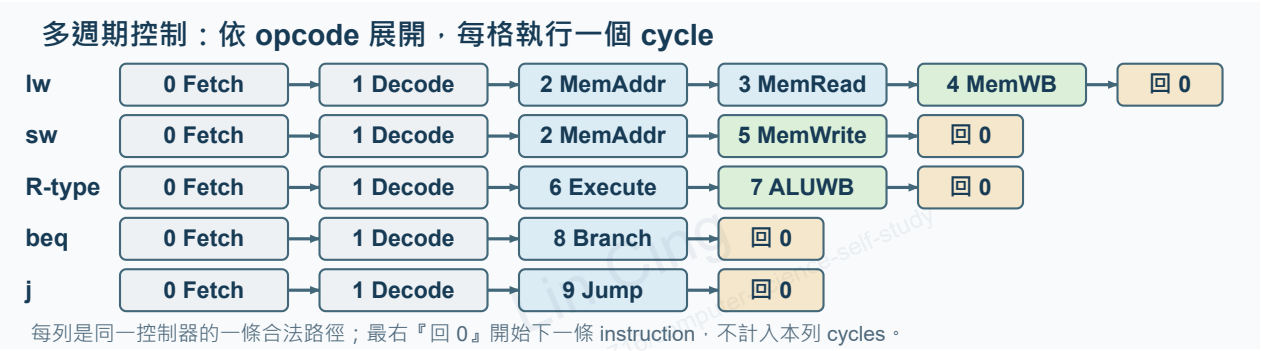
最佳化只縮短 R-type path 不會降低 clock period，因為 `lw` 仍決定週期。反之，任何只增加 `lw` path 的 feature 會拖慢所有 instructions。

Single-cycle 適合小教學 ISA，實際處理器常用 pipelining、multi-cycle control 或 asynchronous/clock-gated 方法控制資源與時序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 多週期資料路徑與控制

多週期設計把 instruction 拆成若干 state，每個 clock edge 只讓一組資料動作的結果進入暫存 state。暫存器 `IR`、`A`、`B`、`ALUOut` 與 `MDR` 分別保存 instruction、兩個 register operands、ALU result 與 memory data；`PC` 仍然保存 architectural instruction address。




| Instruction | State path | Cycles |
|---|---|---|
| R-type | Fetch → Decode → Execute → ALUWB → Fetch | 4 |
| `lw` | Fetch → Decode → MemAddr → MemRead → MemWB → Fetch | 5 |
| `sw` | Fetch → Decode → MemAddr → MemWrite → Fetch | 4 |
| `beq` | Fetch → Decode → Branch → Fetch | 3 |
| `j` | Fetch → Decode → Jump → Fetch | 3 |



# 多週期資料路徑與控制 （續）

表中每一個 state 各占一個 cycle；最後的 Fetch 是下一條 instruction 的第一個 cycle。`sw` 從 MemAddr 直接到 MemWrite，不經 MemRead。

| State | 本次 clock edge 保存的資料或提交的效果 | 關鍵 write enable／選擇 |
|---|---|---|
| 0 Fetch | IR←Memory[oldPC]；PC←oldPC+4 | IorD=0；MemRead=1；IRWrite=1；PCWrite=1 |
| 1 Decode | A←Reg[rs]；B←Reg[rt]；ALUOut←PC+branch displacement | 不寫 architectural registers／memory |
| 2 MemAddr | ALUOut←A+signext(imm) | ALU 選 base 與 immediate |
| 3 MemRead | MDR←Memory[ALUOut] | IorD=1；MemRead=1 |
| 4 MemWB | Reg[rt]←MDR | RegDst=0；MemtoReg=1；RegWrite=1 |
| 5 MemWrite | Memory[ALUOut]←B | IorD=1；MemWrite=1 |
| 6 Execute | ALUOut←A op B，op 由 funct 決定 | ALUOp 選 R-type 解碼 |

## 多週期資料路徑與控制 （續）

| State | 本次 clock edge 保存的資料或提交的效果 | 關鍵 write enable／選擇 |
|---|---|---|
| 7 ALUWB | Reg[rd]←ALUOut | RegDst=1；MemtoReg=0；RegWrite=1 |
| 8 Branch | A=B 時 PC←先前保存的 branch target | PCWriteCond=1；PCSrc=branch；ALU 比較 A−B |
| 9 Jump | PC←{當前PC[31:28],IR[25:0],00} | PCWrite=1；PCSrc=jump |

未列出的 architectural write enable 均為0；內部暫存器的寫入依本表模型。Decode 的 opcode 分流為 lw/sw→2、R-type→6、beq→8、j→9，其他 opcode 進入明定的 error transition。狀態3只通往4，不通往5；兩條 memory 路徑分別是讀取後寫回暫存器，或直接寫入記憶體。

關鍵 enable 不是裝飾文字。Fetch state 可 asserted `IRWrite`、`PCWrite`、`IorD=0`；Memory state asserted `IorD=1` 與 `MemRead` 或 `MemWrite`；Write-back state 才 asserted `RegWrite`。Fetch 開始令 oldPC=PC；Fetch 的同一時脈邊緣寫入 IR=Memory[oldPC] 與 PC=oldPC+4。Decode 讀 rs、rt 到 A/B，以已增加的當前 PC 計算 ALUOut=PC+(signext(imm)<<2)，不再把 PC 加4。Branch 相等時才令 PC=ALUOut，不相等保留循序 PC。例 oldPC=0x00400008、imm=-2，Fetch 後 PC=0x0040000c，target=0x00400004；重複加4會錯成0x00400008。Branch/Jump state 選 next-PC source，不能和一般 `PC+4` update 同時無條件 asserted。



## 多週期資料路徑與控制 （續）

逐週期 CPI 必須由 instruction mix 計算。以 45% R-type、20% branch、20% load、15% store 為例：

$$\mathrm{CPI} = 0.45(4) + 0.20(3) + 0.20(5) + 0.15(4) = 4.0.$$

若每 cycle 300 ps，執行 $10^9$ instructions 的時間是

$$T = 10^9 \times 4.0 \times 300\,\mathrm{ps} = 1.2\ \mathrm{s}.$$

同一個假設下，single-cycle 用 800 ps 且 CPI=1，時間是 0.8 s。多週期雖可使用較短 clock，但本題的額外 cycles 超過 clock 縮短比例，因此不一定較快。這個結論只在題目給的元件延遲與 instruction mix 成立；不能把多週期或 pipeline 當成無條件加速器。

FSM 正確性的兩個不變量是：一個 instruction 在同一個 state 不會重複提交 `RegWrite` 或 `MemWrite`；從任何 completion state 回到 Fetch 前，該 instruction 所需的 operand/result state 都已保存。未定義 opcode 要有明確 error transition，不能默默走 default path 並執行任意 memory write。

## 4.5 五階管線模型

五階 pipeline 為：

1. IF：以 `PC` 取 instruction，並計算 `PC+4`。
2. ID：decode、讀 register file，產生 immediate 與 hazard/control 所需欄位。
3. EX：ALU 計算、load/store effective address 或 branch comparison。
4. MEM：執行 data-memory read/write。
5. WB：依 `RegWrite` 寫回 register。

每個 pipeline register 是 clock edge 更新的 state snapshot，不是單純導線：

| Register | 必要 state |
|---|---|
| `IF/ID` | `valid`、該 instruction 的 `PC`、fetched instruction、`PC4=PC+4` |
| `ID/EX` | `valid`、該 instruction 的 `PC/PC4`、`rs/rt/rd`、A/B values、sign-extended immediate、control bits |
| `EX/MEM` | `valid`、該 instruction 的 `PC`、ALU result、store data、destination、`MemRead/MemWrite/RegWrite/MemtoReg` |
| `MEM/WB` | `valid`、該 instruction 的 `PC`、memory read data 或 pass-through ALU result、destination、`RegWrite/MemtoReg` |



## 4.5 五階管線模型 （續）

`EX/MEM` 若漏帶 `RegWrite` 或 destination register，後面的 WB 就無法知道誰要寫哪個 register。`MemRead` 也不是立即完成：load 在 EX 只產生 address，value 要等 MEM 尾端。這個資料相依是 load-use hazard 的來源。

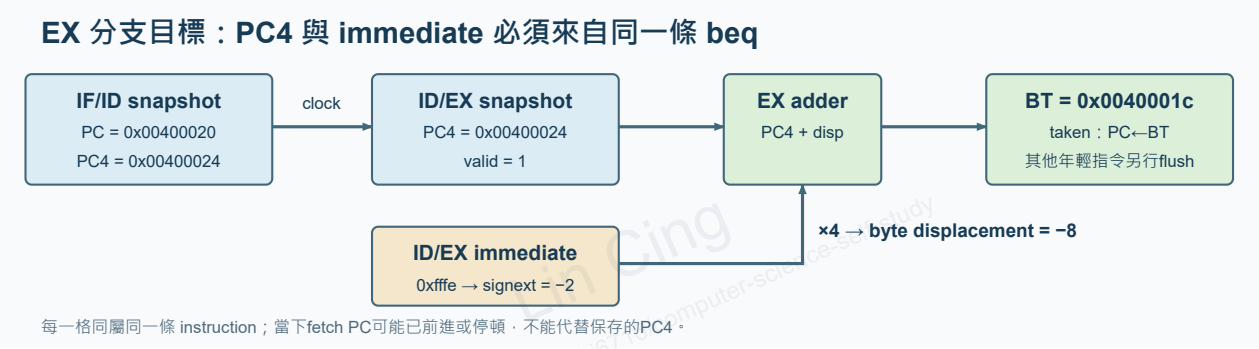
`PC/valid` 與 operand/control 同屬一條 instruction 的 state，必須一起保持或前進；bubble/flush 令被清除項的 `valid=0`，所有 register/memory writes 都須受 `valid` 門控。Stall 保持 PC 與 IF/ID，在 ID/EX 插入無效項，較舊的有效項仍前進。每階保留自己的 instruction PC，才可在 EX arithmetic fault 或 MEM access fault 記錄正確 EPC；此處顯式存 PC 與 PC4 是教學選擇，也可保存 PC4 並以減4還原 PC。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## EX 的 branch target 從哪裡來

本模型在 EX 判斷 beq，所以 IF 算出的 PC4 必須先經 IF/ID、再經 ID/EX，不能用當下已在 fetch 年輕指令的全域 PC。32-bit 位址運算為

$$BT = \left(\mathrm{ID/EX.PC4} + 4\,\mathrm{signext}_{16\to 32}(imm_{16})\right) \bmod 2^{32}.$$




例 beq 位於 `0x00400020`、immediate bits=`0xfffe`。該指令的 PC4=`0x00400024`，signed immediate=−2、byte displacement=−8，故 BT=`0x0040001c`。Taken 時下一個 PC 選 BT；not-taken 繼續既定順序路徑。保存的 PC4 不因較年輕指令的 stall/flush 改變，不能以「目前 fetch PC 減固定常數」替代。



## 無 hazard 時的理想時間

若 stage 完全平衡且 latency 為 $t$，沒有 hazard 的 $n$ 條 instructions 完成時間為

$$C_{\mathrm{ideal}} = k + (n-1), \qquad k = 5.$$

Steady state 每個 cycle 完成一條 instruction，但單筆 latency 仍是 $5t$。例如 $n = 100$ 時是 104 cycles，理想 IPC 接近 1。以下長程穩態模型以退休指令數為分母，且各 penalty 是不重疊、不重複計數的實際額外週期：

$$\mathrm{CPI}_{\mathrm{pipe}} = 1 + \mathrm{stall}_{\mathrm{loaduse}} + \mathrm{mispredict_cycles} + \mathrm{structural_bubbles}.$$

## Pipeline 的理想時間

若 stages 完全平衡，stage latency 是 $t$，執行 $n$ 條 instructions 的理想 cycles 為

$$k + (n - 1),$$

其中 $k = 5$。Steady-state throughput 是每個 cycle 完成一條 instruction，但單筆 latency 仍是五個 stages。

以 $n = 100$：理想 cycles 為 $5 + 99 = 104$。加速比相對於 ideal single-cycle 為

$$\frac{500}{104} \approx 4.81.$$

實際 single-cycle 的 stage delay 不一定可比，因此這只是相同 stage 假設下的比較。

在本例五級等延遲且忽略 pipeline-register overhead 的模型中，指令數趨於無窮時 speedup 接近 5；單一指令仍依序通過五級，其 latency 並未縮短。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Structural hazard

Structural hazard 發生於兩個同時活躍的 instructions 競爭同一個硬體資源。

五階設計通常把 instruction memory 與 data memory 分開，避免 IF 與 MEM 共用單一 memory port。Register file 的 write 在 cycle 前半或後半、read 在 cycle 前半或後半，可讓同一 cycle 的不同 instructions 合法共享；這需要明確 timing convention。

Register file 通常支援兩個 reads 與一個 write；需要第四個 operand 的 instruction 或更多 concurrent accesses 會增加 ports 與面積。

改 higher associativity、更多 ALUs 或更多 register ports 不是免費；先找出 resource conflict 的頻率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Data dependence

Data dependence 是程式資料流，例如

```
add $8, $9, $10
sub $11, $8, $12
```

`sub` 需要 `add` 產生的 `$8`。True dependence（真相依）不可消除；可透過 forwarding、scheduling 或更多 state 隱藏。

Anti-dependence 與 output dependence 是名稱重用造成的 ordering constraint，例如

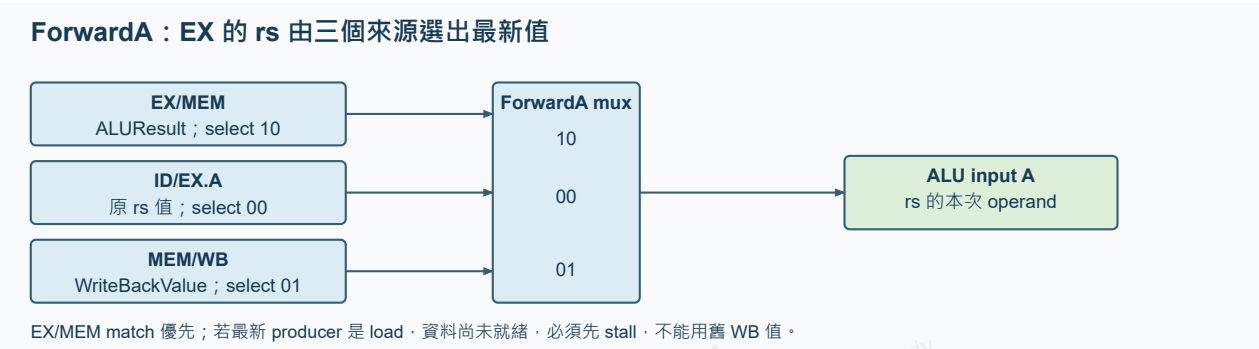
```
add $8, $9, $10
sub $8, $11, $12
```

Register renaming 可把兩個 `$8` 映射到不同 physical registers，消除名稱衝突。

ISA-visible register 名稱相同不等於真資料依賴的 value 是由哪個 instruction 產生。



## Forwarding 的 correct path




Forwarding（前送）把 EX/MEM 或 MEM/WB 的 produced value 繞回 ALU input，避免等 WB。

對 `add $8, $9, $10` 後接 `and $4, $8, $5`，EX/MEM 在下一 cycle 已持有 `$8` result，可用 `ForwardA=EX/MEM`。

判定必須檢查：

- producer 的 `RegWrite` 已 asserted。
- producer destination 非 `$zero`。
- consumer 的 `rs` 或 `rt` match 該 destination。
- 對 load，EX stage 只有 address，沒有 data；不能把 address 當 loaded value 前送。

## Forwarding 的 correct path （續）

對每個在EX使用的source register s，令 E 表示 `EX/MEM.RegWrite && EX/MEM.rd!=0 && EX/MEM.rd==s`，W 表示 `MEM/WB.RegWrite && MEM/WB.rd!=0 && MEM/WB.rd==s`。Mux有三個實際輸入：ID/EX保存的register值、EX/MEM的ALU結果、MEM/WB最終write-back值。選取規則如下。

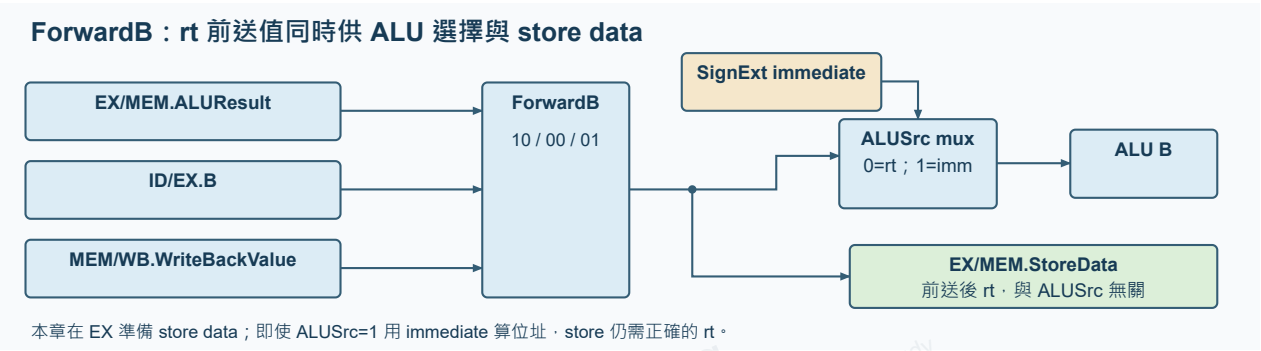
| 條件 | 選入ALU的值 |
|---|---|
| E且producer不是load | EX/MEM.ALUResult，編碼10 |
| 非E且W | MEM/WB.WriteBackValue，編碼01 |
| 非E且非W | ID/EX原值，編碼00 |
| E且producer是load | 資料尚未產生，consumer必須先stall，不能前送address或較舊WB值 |

ForwardA對rs套此規則。ForwardB對作為register operand的rt套此規則，再由ALUSrc mux決定ALU的B輸入是此前送值或sign-extended immediate。Store data也要保存/前送正確rt值。本模型在EX準備store data，所以把緊接load的store-data consumer也保守stall一cycle；另設MEM-stage store forwarding可省此stall，但須另外給路徑與條件。

當連續兩個producer都寫s，EX/MEM是較新的定義，必須優先於MEM/WB。例如 `add $1,...; sub $1,...; and $2,$1,$3`，and需要sub結果，不能使用add的舊結果。

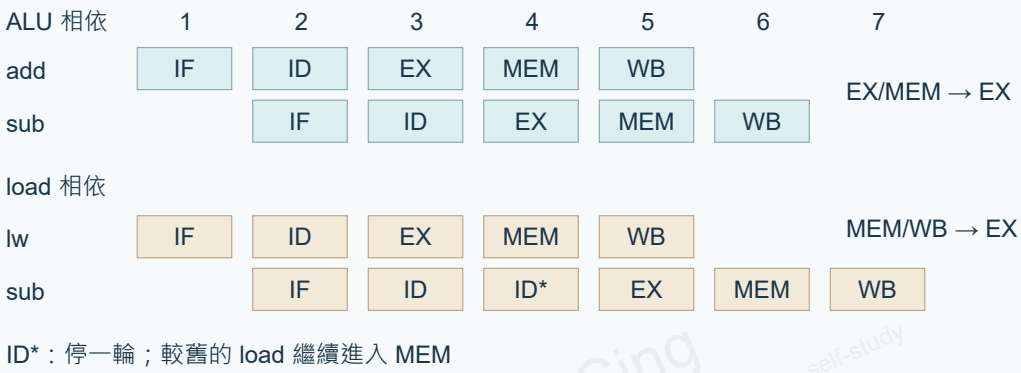


## Store data 的前送與 ALUSrc




ALUSrc只選擇ALU第二輸入，不決定store寫入memory的資料。Store的address採base加immediate，仍需以最新rt值作data。圖中的分叉點同時送往ALUSrc的register輸入與EX/MEM.StoreData；兩個用途不能混成同一個mux選擇。

## 兩種相依：為何一個要停、一個不用？




假設五階單發射、單週期記憶體、EX/MEM 與 MEM/WB 可前送到 EX，沒有其他 hazard。上例 `add $s0,$t0,$t1; sub $t2,$s0,$t3`：加法在 C3 EX 結束產生結果，C4 從 EX/MEM 前送至 sub 的 EX，無須 stall。

下例 `lw $t0,4($s0); sub $t2,$t0,$s1`：資料在 C4 MEM 結束才就緒，不能在同一 cycle 一開始供 EX。凍結 PC 與 IF/ID 一輪並插入 bubble，sub 在 C5 EX 從 MEM/WB 取得資料。



## Load-use stall

```
lw  $2, 20($1)
and $4, $2, $5
```

`lw` 在 MEM 才取得 data，`and` 下一 cycle 就要在 EX 使用 `$2`。僅有 EX/MEM forwarding 仍沒有 loaded data，因此必須 stall 一 cycle。

Stall 時應：

1. PC 保持原值。
2. IF/ID 保持原值。
3. ID/EX control 清成 no-op，插入 bubble。
4. 讓 `lw` 繼續進 MEM/WB，下一 cycle 前送 loaded value。

若下一條 instruction 不讀 `$2`，不必 stall。比對 `rt` 只適用於會讀 `rt` 的 instruction；`lw` 的 `rt` 是 destination，不能一律當 consumer。

令 `use_rs/use_rt` 表示IF/ID指令是否在EX需要該source，包括本模型在EX準備的store data。R-type與beq兩者皆1；lw只有use_rs=1；sw兩者皆1；j兩者皆0。對這個subset，load目的暫存器是ID/EX.rt。完整停頓條件為：

## Load-use stall （續）

```
stall = ID/EX.MemRead && ID/EX.rt != 0 &&
        ((use_rs && ID/EX.rt == IF/ID.rs) ||
         (use_rt && ID/EX.rt == IF/ID.rt))
PCWrite = !stall
IFIDWrite = !stall
IDEXControls = stall ? all_zero : decoded_controls
```

`lw $2,0($1); lw $2,4($3)`不需要stall，因為第二個lw不讀rt。`lw $0,0($1)`不產生可見register結果，所以不因目的為0形成data hazard；memory fault仍須依例外規則處理。一般ISA還需對每個opcode定義source用途與使用stage，不能直接把全部instruction fields都當sources。



## 逐 cycle 數例：舊值不能代替尚未就緒的新值

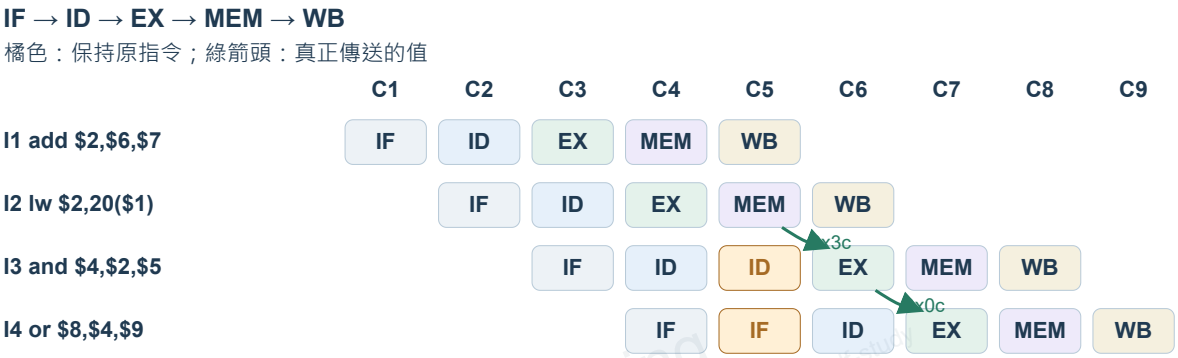
沿用五階、單發射、順序執行的模型；無 cache miss、分支或結構衝突。ALU 結果可由 EX/MEM 或 MEM/WB 前送至 EX；load 資料在 MEM 結束才可供下一 cycle 使用。Register file 在 WB 的前半個 cycle 寫入，ID 後半個 cycle 讀取。

初值：`$1=0x1000`、`$6=0x30`、`$7=0x03`、`$5=0x0f`、`$9=0x80`，且 `Mem[0x1014]=0x3c`；其餘相關暫存器為 0。所有存取對齊且無例外。

```
I1  add $2,$6,$7     # $2 ← 0x33
I2  lw  $2,20($1)    # $2 ← 0x3c，覆寫 I1 的值
I3  and $4,$2,$5     # $4 ← 0x0c
I4  or  $8,$4,$9     # $8 ← 0x8c
```

`I3` 必須讀到最近一次寫入 `$2` 的 `I2`，不是更早的 `I1`。相依關係是 `I2 → I3 → I4`；`I1` 與 `I2` 的目的暫存器相同，但 `I2` 不讀 `$2`，不構成兩者之間的 RAW 相依。

# 九個 cycles：停住前端，較舊指令繼續前進




C4：偵測相依 → C5：EX bubble（valid=0）；I3 留在 ID，I4 留在 IF。
C6：MEM/WB → I3.EX；C7：EX/MEM → I4.EX。I4 在 C9 完成。

每格表示該 cycle 內所處的 stage，cycle 結束才跨越 pipeline register。C4 偵測到 `I2.EX` 的 load 與 `I3.ID` 相依；在 C4 結束的 edge 保持 PC、IF/ID，並向 ID/EX 寫入無效項。因此 C5 沒有有效指令在 EX，並非把 load 停在原處。



# 從時序圖核對控制訊號與數值

| Cycle | 指令位置與可用值 | 控制或運算結果 |
|---|---|---|
| C4 | I2.EX 算出位址 `0x1014`；I3.ID 需要 `$2` | `PCWrite=0`、`IFIDWrite=0`；下一個 ID/EX 為無效項 |
| C5 | I1.WB 寫 `$2=0x33`；I2.MEM 在末端取得 `0x3c`；I3 仍在 ID | I3 讀到並暫存舊值 `0x33`；此時 hazard 已解除，前端可在 edge 前進 |
| C6 | I2.WB 的值為 `0x3c`；I3.EX | `ForwardA=01` 選 MEM/WB，覆蓋暫存舊值；`0x3c AND 0x0f=0x0c` |
| C7 | I3.MEM 帶著 ALU 結果 `0x0c`；I4.EX | `ForwardA=10` 選 EX/MEM；`0x0c OR 0x80=0x8c` |

若略過 C5 的 bubble，I3 會在 C5 的 EX 過早取值：最新 producer 的 EX/MEM 只有 load **位址** `0x1014`，MEM/WB 的 `0x33` 則是舊版本。兩者都不能代替尚未產生的 load 資料；錯用舊值會得到 `$4=0x03`。

## 有、無 forwarding：比較的是同一段程式

| 模型 | I3 的 EX | I4 的 EX | 最後 WB | 額外停頓 |
|---|---|---|---|---|
| 有 EX/MEM 與 MEM/WB 前送 | C6 | C7 | C9 | 1 cycle |
| 完全無前送，只能從 register file 讀 | C7 | C10 | C12 | 4 cycles |

無前送時，I3 在 C4、C5 的 ID 被擋住，直到 C6 的 WB 寫入新 $2 後才能讀取；I4 則在 C7、C8 的 ID 等待，直到 C9 的 WB 寫入 $4。由於允許同 cycle 先寫後讀，不再額外等待一個 cycle。

四條有效指令的理想完成時間為 4+4=8 cycles；上述兩模型分別為 8+1=9 與 8+4=12。有限序列的 CPI 為 9/4 與 12/4，包含填滿與排空成本，不能把它直接當成長程式穩態 CPI。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Control hazard、branch delay 與 flush

Branch 改變 fetch path。五階 pipeline 在 branch 尚未到 EX 前，IF/ID 已抓入較年輕 instructions；若 taken 且 prediction 是 not taken，就必須丟棄 wrong-path instructions。

branch penalty 由 resolve stage 與 predictor 決定，不等於 architectural delay slot。若要估計 per-instruction cycle cost，可寫成

$$\text{mispredict_cycles} = f_{\text{branch}} \sum_{r} P(\text{wrong}_r \mid r) C_r.$$

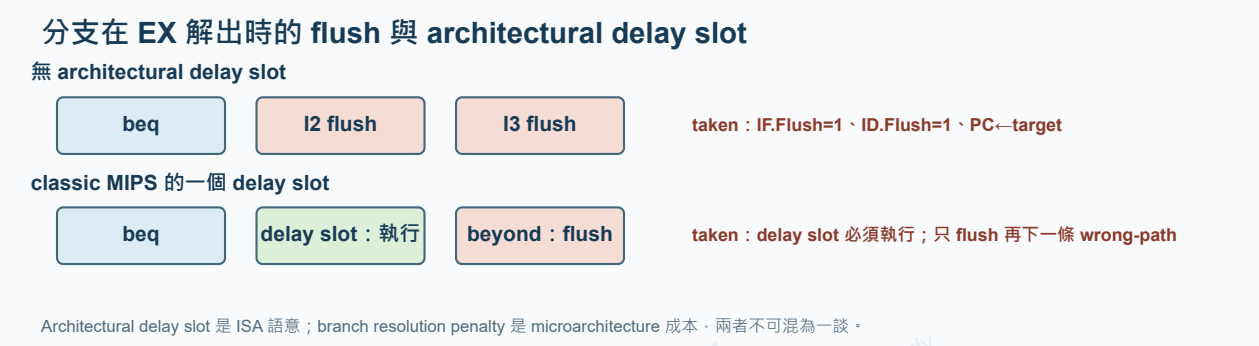
例如 20% instructions 是 branches，其中 60% taken，predictor 一律 not taken，且 taken 在 EX 解出後 flush 兩個 younger instructions：

$$0.20 \times 0.60 \times 2 = 0.24 \,\text{cycles/instruction}.$$

Classic MIPS 的架構另有一個 branch delay slot。beq 後緊接的 instruction 無論 taken 與否都屬於 architectural execution；taken 時只 flush delay slot *之後* 的 wrong path。若在 EX 解出時，delay slot 在 ID，下一條在 IF：

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Control hazard、branch delay 與 flush （續）




No-delay-slot model 需 flush ID 與 IF 的兩個 younger instructions；one-delay-slot model 只 flush delay slot 之後的 IF instruction，slot 必須照 ISA 語意執行。不能把「flush 幾個 stages」和「delay slot 是否執行」混成同一條規則。

預測器可以早於 EX 做 direction guess，但事件順序仍是 prediction、fetch、resolve、misprediction recovery。BTB hit 只表示有 target 可用，不表示 direction 一定正確。



## Pipelined CPI 的完整邊界

以同樣 20% branch、60% taken、one-cycle load-use bubble 出現於 10% instructions 的模型：

$$\mathrm{CPI} = 1 + 0.20(0.60)(2) + 0.10 = 1.34.$$

這個 1.34 只描述題目明確列出的 penalties。Structural hazard、cache miss、branch target miss、long-latency functional unit 與 front-end limit 都必須各自加入，不能以「五階所以 CPI=1」抹去。

## Dynamic branch prediction

2-bit saturating counter 可抵抗單次雜訊。狀態例如 strongly not taken、weakly not taken、weakly taken、strongly taken；實際 outcome 為 taken 時向 strongly taken 更新，not taken 時向 strongly not taken 更新；更新方向不由本次預測正確或錯誤單獨決定。

Correlating predictor 以 global history 預測具相關性的 branches。Tournament predictor 依不同 predictor 的歷史表現選擇。

Branch target buffer（BTB，分支目標緩衝）快取 target；prediction 仍要同時判斷是不是 branch 以及 taken direction。BTB miss 不等於 direction prediction 錯。

測試需覆蓋 cold start、alternating pattern、correlated branches 與 alias。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 4.9 Exception 與 interrupt

Exception 是由目前 instruction 引起的同步事件，例如 overflow 或 undefined instruction。Interrupt 常指外部非同步事件，例如 I/O request。不同 ISA 的命名不盡相同。

Pipeline exception handling 要保存 faulting PC、cause 與必要 address，跳至 handler。Handler 修復或終止 process 後，若可恢復則由 exception return instruction 回到 architecture-defined PC。

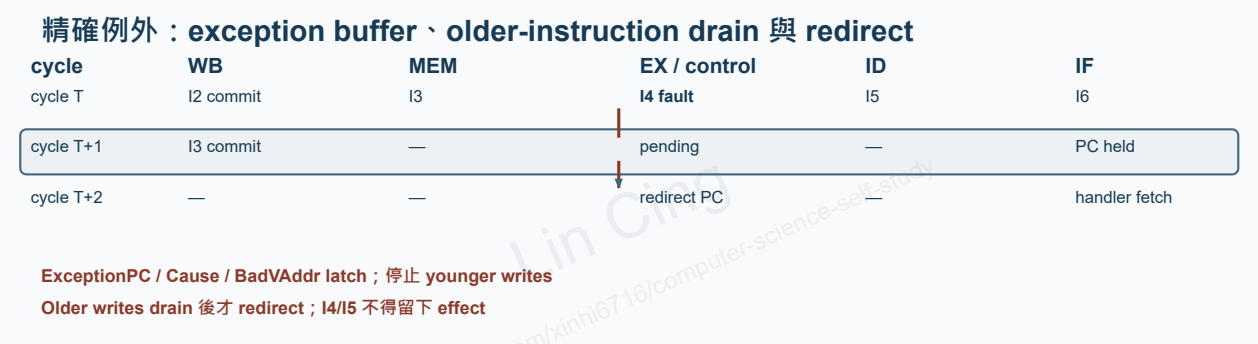
Handler 執行前的 architectural state 必須符合 ISA 規定。若較早 instruction 已 commit、較晚 instruction 被 flush，程式仍可建立單一一致的 interrupted state。

先分清 precise exception 與只保證 imprecise machine check 的差異。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Precise exception 的週期與狀態

在單發射五階 pipeline，令 faulting I4 在 EX 發現 arithmetic overflow。此 cycle 的合法 stage snapshot 是 I2 在 WB、I3 在 MEM、I4 在 EX、I5 在 ID、I6 在 IF；I1 已在更早 cycle commit。I2、I3 沒有 exception；本例的 I3 是在 WB 提交暫存器結果的非 store 指令；I4 與較年輕的 I5、I6 必須阻止 architectural writes。




| Cycle | WB | MEM | EX / control | ID | IF |
|---|---|---|---|---|---|
| T | I2 commit | I3 | I4 arithmetic fault | I5 | I6 |
| T+1 | I3 commit | — | exception pending | — | PC held |
| T+2 | — | — | PC←handler，送出保存的 cause | — | handler fetch |



# Precise exception 的週期與狀態 （續）

本表以每 cycle 開始後的 stage 活動為準，commit 在該 cycle 尾完成：I3 在 T+1 尾提交後，controller 在 T+2 開始選 handler PC，首條 handler 指令於 T+2 的 IF 取得，尚未進入 ID。

在 T 的同 cycle control 中，硬體記下 ExceptionPC=I4.PC、cause 與依例外類型必要的位址（算術 overflow 不一定有 bad address），deassert I4 與 younger writes，並 flush I5、I6。Precise exception 的契約是 handler 看到的所有 architectural effects 等於依序執行完 I1、I2、I3，且 I4 與更晚的 register/memory writes 完全不出現。為了讓較舊的 I2、I3 先完成，controller 可先把 exception 放進 exception buffer，停止 fetch 並 drain older stages；I3 在 T+1 commit 前，handler 不能先看到它的 effects。

若有多個 exceptions 同 cycle，必須由 spec 指定 priority。通常採用最舊 instruction 的 synchronous exception；但 asynchronous interrupt、machine check 與 precise/imprecise policy 可能不同。Handler 入口前也要保存/切換 privilege、status、EPC、cause；僅在例外類型需要時保存 bad-address registers。

Out-of-order execution 可用 ROB 完成相同效果：年輕 instruction 可先執行，但 commit 必須依 program order。I4 成為 ROB head 且之前 I1 至 I3 都 commit 後，才丟棄 I4 與所有較年輕 entries，再清空 speculative rename mapping 並 redirect。

## 4.10 ILP 與 multiple issue

Instruction-level parallelism（ILP，指令層平行）是同一執行流中可重疊的 independent instructions。Pipeline 在 stages 間重疊 instruction；multiple issue 在同一 cycle 啟動多條。

Static multiple issue 由 compiler 把 instructions 排成 issue packets。VLIW（very long instruction word，超長指令字）把多個 operations 編碼在 fixed packet，硬體控制較簡單，binary portability 取決於 ISA/model。

Dynamic multiple-issue 由硬體在 runtime 從 instruction window 選 ready instructions。它可跨越 compiler 未預期的 latency，但需要 scoreboard、wakeup/select、ROB 等大量狀態。

峰值 IPC 4 表示理想寬度，不等於所有程式都能維持 4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Superscalar 與 dynamic scheduling

Superscalar（超純量）在一個 cycle 嘗試 fetch/decode/issue 多條 instructions。Issue logic 檢查 operand readiness 與 functional-unit availability；out-of-order execution 可讓後續 ready instruction 先執行。

Speculation 在 branch outcome 未知時沿預測路徑執行。錯誤預測時，ROB 或 checkpoint 丟棄 younger results，register mapping 回到正確狀態。

Register renaming 把 ISA registers 映射到更多 physical registers，消除 false dependencies。它不改變 architectural register 的可見數量。

OoO execution 不等於 OoO commit。很多處理器以 in-order commit 保存 precise exception。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 亂序執行、rename 與 ROB 的逐 cycle 追蹤

固定單一 dispatch/cycle、單一 commit/cycle，依 program order 配置 ROB 與新 physical destination；每條指令最早在 dispatch 的下一 cycle 開始執行。獨立 load unit latency=4 cycles，pipelined integer ALU latency=1 cycle。結果在 cycle 結束可用，只能從下一 cycle 供 consumer 或 commit。無 cache miss、branch、fault 與其他 structural hazard；ALU 不溢位。

```
I1: lw  r1,0(r2)
I2: add r3,r1,r4
I3: add r5,r6,r7
I4: add r1,r8,r9
```

初始 r1/r2/... 對應 p1/p2/...。Rename 逐條配置 I1→p11、I2→p12、I3→p13、I4→p14。I2 在 dispatch 時已把 r1 source 綁為 p11；I4 雖把 speculative r1 mapping 改成 p14，也不能改寫 I2 保存的 source tag。RAW I1→I2 仍存在；I1/I4 的同名 destination 與 I2/I4 的名稱衝突由不同 physical registers 分開。



# 亂序執行、rename 與 ROB 的逐 cycle 追蹤 （續）

**執行結果可亂序，commit 仍依 I1 → I2 → I3 → I4**

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| **I1: lw → p11** | D | LD1 | LD2 | LD3 | LD4 | C | | | |
| **I2: add → p12** | | D | wait | wait | wait | EX | C | | |
| **I3: add → p13** | | | D | EX | ready | ready | ready | C | |
| **I4: add → p14** | | | | D | EX | ready | ready | ready | C |

D：dispatch；EX/LD：執行；wait：等operand；ready：等ROB head；C：commit

I2 source = p11 (I1); I4 destination = p14. Result end of cycle → use next cycle.

| 指令 | Dispatch | 執行 | 結果 ready（cycle 尾） | Commit | 原因 |
|---|---|---|---|---|---|
| I1 | 1 | load cycles2–5 | 5 | 6 | ROB head ready後才能提交 |
| I2 | 2 | EX cycle6 | 6 | 7 | 讀 p11，必須等 I1 結果 |
| I3 | 3 | EX cycle4 | 4 | 8 | ready仍等較舊I1/I2提交 |
| I4 | 4 | EX cycle5 | 5 | 9 | 新p14不覆寫I1的p11 |

## 亂序執行、rename 與 ROB 的逐 cycle 追蹤 （續）

例如取 memory[0(r2)]=10、r4=1、r6=2、r7=3、r8=4、r9=5，I3先算得p13=5，I4算p14=9，I1算p11=10，I2最後讀p11得p12=11。最終 architectural r1=9、r3=11、r5=5，等同程式順序；若I2錯讀p14，r3會錯成10。Ready、executed與committed是三種不同狀態。

另取 r4=2147483647、保持 I1 的 load 值10，I2 的 signed add 在 EX cycle6 發現 overflow；cycle6仍可先commit較舊I1，I2記下fault但不commit。輪到I2為ROB head時，捨棄I2/I3/I4，恢復到committed rename map，r1對應p11而非p14；handler看到r1=10，r3/r5仍是執行本段前的architectural值。已算出的p13/p14沒有對程式提交，不能以「先執行完」為由保留。

這個模型未處理 speculative stores、memory disambiguation或分支checkpoint；加入它們時還需 store queue、例外與恢復規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 多發射資源邊界

設理想每 cycle 可 issue 4 instructions，但程式中只有 20% 的相鄰指令 independent，其他時刻只能 issue 1。若 workload 的 instruction energy 與 fetch bandwidth 不變，平均 IPC 應以動態 issue 分布計算，不能把峰值直接乘上 4。

限制可能來自：

- data/branch dependencies。
- instruction and data cache bandwidth。
- branch predictor accuracy。
- functional-unit mix 與 register ports。
- reorder buffer size。
- power density 與 thermal constraint。

分析 workload 的 IPC histogram，比只報 peak IPC 更能解釋真實效能。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 4.11 Opteron X4 案例

Opteron X4（Barcelona）fetch x86 instructions，decode/translate 成 AMD 所稱的 RISC operations（Rops），再由 dynamic scheduled speculative pipeline 執行。本章案例模型描述最多每 cycle issue 3 Rops，整數 pipeline 約 12 stages，浮點 execution queue 更長。

可能瓶頸包括：

- complex x86 instruction 拆成多個 Rops。
- hard-to-predict branches 造成 restart。
- 長依存或 cache misses。
- translation/rename/ROB 等 front-end 資源。

這些歷史數字用於理解 microarchitecture，不能當成 2026 年任何現行 CPU 的規格。

此案例的 issue 寬度及 pipeline 深度只適用於上述 Barcelona 模型；換架構時需重新取得前端、執行端與記憶體限制，不能由歷史峰值推算現代程式時間。



## 4.12 HDL 與模擬

Hardware description language（HDL，硬體描述語言）描述 module 的 ports、state、combinational equations 與 clocked behavior。Verilog 的 `always @(posedge clk)` block 表示 clocked process；blocking 與 nonblocking assignment 的語意不能混用而不檢查。

HDL 模擬只能證明 testbench 覆蓋範圍內的行為。Synthesis、timing analysis、formal assertion 與實機測試回答不同問題。

以時序 assertion 可以檢查 hazard、flush 與 commit order。Simulation 覆蓋到的 testbench 案例、synthesis 結構、static timing 與實機量測各自回答不同問題，不能互相取代。

可檢查的時序條件包括：stall 且沒有 redirect 時，下一個 edge 的 PC 與 IF/ID 必須保持；valid=0 的指令不得寫 architectural state；faulting instruction 與其後的指令不得 commit；ROB commit 的指令序號必須依 program order 遞增。這些條件分別對應停頓、bubble 與 precise exception 的正確性。

## 4.13 設計誤解

「Pipeline 能降低每條 instruction latency」不成立。Balance 的五階 pipeline 讓每條 instruction 經五個 stages；收益來自許多 instructions 重疊。

「Forwarding 可解決所有 data hazards」忽略 load-use、long-latency units 與真資料依賴。

「Pipeline 設計與 technology 無關」忽略邏輯深度、clock distribution、power 與 memory latency 的變化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 4.14 章末模型收束

設計一個 processor 的核心檢查是：

1. 逐段驗證每條 instruction 完成後的 architectural state。
2. 列出每個 stage 的資源與結構衝突。
3. 證明 branch 與 exception 能恢復正確狀態。
4. 在相同 workload 下報告 latency、throughput、IPC、frequency 與 energy。
5. 測試資料依賴、memory access、wrong-path behavior 與邊界案例。

Single-cycle 易理解但 clock 被最長 path 支配；五階 pipeline 改善 throughput；forwarding、stall、prediction、precise state 與 dynamic scheduling 處理重疊帶來的代價。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 4.15 處理器技術演進

從單週期到多週期與管線，主要改變是把長組合路徑分配給不同時段或不同指令，並用暫存狀態追蹤尚未完成的工作。管線重疊引入資料與控制危害；前送和停頓維持值的正確性，分支預測則用恢復成本換取前端不中斷。

動態排程及暫存器重新命名讓後續就緒指令繞過尚未完成的運算，但真正資料相依仍須等待。亂序執行配合依序提交，把執行效率與精確例外的程式順序契約分離。更寬發射與更大視窗也增加選擇邏輯、線路、能耗與驗證成本，所以歷史進展不能只用管線階數或峰值IPC排序。

比較每個時代採用技術所解決的限制、增加的狀態，以及出錯時必須恢復的狀態，避免把歷史案例當成現行處理器規格。

### 參考資料與署名

Lin Cing，《計算機組織》手寫筆記。 Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 4. CSO course slides, processor datapaths and pipelines.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

## 高速處理器為何需要記憶體階層

Processor 每 cycle 可以執行運算，DRAM 存取卻可能需要數百個 processor cycles。若每次 load/store 都直接等待 DRAM，快速 datapath 會長時間閒置。

Memory hierarchy 用不同容量、成本與延遲的儲存層共同提供「近似快速且近似無限」的記憶體幻象。它依賴 locality（局部性），並不保證任意程式都得到相同效能。

本章依序建立 cache、virtual memory、TLB、virtual machine 與 coherence。每個結論都必須附上容量、block/page、miss penalty 與 write policy。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 位址、page 與 cache block 的單位

本章會重複使用四種位址：

1. VA（virtual address，虛擬位址），process 執行 load/store 時使用。
2. VPN（virtual page number，虛擬頁號），VA 去掉 page offset。
3. PA（physical address，實體位址），page translation 後存取 main memory 或 physically addressed cache。
4. Cache block address，PA 去掉 block offset。

Byte address 的最小單位是 1 byte。以本章 4-byte word 為例，word index 乘 4 才轉成 byte offset；byte address 除以 block size 才得到 block number。手算若混用單位，看似相同的低位元會得到錯誤 mapping。

本章的 $B$ 表示 block size bytes，$P$ 表示 page size bytes。公式會在首次使用時重新定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 5.1 局部性與記憶體階層成本

Storage hierarchy 通常由上到下為 registers、L1 cache、L2/L3 cache、main memory、flash/disk 與遠端儲存。越上層越快、每 byte 越貴且容量越小；越下層越慢、越便宜且容量越大。

理想是讓常用資料靠近 processor，低頻資料留在下層。Locality 是此設計成立的經驗前提，不是硬體能對任意程式保證的定理。

Temporal locality（時間局部性）表示近期用過的位置很可能近期再用。Spatial locality（空間局部性）表示使用某位置後，附近位置很可能很快被使用。

以連續 4-byte 整數陣列為例，依序讀取 byte addresses 0、4、8、12、16。假設 cache 原本為空、block 為 16 bytes、沒有 prefetch 且過程中不發生 eviction：前五次依序是 miss、hit、hit、hit、miss；前三個 hit 利用同一 block 的空間局部性。若隨後再次讀位址 16 且該 block 仍在，這次 hit 則是時間局部性的例子。



## 為何 cache 能改善平均時間

設上層 hit time 為 $H$，miss rate 為 $m$，miss penalty 為 $P_m$（在已付 hit lookup 時間後額外等待的時間）。平均記憶體存取時間（AMAT）為

$$\text{AMAT} = H + mP_m.$$

若 $H = 1$ cycle、$m = 0.05$、$P_m = 80$ cycles，則

$$\text{AMAT} = 1 + 0.05 \times 80 = 5 \text{ cycles}.$$

即使 miss 很慢，95% 的 accesses 可以由 1-cycle cache 滿足。若 hit rate 降至 80%，AMAT 變為 17 cycles，顯示 low hit rate 如何吞掉 cache 的價值。

Miss rate 的分母必須一致。Instruction miss rate 常以 instruction 為分母，data cache miss rate 常以 data references 為分母。

## Memory wall 與資源取捨

Processor performance 與 DRAM latency 的改善速度不一致，形成 memory wall。提高 capacity、associativity 或 block size 可能降低某些 misses，但也可能增加 hit time、miss penalty、面積與 power。

一個 hierarchy 設計是多目標最佳化：

- L1 優先降低 hit time，並避免 drag critical path。
- 下層 cache 承受較大容量與較高 associativity。
- Main memory 提供容量與 bandwidth。
- Disk/remote storage 處理容量與持久性，但 miss cost 遠高於 cache。

「增加 cache 一定更快」缺少 workload 與 hit-time 條件；容量增加可能使 index/select path 變慢。



## 5.2 Cache 的基本模型

Cache block（或 line）是 cache 與下層之間傳輸的單位，通常包含多個 words。Cache entry 至少保存 data、tag 與 valid bit；write-back cache 另外保存 dirty bit。

一次 reference 的步驟是：

1. 用 address 的 index 選 set。
2. 比較 set 內各 way 的 tag 與 valid。
3. 若 match，回傳 block 內由 offset 指到的 byte/word。
4. 若 miss，向下一層取得 block，選擇 victim，再寫入 cache。

Valid 防止未初始化 bits 因 tag 恰好相同而被誤認為 hit。

Tag 相同但 valid 為 0 仍是 miss；dirty bit 只表示是否需 write-back，不用來判斷 hit。

## Direct-mapped 映射

Direct-mapped cache 的每個 block 只能放在唯一位置：

$$\text{cache block index} = \text{memory block number} \bmod N_{\text{blocks}}.$$

若 cache 有 $2^q$ blocks，index 是 block number 的低 $q$ bits。Offset 選擇 block 內位置，tag 保存剩餘高位。

若兩個 frequently used blocks 映射到同一 index，即使 cache 其他位置空著，也會互相 evict。這是 conflict miss 的典型來源。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Accessing a cache：一個具體 trace

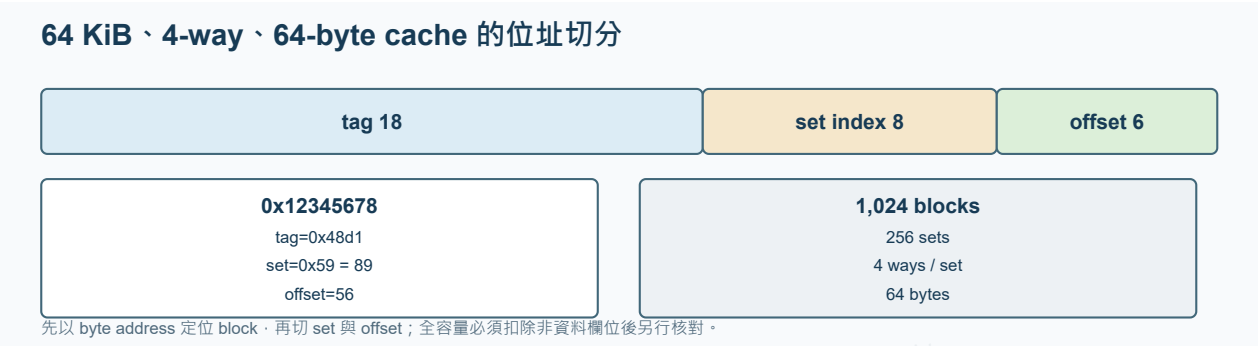
假設 4-block direct-mapped word cache，block size 與 word 相同。Block addresses `0,1,2,3,0,4,0` 的存取為：

| Access | Block | Index | 結果 | 原因 |
|---|---|---|---|---|
| 1 | 0 | 0 | miss | valid=0 |
| 2 | 1 | 1 | miss | valid=0 |
| 3 | 2 | 2 | miss | valid=0 |
| 4 | 3 | 3 | miss | valid=0 |
| 5 | 0 | 0 | hit | tag/valid match |
| 6 | 4 | 0 | miss | evict block 0 |
| 7 | 0 | 0 | miss | conflict with block 4 |

七次有六次 miss，miss rate 為 $6/7$。前四次與第一次碰到block 4的第六次屬compulsory；第七次對block 0的重訪是conflict，因同容量fully associative LRU在第六次會淘汰block 1而保留剛用過的block 0。

每次 miss 都要列 victim 與 valid/tag/dirty 更新，不能只算 hit count。

## Address decode：64 KiB、4-way、64-byte block




$$N_{\text{blocks}} = 2^{16}/2^{6} = 1024, \quad N_{\text{sets}} = 1024/4 = 256.$$

因此：

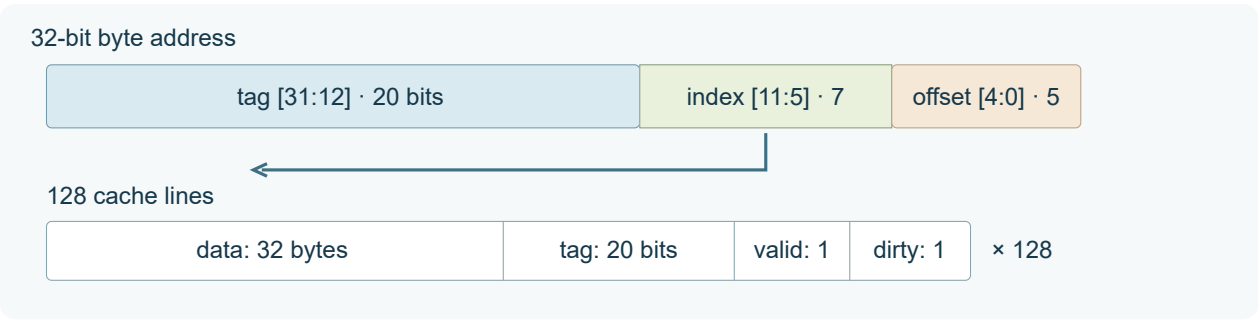
- block offset bits：$\log_2 64 = 6$。
- set index bits：$\log_2 256 = 8$。
- tag bits：$32 - 6 - 8 = 18$。

Address `0x12345678` 的低 6 bits 是 `111000`，偏移 56 bytes；接著 8 bits 是 `01011001`，index 為 89；高位是 tag `0x48d1`。

同樣 block 內的 `0x12345658` 有相同 tag/index、不同 offset，所以前 block 若仍 valid，這次應為 hit。



## 從位址欄位反推容量與額外成本




以 32-bit byte address 的 direct-mapped write-back cache 為例：offset 5 bits 表示每 line 有 32 bytes；index 7 bits 表示共有 128 lines，故資料容量為 128 × 32 = 4096 bytes。

每列還存 20-bit tag、1 個 valid 與 1 個 dirty，共 22 bits；全 cache 的額外成本為 128 × 22 = 2816 bits = 352 bytes。只計上述欄位時，總儲存為 4448 bytes，是 data capacity 的 1.0859375 倍；ECC 與其他控制電路另計。

## Handling cache misses

Simplified in-order processor 的 read miss 通常會 stall。Controller 要保存 miss address、送 read request、等待下層、選 victim、更新 data/tag/valid，再讓 instruction 重試。

Instruction miss 的重試 PC 是發出 request 的 instruction address，不是 PC 已被更新後的值。Data miss 要保存 load/store 所需 address 與 store data。

Write-back dirty victim 必須先 write-back 或放入 write-back buffer，再 fetch new block。Critical-word-first 可以先把 requested word 送回 processor，再完成 block fill。

Cache miss 通常是 pipeline stall 與 controller transaction；page fault 會經 exception 進入 OS，恢復成本與 state 處理不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Memory system 如何支撐 cache

Block fill 的 miss penalty 可拆成 address transfer、DRAM latency、burst transfer 與 cache update。以 four-word block、one-word-wide memory 為例，若 address 1 cycle、每次 DRAM access 15 cycles、每 word transfer 1 cycle：

$$1 + 4(15) + 4(1) = 65 \text{ bus cycles.}$$

Wider memory/bus 可平行取兩個 words，總時間可降為

$$1 + 2(15) + 2(1) = 33 \text{ cycles.}$$

這個例子假設 burst 次數依 memory width 減少，且 transfer 花費依 bus width 降低；不同 interconnect 要重新建模。

Block size 增加可提高 spatial locality 利用，卻也可能放大 miss penalty 與污染 cache。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 交錯式記憶體：讓不同 bank 同時工作

四個連續 words 分別放在四個獨立 bank；假設一個共用地址操作在 cycle 1 同時啟動四個 bank，各 bank 延遲 15 cycles，而匯流排每 cycle 傳一個 word。資料都就緒後，再依序傳四個 words：總 fill time = 1 + 15 + 4 = 20 bus cycles。

比較同一模型下單 bank 的 65 cycles：收益來自重疊 bank latency，而不是一次在窄 bus 上傳四個 words。若位址無法同時發出、連續資料映到同一 bank，或 bank 回應有額外間距，20 cycles 就不成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Handling writes

Write-through 同時更新 cache 與下層。它簡化 coherence 與 recovery，但每次 store 都消耗 lower-level bandwidth；write buffer 可讓 processor 不必逐筆等 memory。

Write-back 先更新 cache 並設 dirty bit；dirty block 被 evict 或一致性協定要求交付資料時，才將最新內容送往下層或其他 owner。它降低連續寫入的 traffic，但 eviction、coherence 與 recovery 更複雜。

Write-allocate 在 write miss 時先 fetch entire block；no-write-allocate 直接把 write 傳給下層。Policy 可以依 workload、頁面或 instruction 改變，例如整頁清零時不必先 fetch。

小於 cache line 的 store 只更新指定 byte；實作可使用 byte enable 或其他等效機制，不必然執行 read-modify-write，也不必然讀取 main memory。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Write policy 的邊界案例

| 策略 | Write miss 行為 | 優點 | 主要代價 |
|---|---|---|---|
| write-through + no-allocate | 直接寫下層 | 邏輯簡單 | 下層 traffic 高 |
| write-through + allocate | fetch block，之後同步寫下層 | 後續 reads 可能 hit | write bandwidth 高 |
| write-back + allocate | fetch block、修改資料並標 dirty | 連續 writes 省 traffic | dirty eviction |
| write-back + no-allocate | 直接寫下層 | 避免無用 allocation | 仍失去後續 write hit |

表中「allocate」是 placement policy，「through/back」是 lower-level update policy，兩者不可混為一組。

Store buffer 若保存未完成 writes，read-after-write 要看該 address 是否被 buffer 遮住，不能盲目由 cache 回舊值。

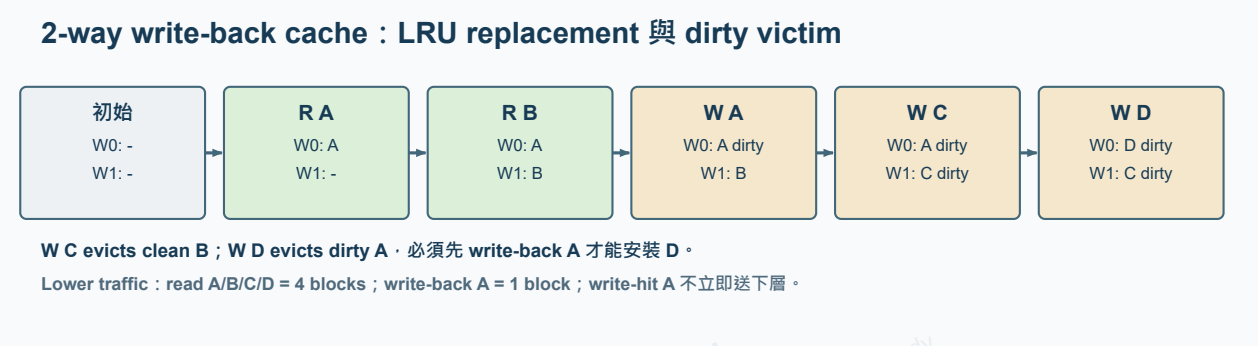


## Write-back、write-allocate 與 dirty victim trace

考慮 2-way set-associative，64-byte blocks，每次 store 只改4 bytes，LRU replacement，write-back、write-allocate。每個 set 初始沒有 valid entry，空 way 選編號較小者。Way 是固定實體槽位，更新 LRU 不搬動其他 line。以下 `A/B/C/D` 均映射同一 set；`dirty` 表示 cache 可能有尚未傳播的寫入。

| Step | Access | Way 0 | Way 1 | Victim 與 lower-level action |
|---|---|---|---|---|
| 0 | initial | invalid | invalid | 無 |
| 1 | read A | A clean | invalid | miss；read A 一個 block |
| 2 | read B | A clean | B clean | miss；read B |
| 3 | write A | A dirty（MRU） | B clean | write hit；不下層 |
| 4 | write C | A dirty | C dirty（MRU） | miss；B clean 可丟；read C 後寫 C |
| 5 | write D | D dirty（MRU） | C dirty | miss；A 是 LRU 且 dirty，先 write-back A，再 read D |

## Write-back、write-allocate 與 dirty victim trace （續）




這個 trace 下層共讀 4 blocks（A、B、C、D），共寫 1 個舊 block A。若控制錯誤地在 `Allocate D` 時先覆寫 A，A 的最新值只存在 cache，之後無法恢復；dirty bit 不影響 hit detection，卻決定淘汰前是否必須保存最新資料。Cache 與 DRAM 仍可能在斷電時失去內容；正確 write-back 不等於持久化保證。

換成 write-through、no-write-allocate：

1. Step 3 的 write A 立即送往下層。
2. Step 4 的 write C 不配置 cache，直接送下層，B 不會被 eviction。
3. Step 5 的 write D 也直接送下層。
4. Set 仍只保留 A、B；下層 reads 是 A、B，writes 是 A、C、D。



## Write-back、write-allocate 與 dirty victim trace （續）

只計 payload bytes，不含 headers/acknowledgments，也不在序列末尾強制 flush。WB/WA 的4個64-byte fill加1個64-byte write-back為320 bytes；WT/NA的2個64-byte fill加3個4-byte stores為140 bytes。兩者都有5次下層動作，卻不是相同大小的5個block transfers。WB最終C、D仍dirty，若要求再寫回下層，需另加128 bytes；持久性仍需平台的flush及ordering承諾。

在 write-invalidate coherence 模型下，cache 必須取得獨占寫入權限才能完成該 block 的 store；write-back 延後下層更新，但處理器仍須依記憶體模型取得正確的讀寫結果。其他協定可能採不同的共享及更新規則。Dirty victim 的最新資料必須先送往 memory、可靠的 write buffer 或有完成保證的 other owner，才能重用其 storage；clean victim 可依協定直接丟棄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## FastMATH cache 案例

本章案例的 Intrinsity FastMATH 是 MIPS embedded processor 教學案例。它使用 separate instruction and data caches，讓 pipeline 在同一 cycle 取 instruction 與存取 data；每個 cache 為 16 KiB、4K words、16-word blocks。

以 4K words 且每 block 16 words 計算，共有 256 blocks。若 architecture 的 cache 是 direct-mapped，index 有 8 bits、offset 有 6 bits；詳細實作仍以該處理器圖與規格為準，不能只由容量猜出全部 timing。

Separate caches 消除 instruction fetch 與 data access 的 structural conflict，但 instruction/data traffic 不再共享同一 capacity。

歷史實例的 cycle numbers 是模型輸入，換到今日 processor 必須重測。

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## 5.3 Cache performance

Processor time 可寫成

$$\text{CPU time} = (CPU_{\text{exec}} + CPU_{\text{stall}}) \times T_{\text{clock}}.$$

若每次 L1 miss 固定 penalty $P_m$，instruction miss rate 為 $m_i$，data references per instruction 為 $f_d$，data miss rate 為 $m_d$：

$$\text{CPI} = \text{CPI}_{\text{base}} + m_i P_m + f_d m_d P_m.$$

這是 in-order、每次 miss 全 stall 的簡化模型。Out-of-order processor 可重疊部分 misses，需用 stall distribution 或模擬。

不要先把 instruction 與 data miss rate 平均成單一 miss rate；兩類 request 的分母不同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 分離式與合併式 cache：先對齊分母

考慮 100 條退休指令，每條恰好一次 instruction fetch，另有 30 次 data references；不計投機取指或額外訪存。分離式的 I-cache 有 2 次 miss、D-cache 有 3 次 miss，故 miss rates 分別為 `2/100=2%` 與 `3/30=10%`。

若每次 miss 均造成互不重疊的額外 80 cycles，且 base CPI 不含這些停頓，追加 CPI 為 `(2 + 3)×80/100 = 4`。以每條指令計，也可寫成 `1×0.02×80 + 0.3×0.10×80 = 4`。

若另一個合併式 cache 在此 trace **也恰好**有 5 次 miss，則其 miss rate 以全部 130 次 references 為分母：`5/130`；每條指令有 `1.3` 次 references，追加 CPI 是 `1.3×(5/130)×80 = 4`。這只是分母換算，並不保證兩種配置的 miss 數相同；單埠合併式 cache 的取指／資料競爭也須另計，且不可重複計數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## AMAT 的數字例

令 L1 hit time 為 1 cycle、miss rate 為 6%、main-memory miss penalty 為 80 cycles：

$$\text{AMAT} = 1 + 0.06 \times 80 = 5.8 \text{ cycles}.$$

若 base CPI 是 1.4，instruction miss rate 2%，每 instruction 有 0.25 次 data references，data miss rate 8%，則 memory stalls 對 CPI 的追加為

$$0.02(80) + 0.25(0.08)(80) = 1.6 + 1.6 = 3.2,$$

總 CPI 是 4.6。若 clock 為 2 GHz，10 億 instructions 的 CPU time 是

$$10^9 \times 4.6 / (2 \times 10^9) = 2.3 \text{ s}.$$

Data misses 只在 data reference 發生，不能對所有 instructions 直接乘 data miss rate。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 更靈活的 block placement

Associativity 把 cache blocks 分成 sets：

$$N_{\text{sets}} = \frac{N_{\text{blocks}}}{E},$$

其中 $E$ 是 way count。Direct-mapped 等於 $E = 1$；fully associative 只有 1 set；set-associative 是兩者之間。

若總 data capacity 不變，associativity 每加倍一次，set 數減半、index 少 1 bit、tag 多 1 bit、comparators 約加倍。較多 ways 可降低 conflict misses，但可能拉長 tag compare 與 output select path。

Tag storage 也增加。64 KiB 例子中 direct-mapped 與 4-way 的 tag 陣列不同，不能說資料容量相同就硬體相同。



## Locating a block 與 replacement

Set-associative hit 的步驟是：

1. Index 選 set。
2. 同時比較該 set 所有 ways 的 valid 與 tag。
3. 以 match/way select 讀出對應 data。
4. 若多個 match，表示 cache state 或協議錯誤，而不是任選一個。

Replacement choices 包含 random、LRU 與近似 LRU。2-way LRU 只需一個 recency bit；高 associativity 的 exact LRU state 快速成長，實作常改為 pseudo-LRU。

```
2-way set:  [A, B], recent=A
access B  -> recent=B
miss      -> victim A
```

Victim selection 不影響 architectural correctness，但可明顯改變 miss rate。Coherence cache 的 replacement 還要處理資料 owner 與 invalidation。

## Multilevel caches 與 AMAT 的分母

令 $T_\ell$ 是第 $\ell$ 層被存取時的 hit time（L1 miss 後才付 L2 hit time），$M_\ell^{\text{local}}$ 是送到該層的 references 中未在此層命中的比例，$P_{\text{mem}}$ 是最後仍 miss 後額外付出的 memory penalty。兩層模型是

$$\text{AMAT}_1 = T_1 + M_1 T_2 + M_1 M_2^{\text{local}} P_{\text{mem}}.$$

代入 $T_1 = 1$、$M_1 = 0.04$、$T_2 = 8$、$M_2^{\text{local}} = 0.25$、$P_{\text{mem}} = 100$：

$$\text{AMAT} = 1 + 0.04(8) + 0.04(0.25)(100) = 1 + 0.32 + 1.00 = 2.32.$$

| Quantity | 分母 | 本例 |
|---|---|---|
| $M_1$ | 所有 L1 references | 0.04 |
| L2 local miss $M_2^{\text{local}}$ | 只算 L1 misses | 0.25 |
| L2 global miss $M_1 M_2^{\text{local}}$ | 所有 L1 references | 0.01 |
| L2 hit service fraction | 只算 L1 misses | 0.75 |

若 L1 misses 中有 75% 由 L2 命中，其成本是 8 cycles；剩下 1% 的全部 references 再付 100 cycles。把 25% 直接乘 100 會把 local denominator 當 global denominator，得到錯誤的 26.32 cycles。

若另定 $P_{2,\text{miss,total}} = T_2 + P_{\text{mem}} = 108$，它是「已知 L2 也 miss」時，從開始 L2 access 到 memory 回覆的總服務時間；L2 hit 仍只需8。因此必須條件平均：

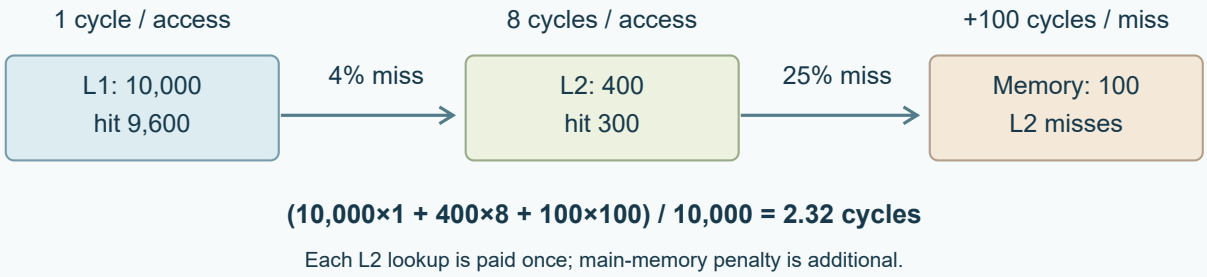


## Multilevel caches 與 AMAT 的分母 （續）

$$\text{AMAT}_1 = T_1 + M_1\left[(1 - M_2^{\text{local}})T_2 + M_2^{\text{local}} P_{2,\text{miss,total}}\right] = 1 + 0.04[0.75(8) + 0.25(108)] = 2.32.$$

只有把 $\overline{P}_{1,\text{miss}}$ 明定為「每次 L1 miss 的期望服務成本」，即 $T_2 + M_2^{\text{local}} P_{\text{mem}} = 33$，才可簡寫 $\text{AMAT}_1 = T_1 + M_1 \overline{P}_{1,\text{miss}} = 1 + 0.04(33)$。把條件於 L2 miss 的108直接乘 L1 miss rate會錯得 5.32；penalty 是否包含L2時間，不能取代對L2 hit/miss的條件區分。

## 用 10,000 次存取看懂多層 AMAT




**(10,000×1 + 400×8 + 100×100) / 10,000 = 2.32 cycles**

Each L2 lookup is paid once; main-memory penalty is additional.

假設這 10,000 次存取符合前頁比例：L1 miss `10,000×0.04=400` 次；其中 25% 即 100 次也 miss L2，剩下 300 次在 L2 命中。

每次都付 L1 的 1 cycle，400 次付 L2 的 8 cycles，100 次再付額外 100 cycles。總成本為 `10,000+3,200+10,000=23,200` cycles，平均 `23,200/10,000=2.32` cycles。這裡 100 cycles 是 L2 lookup 之後新增的 penalty，不可把已計入的 8 cycles 再算一次。



## 5.4 Virtual memory 的兩個目的

Virtual memory 將 process 的 virtual address space 映射到 physical memory。它讓每個 process 可以在一致的虛擬位址配置下執行程式，同時由 page table 與 privilege 控制避免任意互相讀寫。

第二個目的是擴大可用容量。Main memory 像 disk-backed storage 的 cache，只保存活躍 pages；page fault 時再把需要的 page 載入。

Modern systems 最關鍵的價值通常是 protected sharing 與簡化 allocation。歷史上手動 overlay 由 programmer 安排程式片段，virtual memory 把大部分搬移工作交給 OS 與硬體。

Virtual memory 不等於「把資料無限加速」。它提供位址空間與保護；真正效能仍取決於 page residency 與 locality。

## Page table 與位址翻譯

設 page size 是 4 KiB，則 page offset 有 12 bits。32-bit VA 的 top 20 bits 是 VPN；32-bit PA 的 top bits 是 physical page number（PPN）。

Page table 的 index 是 VPN，entry（PTE）通常包含：

- valid/present。
- PPN 或 disk location。
- read、write、execute 與 user/kernel permission。
- dirty 與 reference bits。
- 可能包含 cacheability/coherence 屬性。

Translation 是

$$\mathrm{PA} = \mathrm{PTE.PPN} \parallel \mathrm{VA.page\ offset}.$$

Offset 不翻譯，因為 page 與 frame 內部配置相同。

Page table entry 的完整欄位依 ISA/OS 而異；本表是教學共同模型。



## Page fault 不只是一種 miss

TLB miss 表示硬體 translation cache 找不到 entry，page 可能仍在 memory。Page fault 則表示所需 page 不在 memory，或 access 不符合權限，OS 需要介入。

以下僅示範合法且可恢復的 not-present fault；權限違規須先判斷是否存在合法修復方式，不能直接照此載入：

1. 保存 faulting VA、access type 與可恢復 state。
2. OS 找到 free frame 或選 victim。
3. 若 victim dirty，先依其 backing 與同步規則保存內容，再重用 frame。
4. 依 mapping 從 backing storage 讀入、建立 zero-filled page，或以其他合法方式提供內容。
5. 更新 page table，必要時 invalidate stale TLB entries。
6. 重新執行 faulting load/store。

需要從backing storage調入的major page fault以儲存I/O時間計，遠高於cache miss；demand-zero、copy-on-write等minor fault可能只配置或複製memory，不一定做disk I/O。OS 優先降低 fault frequency，而不是只追求 perfect replacement。

Store page fault 不得在 page 保護或 space 確認前留下 partial architectural write。

## Virtual memory 的 writes

Virtual memory 幾乎使用 write-back。若每次 store 都等 backing storage，成本會達數百萬 cycles；write buffer 無法吸收這種延遲。

在此簡化模型中，對 page 的 write 會設 dirty bit，表示內容可能有尚未保存的修改；寫入相同值也可能設 dirty，因此 dirty 不證明資料必然不同。Eviction 時依 backing 與 OS policy 保存 dirty page；clean page 在有可重建的 backing 時可丟棄。

Invalid PTE 可能代表 page 未配置、已 swap、保護違規或 file mapping 的特殊狀態。OS 以 fault address、error code 與 VMA/page-table 狀態決定處理方式。

Dirty 是保守的修改紀錄，不是最後寫入的時間戳；清除時須配合 writeback 與並行寫入的同步，避免漏記後續修改。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Making translation fast：TLB

TLB（translation-lookaside buffer，位址翻譯旁看緩衝）是 page-table entries 的小型 cache。Entry 可包含 VPN tag、PPN、valid、dirty、access permissions 與 ASID。

TLB hit 可由 VPN 快速得到 PPN。TLB miss 不一定表示 page fault：page 可能已在 memory，只需 page walk 並 install TLB entry。

TLB 通常 fully associative 或高 set-associative，因為容量小而 miss cost 重要。TLB entries 的總數遠少於 page-table entries，所以需要 tag；page table 則由 VPN 完整索引。

TLB dirty bit 要隨 PTE 更新同步；context switch 可能 flush TLB，或用 ASID 區分不同 process。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# TLB miss、兩層 page walk 與 cache 查詢

沿用 32-bit VA、4 KiB pages、PTE 4 bytes、每層 1024 entries。令 `VA=0x00403abc`：

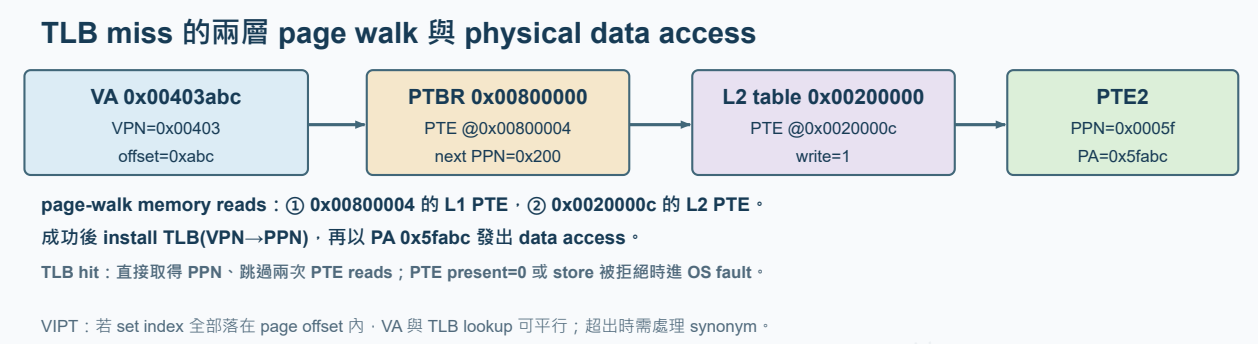
$$\mathrm{VPN} = 0x00403, \qquad \mathrm{offset} = 0xabc.$$

兩層各 10 bits，所以 level-1 index 是 `0x001`、level-2 index 是 `0x003`。以下 PTE contents 是明定的教學輸入：

| Walk step | Address calculation | Read/decision |
|---|---|---|
| 1 | PTBR `0x00800000` + `0x001×4` | read L1 PTE at `0x00800004` |
| 2 | L1 PTE PPN `0x00200` forms table base `0x00200000` | read L2 PTE at `0x00200000 + 0x003×4 = 0x0020000c` |
| 3 | L2 PTE has `present=1`、`write=1`、`PPN=0x0005f` | install TLB mapping `0x00403→0x0005f` |
| 4 | `PA=(0x0005f<<12)\|0xabc=0x5fabc` | data access at `0x5fabc` |



# TLB miss、兩層 page walk 與 cache 查詢 （續）




TLB miss 的成本不是一個抽象常數。在本例沒有 page-walk cache、需要走完兩層的假設下，須讀取兩個 PTE，再加上資料 cache 查詢與可能的 fill；這些 PTE 存取仍可能 hit 或 miss 一般 cache。若 page-table walk 的 intermediate PTE 有專用 cache，實際 access count 必須重新數。

TLB hit 的 VA `0x00403abc` 直接得到 PPN `0x0005f`，跳過 L1/L2 PTE reads。TLB miss 但兩個 PTEs 都 valid 是 translation miss；若 walk 遇到 PTE `present=0`，則是 not-present page fault。若 `present=1`、`write=0` 而 access 是 store，結果是 protection fault。三者的 hardware path 與 recovery 不同。

## TLB miss、兩層 page walk 與 cache 查詢 （續）

對 VIPT（virtually indexed, physically tagged）cache，未翻譯的 page offset 可在 TLB 查詢時提供 set index。簡單安全配置要求 **block-offset bits + index bits <= page-offset bits**。令 block 大小 B、sets 數 S、page 大小 P，條件 BS<=P，等價於 `data capacity / ways <= page size`。只比較 index 欄位寬度與 page-offset 寬度會漏算 block offset。

64 KiB、4-way、64-byte blocks 有256 sets。Block offset佔bits5:0，index佔bits13:6；6+8=14>12，不符合4 KiB page的條件。VA `0` 與 `0x1000` 若映到同一實體頁的相同offset，分別選set0與64，可能形成同一PA的兩份synonyms（別名）副本。改成16-way，則64sets且6+6=12，別名會選同一set。

允許index超出page offset時，須另有page coloring、synonym偵測/維護，或改用PIPT。若set由VA[13:6]選取，卻只比PIPT式的PA[31:14]，就漏比physical bits13:12，可能把不同PA誤認為hit。比較完整20-bit PPN可避免此false hit，卻仍不能單靠tag comparison消除同一PA在不同sets的重複副本。18-bit PIPT tag不能直接當作此VIPT的完整physical識別條件。



## Protection 與 context switch

Page translation 同時提供 protection。PTE 的 user/supervisor、read/write/execute 權限決定 access 是否合法。權限必須出現在 PTE 與 TLB entry；否則 TLB hit 可能繞過正確檢查。

Context switch 時若新 process 使用相同 virtual page number，舊 TLB entry 可能被誤用。解法包括：

- context switch 時 flush TLB。
- TLB tag 加入 ASID（address-space identifier，位址空間識別碼）。

ASID 讓不同 processes 的 translations 共存，降低 switch 後的 TLB misses。OS 仍需管理 ASID 重用與 invalidation。

Cache 若以 physical address 索引，通常不會因相同 VA、不同 PPN 產生同名問題；虛擬定址 cache 還需 alias/context 處理。

## TLB miss、page fault 與恢復

下表中「層級」描述事件來源，不是所有系統都用相同 exception code：

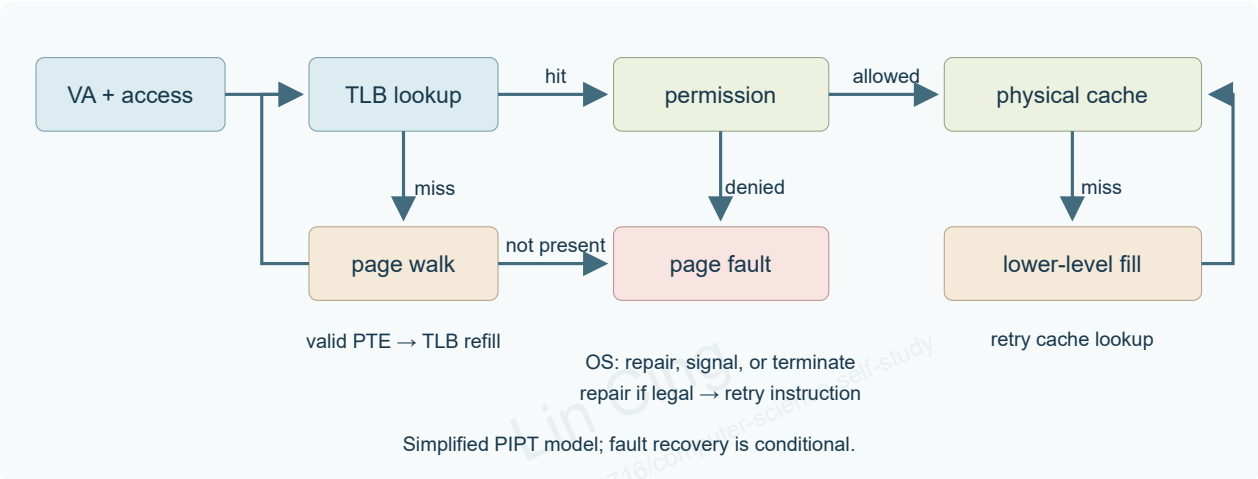
| 事件 | 硬體發現 | 下一步 | 是否重新執行 instruction |
|---|---|---|---|
| Cache miss | tag 不 match/valid=0 | lower-level fill | 完成原請求或重試 |
| TLB miss，PTE valid | TLB 無 match | hardware/OS page walk、檢查權限 | 完成或重試 |
| Not-present fault | PTE not present | OS 判斷；合法時配置或載入 | 依處理結果 |
| Protection fault | permission 不符 | 終止、signal 或 OS 處理 | 依語意 |
| Undefined VA | page table 無 mapping | OS fault | 依語意 |

若 instruction 已修改 register 或部分 memory 才 fault，pipeline 需要 precise exception 或可恢復 state。

TLB miss 與 page fault 共用 address register 不表示同一個 exception。必須追查 PTE 結果。



## 一次存取：三種事件走不同路徑




以簡化的 physically indexed、physically tagged（PIPT）cache 為例：TLB miss 先查 page table；有效映射可回填 TLB，不等於 page fault。無映射或權限不符會交給 OS 判斷：合法時可能修復並重試，否則送出錯誤或終止。

翻譯與權限檢查通過後才查 physical cache；cache miss 向下層取 block，再完成或重試。這只是教學路徑：VIPT 等設計可並行部分查詢，實際 fault 與 refill 機制也依 ISA／OS 而異。

## 5.5 四個共同問題

Cache、TLB 與 virtual memory 都可以用四個問題描述：

1. Block/page 可以放在哪裡？direct、set-associative 或 fully associative。
2. 如何找到？index、parallel tag search 或 lookup table。
3. Miss 時替換誰？random、LRU、pseudo-LRU 或 OS policy。
4. Write 如何傳播？write-through、write-back、allocate policy。

Table lookup 可以減少 comparison hardware，但需要額外 memory access。Virtual memory 接受 page-table overhead，因為 miss cost 極高，且完全關聯式 placement 給 OS 最大配置彈性。

「fully associative 最好」缺少 hit-latency 與 comparator 成本；以下以 fully associative LRU 作 shadow 定義 3C，因此該基準本身沒有這個定義下的 conflict miss，仍可能有 compulsory 與 capacity misses。



## 三類 miss：先固定比較基準

以下以相同 block size、相同 data capacity、每筆存取都更新的 fully associative LRU shadow 為基準。先確認實際 cache 發生 miss，再分類：

| 類型 | 這次 miss 的判定條件 | 常見改善方向 | 可能代價 |
|---|---|---|---|
| Compulsory（首次存取） | 這個 block 從未被存取 | 較大 block、prefetch | 流量、pollution |
| Capacity（容量） | 已存取過，shadow 也 miss | 增大容量、改善 locality | hit time、成本 |
| Conflict（映射衝突） | 已存取過，shadow hit | 增加 associativity | 比較與選取成本 |

這是對指定 trace 與比較基準的逐筆分類。第一次碰到 block 的 miss 優先列為 compulsory；「工作集大於容量」或「本次換掉同一 line」都不足以判定其餘每一筆 miss。

## 用同一串存取區分三類 miss

固定 32-bit byte addresses、64 B data capacity、16 B blocks，cache 初始為空，只有 read、無 prefetch 或 coherence invalidation。實際 cache 是 4-line direct-mapped；比較基準是同樣容納 4 blocks 的 fully associative **LRU shadow cache**。

```
block sequence: 0, 4, 0, 1, 2, 3, 4, 0, 1
byte addresses: 0x0000, 0x0040, 0x0000, 0x0010, 0x0020,
                0x0030, 0x0040, 0x0000, 0x0010
```

每次都讀 block 內 offset 0。DM 的 offset 為 4 bits、index 為 2 bits、tag 為 26 bits；`line=block mod 4`、`tag=floor(block/4)`。Block 0 與 4 分別具有 tag 0 與 1，卻同樣競爭 line 0。Shadow 的索引不採此規則，四個位置可放任意 blocks。

先查 actual cache：若 hit 就不屬於 3C；若 miss，第一次碰到此 block 是 compulsory，其餘才以 shadow 的 hit/miss 判為 conflict/capacity。**兩個 cache 都處理每一筆存取，包括 actual hit**，且 LRU hit 也會更新最近使用次序。



## 第三次讀取：容量還有空位，卻已發生衝突

同容量：**4 blocks × 16 B = 64 B**；初始皆空

**#1 block 0 (0x0000)**

Direct-mapped：line = block mod 4

| | |
|---|---|
| line 0 | block 0 |
| line 1 | — |
| line 2 | — |
| line 3 | — |

FA shadow：LRU → MRU

| | | | |
|---|---|---|---|
| 0 | — | — | — |

**DM miss / FA miss**

首次碰到此 block

**#2 block 4 (0x0040)**

Direct-mapped：line = block mod 4

| | |
|---|---|
| line 0 | block 4 |
| line 1 | — |
| line 2 | — |
| line 3 | — |

FA shadow：LRU → MRU

| | | | |
|---|---|---|---|
| 0 | 4 | — | — |

**DM miss / FA miss**

首次碰到此 block

**#3 block 0 (0x0000)**

Direct-mapped：line = block mod 4

| | |
|---|---|
| line 0 | block 0 |
| line 1 | — |
| line 2 | — |
| line 3 | — |

FA shadow：LRU → MRU

| | | | |
|---|---|---|---|
| 4 | 0 | — | — |

**DM miss / FA hit**

conflict：0 曾被 4 淘汰

DM 的四列是固定 line 編號；FA 圖的四格則依 LRU → MRU 排列，不是固定 way 編號。第 2 次讀 block 4 時，DM 必須淘汰 line 0 的 block 0；FA 尚有空位，因此兩者可同時保留。第 3 次重訪 block 0，DM miss、shadow hit，這一次才是 conflict miss。

## 第 1–5 次：先看 compulsory 與 conflict

表內為存取**後**的狀態，– 表示 invalid。H/M 是本次 hit/miss；FA 格內依 LRU → MRU 排列。

| # | block | DM [line 0,1,2,3] | DM | FA LRU → MRU | FA | DM 分類 |
|---|---|---|---|---|---|---|
| 1 | 0 | `[0,–,–,–]` | M | `[0]` | M | compulsory |
| 2 | 4 | `[4,–,–,–]` | M | `[0,4]` | M | compulsory |
| 3 | 0 | `[0,–,–,–]` | M | `[4,0]` | H | conflict |
| 4 | 1 | `[0,1,–,–]` | M | `[4,0,1]` | M | compulsory |
| 5 | 2 | `[0,1,2,–]` | M | `[4,0,1,2]` | M | compulsory |

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## 第 6–9 次：分清 capacity 與 hit

第 6 次後 shadow 含 4 個 blocks；繼續逐筆更新，而不是重新從 empty 開始。

| # | block | DM [line 0,1,2,3] | DM | FA LRU → MRU | FA | DM 分類 |
|---|---|---|---|---|---|---|
| 6 | 3 | `[0,1,2,3]` | M | `[0,1,2,3]` | M | compulsory |
| 7 | 4 | `[4,1,2,3]` | M | `[1,2,3,4]` | M | capacity |
| 8 | 0 | `[0,1,2,3]` | M | `[2,3,4,0]` | M | capacity |
| 9 | 1 | `[0,1,2,3]` | H | `[3,4,0,1]` | M | hit |

DM 共 8 misses：5 compulsory、1 conflict、2 capacity。第 7、8 次是重訪且 shadow 也 miss；不能只因 DM 在 line 0 發生替換，就把它們都列為 conflict。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Shadow 的 miss 不一定是 actual 的 miss

**#9 讀 block 1（byte address 0x0010）**

**Direct-mapped**
固定 line 0 / 1 / 2 / 3

| block 0 | block 1 | block 2 | block 3 |
|---|---|---|---|

**HIT：block 1 一直保留在 line 1**
狀態不變；不是任何一種 miss。
#7、#8 只替換 line 0。

**Fully associative LRU shadow**
存取前：LRU → MRU

| block 2 | block 3 | block 4 | block 0 |
|---|---|---|---|

**MISS：#8 已淘汰 block 1**
淘汰 2 → 新次序 [3, 4, 0, 1]
shadow 仍須更新，但 actual 不計 miss。

第 9 次是 **actual hit、shadow miss**，不計入實際 cache 的任何 miss 類別。全段 DM 與 FA 各有 8 misses：第 3 次的 DM miss／FA hit，恰與第 9 次的 DM hit／FA miss 抵銷。總數相減雖為 0，仍有一次 conflict；必須逐筆判定。



## 改成 2-way：同容量不等於同樣的替換結果

仍為 64 B、每 block 16 B，2-way 時有 2 sets，`set=block mod 2`，各 set 各自維護 LRU。全段 hit/miss 為 `M M H M M M M M H`，共 7 misses。第 9 次前 set 0 為 `[4,0]`、set 1 為 `[1,3]`（皆由 LRU 排至 MRU），所以 block 1 仍 hit。

| 設計 | hit 次數 | miss 次數 | hit time H | 總存取時間（額外 miss penalty = 20 cycles） |
|---|---|---|---|---|
| DM | 1 | 8 | 1 cycle | `9×1+8×20=169 cycles` |
| 2-way LRU | 2 | 7 | 1.25 cycles | `9×1.25+7×20=151.25 cycles` |

此表假設每次存取皆付 H，miss 再額外付 penalty，且全部序列化、沒有重疊。本例中 2-way 較快的條件是 `9×1.25+7P < 9×1+8P`，即 `P>2.25 cycles`。

FA 允許的 placement 更多，但 **LRU 不是最少 miss 的最佳替換演算法**。本例 2-way 反而少一次 miss；不能把「可放的位置更多」誤解成「任意替換策略對每一串存取都較好」。

## 5.6 Virtual machines

System virtual machine 提供一個 guest 可執行完整 OS 的 ISA-level 環境。VMM（virtual machine monitor，虛擬機器監控器）隔離 guests，並控制 privileged state、translation、interrupts 與 I/O。

Guest 以為自己掌握 machine，實際上 privileged operations 必須由 VMM 介入或安全地直接執行。Virtualizability 取決於 ISA 是否讓 sensitive privileged state 都能被 trap 或由硬體管理。

Virtual memory 在 VM 中可能有三層：guest virtual memory、guest physical/real memory 與 host physical machine memory。

Java VM 之類的 process-level VM 不提供完整系統 ISA 與 device 隔離；不要和 system VM 混為一談。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## VM 的 translation 與 I/O

沒有硬體 nested paging 時，VMM 可用 shadow page table 把 guest VA 直接映射到 host PA，並保護/追蹤 guest 對其 page tables 的修改。Shadow update 與 TLB invalidation 是 overhead。

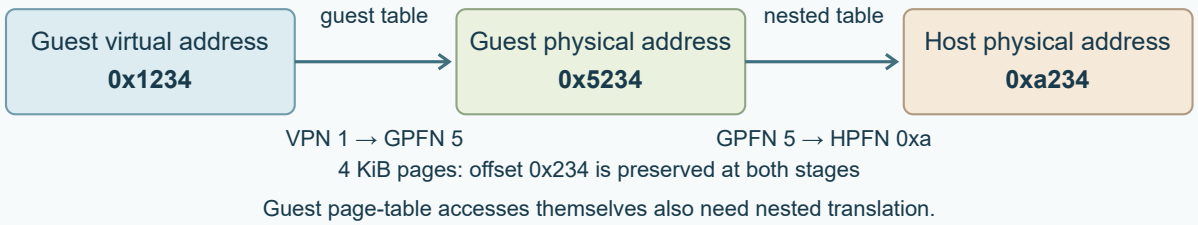
有 nested paging 時，硬體可執行兩層 walk，減少 VMM traps，但每次 TLB miss 可能多層存取。

I/O virtualization 特別複雜，因為 device、DMA、interrupt、queue 與 driver 都要隔離。VMM 可用 emulated device、passthrough 或硬體 I/O MMU。

本例的 x86/MIPS 歷史實作是本章案例範圍；今日硬體支援更豐富，結論仍以可 trap、可隔離與成本判斷。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 雙層分頁：一次位址翻譯的實例



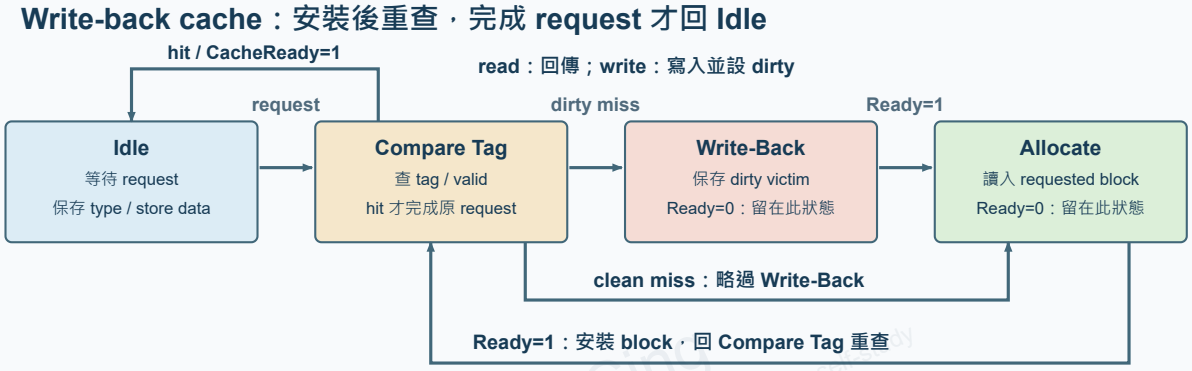

假設兩層皆用 4 KiB page，權限允許且所有映射有效。Guest 程式發出 GVA `0x1234`：guest VPN 為 `1`，offset 為 `0x234`。Guest page table 把 VPN `1` 映射到 guest physical frame `5`，得到 GPA `0x5234`。



## 巢狀翻譯的額外查表

Nested page table 再把 guest frame `5` 映射到 host frame `a`，得到 host PA `0xa234`；兩次翻譯均保留 offset。Guest page table 自身位於 guest physical memory，因此讀取 guest PTE 時，也要把該 PTE 的 guest physical address 經 nested table 翻譯。TLB 或 walk cache 若命中，實際讀取次數會減少，不能一律假設固定成本。

# 5.7 Cache FSM 控制




本章案例簡化 write-back cache controller 有四個 states：

1. Idle：等待 processor request。
2. Compare Tag：判斷 hit 或 miss；write hit 設 dirty。
3. Write-Back：dirty victim 寫回 memory，等待 `Ready`。
4. Allocate：fetch requested block，等待 `Ready`，完成後重查 tag。



# 5.7 Cache FSM 控制 （續）

Memory latency 可變，因此 FSM 以 `Ready` 而非固定 cycles 等待。未 assert 的 control signal 預設 deasserted；multiplexer select 仍要指定有效 value。

Write-back 與 Allocate 的 ordering 必須保證 dirty victim 在 frame 被覆寫前保存。

## FSM transition trace

以下 trace 對應 clean read miss 後 hit：

```
cycle 1: Idle + request -> Compare Tag
cycle 2: miss + victim clean -> Allocate
cycle N: memory !Ready -> stay Allocate
cycle N+1: Ready -> Compare Tag
cycle N+2: valid && tag hit -> set cache-ready -> Idle
```

Dirty write miss：

```
Compare Tag -> Write-Back -> Allocate -> Compare Tag -> Idle
```

Controller 也需要保存原 request type 與 store data；Allocate 完成後不能把 write 誤當成 read。

Early restart、critical-word-first 或 write buffer 會增加 concurrency，也增加 state 與 correctness obligations。



## 5.8 Cache coherence

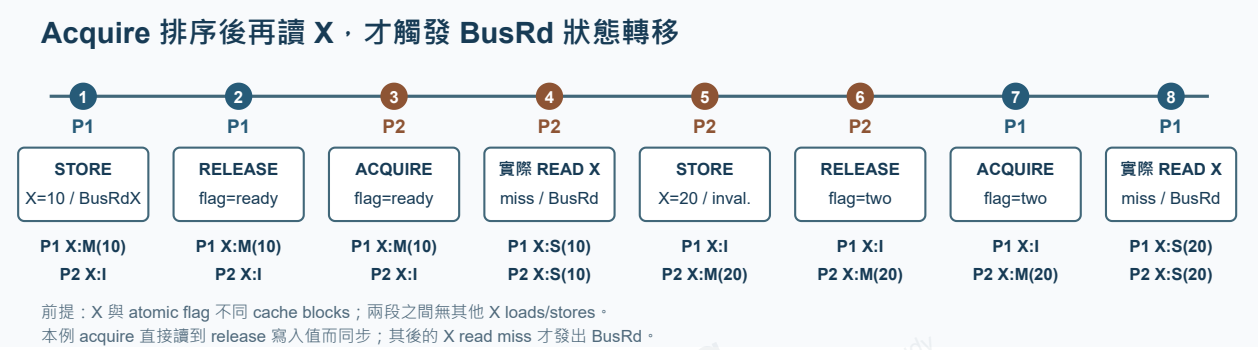
共享記憶體多處理器使用 private caches，因此同一 physical address 可能有多份 cached copies。Coherence 規定已排序的 read 可看到哪個 value，以及同一位置的 writes 如何被序列化。硬體層未排序的存取可能依 execution order 得到不同結果；read 看到舊值本身不證明 coherence protocol 壞掉，仍需看 consistency model 的排序。在 C++ 等語言中，非 atomic 的 data race 可能是 undefined behavior，不能直接把硬體層觀察當成語言保證。

Coherence 至少保證：

1. 同一 processor 在 program order 先寫同一位置，且其間沒有其他 processor 對同址的有效 write 時，後續 read 看到自己的 write。
2. 對同一位置的 writes 被所有 processors 以相同順序觀察。
3. 若 synchronization 已把 read 排在某個 write 之後，且其間沒有更晚的同址 write，該 read 回傳該 write 的 value。

第三點需要 memory consistency model 才能精確定義跨 thread 的「後續」。Coherence 管同一位置及其可見 value，consistency 管跨位置與 synchronization 的 ordering。只有當 atomic operation 依規格建立 synchronization，而且該關係涵蓋相互衝突的 accesses 時，才排除那一組 data race；它不會修正其他未排序存取，也不會把任意舊值讀取自動變成違規。

# 其他 write 介入後的 Write-back MSI 時序

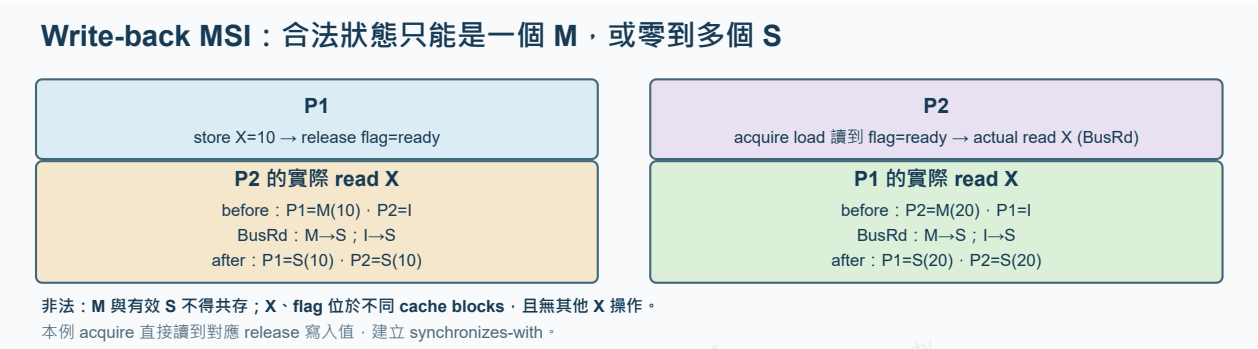



先看 P1 寫入 10，再發布 `ready`；P2 的 acquire 必須讀到這個 release 值，才能把先前的寫入排序在 P2 後續讀取之前。Acquire 本身不讀 X；P2 之後的 X read miss 才發出 BusRd，由 P1 的 M owner 提供 10。

P2 隨後寫入 20 並發布 `two`；P1 成功 acquire 後再讀 X，才能沿相同推理取得 20。本圖假設旗標與 X 位於不同 block，且兩段之間沒有其他 X 存取；共享 block 仍可同步，但會有額外的 coherence traffic。



# MSI 狀態不變量與 acquire/read 的差別




工作例明定 X 與 atomic `flag` 位於不同 cache blocks，且兩段程式之間沒有其他 X 操作。P1 store `X=10`，再以 release store 寫 `flag=ready`；在本例中 P2 的 acquire load 直接讀到這個 release 寫入值，兩者建立 synchronizes-with，接著**實際 read X**。P2 store `X=20`，再 release `flag=two`；P1 的 acquire load 實際讀到 `two` 後，才再次**實際 read X**。

| 事件（program order） | P1 X | P2 X | flag 與 ordering 關係 |
|---|---|---|---|
| 初始 | X:I | X:I | X 未 cached；flag 尚無 release/acquire 關係 |
| P1 store X=10；write miss / BusRdX | X:M(10) | X:I | P1 在自身 cache 完成 write；尚未建立與 P2 的跨 thread happens-before |

## MSI 狀態不變量與 acquire/read 的差別 （續）

| 事件（program order） | P1 X | P2 X | flag 與 ordering 關係 |
|---|---|---|---|
| P1 release flag=ready | X:M(10) | X:I | release 排序 P1 先前的 X store 與 atomic flag write；不要求把 dirty X flush 到 memory |
| P2 acquire flag=ready | X:M(10) | X:I | acquire load 實際讀到該 released value 才 synchronize-with；**不讀 X，也不改變 X cache state** |
| P2 acquire flag=ready 後，實際 read X；read miss / BusRd | X:S(10) | X:S(10) | P1 的 M owner 供給 10；P1 M→S，P2 I→S，memory 更新為 10 |
| P2 store X=20；BusRdX/upgrade | X:I | X:M(20) | P2 取得 exclusive permission，P1 的有效 copy 失效 |
| P2 release flag=two | X:I | X:M(20) | release 排序 X=20 與 atomic flag=two |
| P1 acquire flag=two | X:I | X:M(20) | acquire load 實際讀到 `two` 後才 synchronize-with；**P1 尚未讀 X** |
| P1 acquire flag=two 後，實際 read X；read miss / BusRd | X:S(20) | X:S(20) | P2 的 M owner 供給 20；P2 M→S，P1 I→S，memory 更新為 20 |



## MSI 狀態不變量與 acquire/read 的差別 （續）

Write-back MSI 的不變量是同一 block 最多只有一個 M；M 存在時，任何其他 cache 都不能保有有效 S。本表的合法 read miss 都由原 M owner 提供資料，並同時把 owner 降為 S。`acquire flag` 不是 `load X`；真正讀 X 的 load 才觸發 BusRd 與上述狀態轉移。若 `X` 與 `flag` 共用 cache block，false sharing 另會觸發該 block 的 coherence traffic；本表不把那個額外效果混入 X 的狀態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Snooping 與 write-invalidate

Snooping protocol 讓每個 cache 觀察 shared interconnect 的 block request，沒有 central directory。它適合 broadcast 可負擔的系統，processor 數增加後 traffic 與 scalability 成問題。

Write-invalidate protocol 在寫入前取得 exclusivity，使其他 cached copies invalid。常見簡化 state：

- I：Invalid。
- S：Shared，可讀且可能有多份。
- M：Modified，只有此 cache 有有效 copy，main memory 可能 stale。

Read miss 可能在 S 或 M 找到 supplier；write hit 在 S 需發 invalidate；M owner 被另一 read 觀察時須提供最新 data 並調整 state/write-back policy。

BusRd/BusRdX、cache-to-cache transfer 與 writeback 的行為依 protocol；state table 必須完整指定。



## 5.9 Cache controller 的設計

Cache controller 是處理 variable-latency memory 的狀態機。`Idle` 接收 request，`Compare Tag` 判定 hit/miss，`Write-Back` 保存 dirty victim，`Allocate` 等待 lower-level block。Memory 尚未回應時 `Ready=0`，完成時安裝 block、更新 valid/tag 並清除新 block 的 dirty，再回 `Compare Tag` 重查。只有重查命中並完成原 read 或 write 後，才 assert `Cache Ready`；原 request 是 write 時還要寫入保存的 store data 並設 dirty。

Controller 必須保存 request type、store data、fault/retry 所需的 state，以及 victim ownership。`Write-Back` 必須在 victim slot 被覆寫前完成，或先把 victim 安全放入有完成保證的 buffer；`Allocate` 不可越過這個 ordering condition。

可檢查的設計不變量包括：任何 valid cache line 的 tag 唯一指向其 data、dirty line 在被無備份覆寫前已保存、response 只在 block/data ready 後產生，以及 reset/error 不得留下半完成 transaction。這些 invariants 可用 assertion 或 model checker 測試。

## 5.10 Real systems

本章案例 AMD Opteron X4（Barcelona）與 Intel Nehalem 案例都把 larger caches、TLB 與 memory controller 放在 die 上。Dies 有私人 L1/L2、shared L3 與多層 TLB，降低 off-chip communication，但增加 area、power 與 coherence 複雜度。

Nehalem 案例的公開教材數據是每 core 私人 instruction/data cache、L2，以及四 cores 共享 L3。本章案例另一個關鍵是 lower-level caches 可由 data outside this core fill，並可採 victim cache 式關係。

這些 2008 年尺寸用來理解 cache sharing、inclusion 與 on-die traffic tradeoff，不是今日 purchase benchmark。

若 L3 non-inclusive，L2 miss 仍可能由 L3 hit；不能假設上層缺失一定能由直接下層找到。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 5.11 Fallacies and pitfalls

第一類錯誤是忽略 byte addressing。32-byte data cache、4-byte blocks 有 8 blocks；byte address 36 先整除 4 得到 block 9，再 mod 8 得到 block index 1。

第二類錯誤是把 block size 當成 block count。容量除以 block size 才是 blocks，不是把 address 除以 cache capacity。

第三類錯誤是把 local/global miss rates、instruction/data references 或 ns/cycle 混用。

第四類錯誤是只看 hit rate。更大的 L1 可能 miss rate 較低，但 clock path 變長，整體 AMAT 反而上升。

每題先寫單位與分母，通常可避免大半的手算錯誤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 5.12 Concluding remarks

Memory hierarchy 的統一原理是 locality、block transfer、placement/search、replacement、write propagation 與 miss cost。Cache 與 virtual memory 的參數差很多，policy 結構卻相似。

多層 cache 可以把 miss rate、hit time 與 miss penalty 交給不同層最佳化。Software 也能用 cache blocking、loop ordering 與 prefetch 改善 locality，但最佳化需以測量驗證。

「增加 L3」不是萬用加速器；working set、bandwidth、coherence 與 OS 排程共同決定結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 5.13 Historical perspective

早期 virtual memory 與 cache 是面對有限 memory、program relocation 與 protection 的工程解。分頁、TLB、set-associative cache 與 snooping coherence 逐步成為共同設計模式。

Hierarchy 變得複雜後，硬體仍保留可觀察模型：process 看到 virtual memory，ISA 規定 memory access 語意，coherence/consistency 規範 shared-memory behavior。效能實作可以不同，architectural correctness 不能悄悄改變。

歷史數字顯示技術比例會變，但 locality、protection 與 data movement 成本不會因某一代製程技術而消失。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 章末驗收

完成本章後，應能：

1. 由 byte address 手工切出 tag/index/offset。
2. 對任意 trace 列 hit/miss、victim、dirty/write-back。
3. 從 AMAT 推到 CPI 與 elapsed time，並說明 stall 假設。
4. 畫出 page walk、TLB hit、TLB miss 與 page fault 的分支。
5. 對 write-invalidate protocol 寫出 state transition。
6. 說明 3Cs、local/global miss 與 consistency model 的邊界。

先完成本章獨立練習題 PDF，再以詳解 PDF 核對單位、假設與每一步。

### 參考資料與署名

Lin Cing，《計算機組織》手寫筆記。 Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 5. CSO course slides, memory hierarchy.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

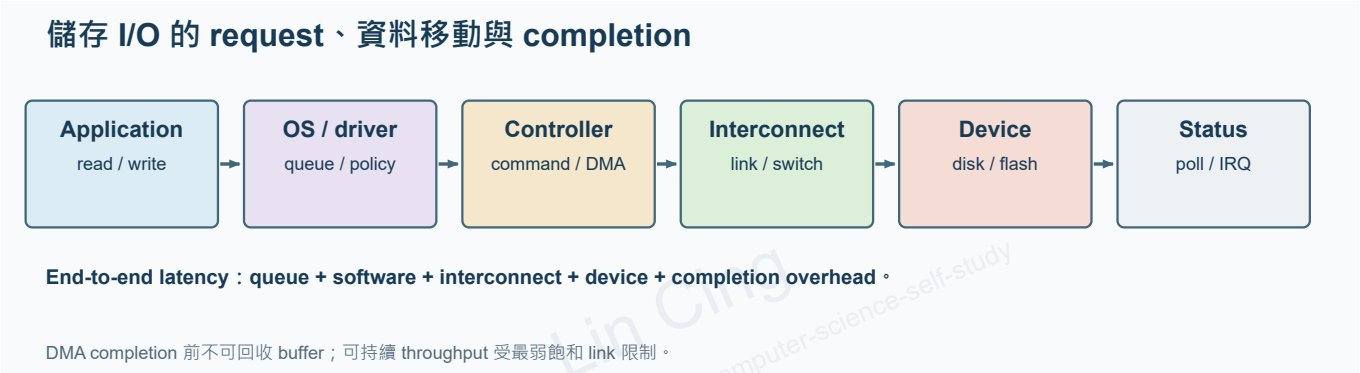
## 從一筆 request 看完整系統

應用程式讀取檔案時，檔案系統先把檔案位置轉成裝置區塊，driver（驅動程式）提交命令，controller（控制器）存取媒體並搬移資料，最後才回報完成。CPU 發出命令、資料抵達記憶體、應用程式獲准使用 buffer，是三個不同事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從一筆 request 看完整系統 （續）




效能分析先固定 request 大小、讀寫比例、queue depth（未完成請求數）、cache 狀態及完成的語意。寫入進入揮發性控制器 cache，不等於斷電後仍可讀回。容量、平均延遲、吞吐量與可靠度也不能用同一個數字代表。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 容量、頻寬與 IOPS 的分母

本章 GB、TB 分別採 $10^9, 10^{12}$ bytes；GiB、TiB 分別採 $2^{30}, 2^{40}$ bytes。Link 的 bit/s 須除以8才變成 byte/s，還需扣除協定額外流量。

若每個完成的request都是4 KiB，100,000 IOPS的payload頻寬是

$$100000 \times 4096 = 409600000\ \mathrm{B/s} = 0.4096\ \mathrm{GB/s}.$$

若改成64 KiB而IOPS不變，則是6.5536 GB/s，但裝置與link未必能承受。IOPS不是與request大小無關的常數。Response time從提交到完成；throughput是穩定狀態每秒完成數，不能取任意一筆latency的倒數代表並行系統吞吐量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 6.1 依工作負載選設計目標

互動式查詢重視尾端延遲；連續備份重視持續頻寬與恢復能力；交易系統還要求commit後的持久性與失敗原子性。相同SSD數量可以因request pattern與完成條件不同而產生不同瓶頸。

設計的限制包括裝置、controller、interconnect、記憶體及CPU處理命令的能力。擴充裝置還會消耗電力、queue entries、interrupt資源及故障恢復時間。高峰值頻寬無法單獨保證低延遲或高可用度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.2 Failure、reliability 與 availability

服務規格先定義可接受行為。實際服務偏離規格是failure（失效）；由失效狀態恢復符合規格是restoration（恢復）。元件fault可能被冗餘遮蔽，未必成為使用者看見的服務failure。

Reliability $R(t)$ 是從正常狀態起，至時間t仍連續正常的機率。Availability是觀察時刻服務正常的機率，或長期正常時間比例，允許中途失效後修復。修得很快可提高availability，卻不會把中途曾失效的歷史改成連續可靠。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 從更新週期推導可用度

假設系統反覆經歷正常時間U與修復時間D，各週期的(U_i,D_i)獨立同分布、非負，且0<E[U]+E[D]<∞。由大數法則，經歷k個完整週期後，正常比例為

$$A_k = \frac{\sum_{i=1}^{k} U_i}{\sum_{i=1}^{k} (U_i + D_i)} \longrightarrow \frac{E[U]}{E[U] + E[D]}.$$

因此 $A = MTTF/(MTTF + MTTR)$。這個長期比率不要求每次正常與修復時間都剛好等於平均值，也不給出每年最大停機次數。

MTTF=100,000小時、MTTR=4小時時，A=0.9999600016。8,760小時的長期平均停機量約為 $8760(1 - A) \times 60 = 21.02$ 分鐘。MTTR降到0.5小時後約2.63分鐘，減少18.39分鐘。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

### 平均失效時間不等於故障期限

只有另假設固定hazard $\lambda$ 的指數壽命，才有

$$R(t) = e^{-\lambda t}, \qquad MTTF = 1/\lambda, \qquad P(T \le t) = 1 - e^{-t/MTTF}.$$

MTTF=100,000小時時，一年內失效機率在此模型下是 $1 - e^{-8760/100000} \approx 8.39\%$，不是保證可用100,000小時，也不是「一年最多一次」。元件老化、共同供電與環境損壞會破壞固定hazard或獨立性假設，不能只憑平均值外推實際風險。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



### 6.3 磁碟幾何與邏輯容量

磁碟由盤面、磁軌及sector（扇區）組成，同一磁頭半徑的各盤面磁軌稱cylinder。舊式固定幾何模型的payload容量為

$$C = \text{cylinders} \times \text{surfaces} \times \text{sectors/track} \times \text{bytes/sector}.$$

10,000 cylinders、4 surfaces、500 sectors/track、512 bytes/sector得到10,240,000,000 bytes，即10.24 GB或約9.5367 GiB。若這些已是可定址的logical sectors，不能再把raw媒體的gap/ECC比例任意扣一次。現代裝置使用LBA與控制器轉換，不把這個固定幾何公式當作實際物理配置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 一筆 random read 的各段時間

忽略排隊時，存取時間為

$$T_s = T_{seek} + T_{rotation} + T_{transfer} + T_{controller}.$$

在12,000 RPM且每track500 sectors的模型，轉一圈 $60/12000 = 5$ ms；均勻隨機到達角度的平均旋轉等待2.5 ms；一sector佔 $5/500 = 0.01$ ms。平均seek4.5 ms、controller0.10 ms，得到

$$T_s = 4.5 + 2.5 + 0.01 + 0.10 = 7.11 \text{ ms}.$$

只有一筆outstanding且不重排時，長期完成率約 $1/0.00711 = 140.65$ IOPS。這不是「提高queue depth後吞吐量永遠不變」的定理，因為排程與locality可能改變平均service time。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Sequential request 如何攤提固定成本

同一磁軌上100個相鄰sectors合併成一筆request，明定不跨track、不另付setup及rotation。Transfer為 $100 \times 0.01 = 1$ ms，總時間

$$4.5 + 2.5 + 1 + 0.10 = 8.10 \text{ ms}.$$

Payload為51,200 bytes，平均6.321 MB/s。若拆成100筆互不相關random requests，模型時間是711 ms。差異來自只付一次定位成本，不是把媒體旋轉速度提高100倍。真實連續讀還要明定track crossing、預讀cache與controller pipeline。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.4 NAND 的 page 與 erase block

NOR適合較細粒度讀取與部分直接執行的firmware用途；NAND以較高密度支援大量儲存，通常按page讀寫、按較大的erase block抹除。兩者皆非揮發性，但介面、成本與更新粒度不同。已寫入NAND page不能像DRAM word一樣任意原地重寫；FTL（flash translation layer）將logical page映到新physical page，使舊page失效，再由garbage collection回收erase block。

設一個erase block有8個pages，其中6個有效、2個失效。回收時先把6個有效pages複製到其他空間，再erase舊block，產生8個可寫pages；扣掉保存舊資料所需的6個位置，本輪只淨增加2個可供host更新的位置。

若本輪2個新host pages加上6個搬移pages，NAND共program8 pages，則WA=8/2=4。只計這個循環，不把erase次數直接加成寫入bytes。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Endurance 的粗估與限制

Write amplification定義為 $WA = \text{media寫入bytes} / \text{host寫入bytes}$。Host一天120 GB、media384 GB，WA=3.2。

若physical NAND400 GB、可用P/E預算3000 cycles，且均勻磨耗、WA固定、無其他保留預算，host總寫入量粗估

$$\frac{400 \times 3000}{3.2} = 375000 \text{ GB} = 375 \text{ TB}.$$

這不是產品保固。Wear leveling、壞block備援、溫度、retention與metadata都有額外影響。Overprovisioning增加可搬移空間，通常有助GC；若同時改變working set，觀察到的WA不能全歸因於保留比例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.5 Link、bus 與端到端瓶頸

Shared bus的多個裝置競爭同一傳輸資源；point-to-point link與switch可平行不同路徑，但仍可能共用上游link、controller與memory channel。裝置總峰值不能直接相加成應用吞吐量。

例如6個SSD各80,000 IOPS、HBA120,000、memory path300,000、CPU/driver150,000 IOPS，穩態上界為

$$\min(6 \times 80000, 120000, 300000, 150000) = 120000.$$

再加兩個SSD仍受HBA限制。若HBA升到240,000，下一個上界變成CPU的150,000。這些是固定同一request大小與讀寫mix的容量上界，排隊、lock與error recovery可讓實測更低。



## 6.6 命令、通知與資料搬移彼此獨立

MMIO（memory-mapped I/O）把裝置register映到位址空間，由具適當權限的load/store存取。特定ISA也有獨立I/O指令。裝置register的cacheability與ordering由平台規格決定，不能當作普通可快取RAM。

Polling與interrupt回答「如何得知狀態」；programmed I/O與DMA回答「誰搬資料」。DMA可搭配poll completion，也可搭配interrupt completion。使用interrupt不表示每個data word都由CPU搬移。

Doorbell store只是通知裝置有新descriptor。若descriptor資料尚未對裝置可見，裝置可能讀到舊位址或舊長度；正確順序須依OS DMA與I/O API建立，不是只加一條任意store即可。

## Polling 與 interrupt 的實算

3 GHz CPU，每秒20,000筆I/O。若每筆恰好一次poll、每poll100 cycles，成本為2,000,000 cycles/s，即0.0667% CPU。每筆interrupt handler1500 cycles時，成本30,000,000 cycles/s，即1% CPU。

這個一次poll的模型中，增加runnable threads數不會改變上述算式。若每次提交後忙等50 µs，且不與其他requests共用等待，20,000筆已需要整個core的1秒等待時間；不能沿用「只一次poll」的0.0667%結論。

Hybrid可先短暫poll再sleep，completion queue也可批次處理多筆。比較時需計setup、忙等、context switch、completion批次大小及延遲分布，不能先假設interrupt永遠較快。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



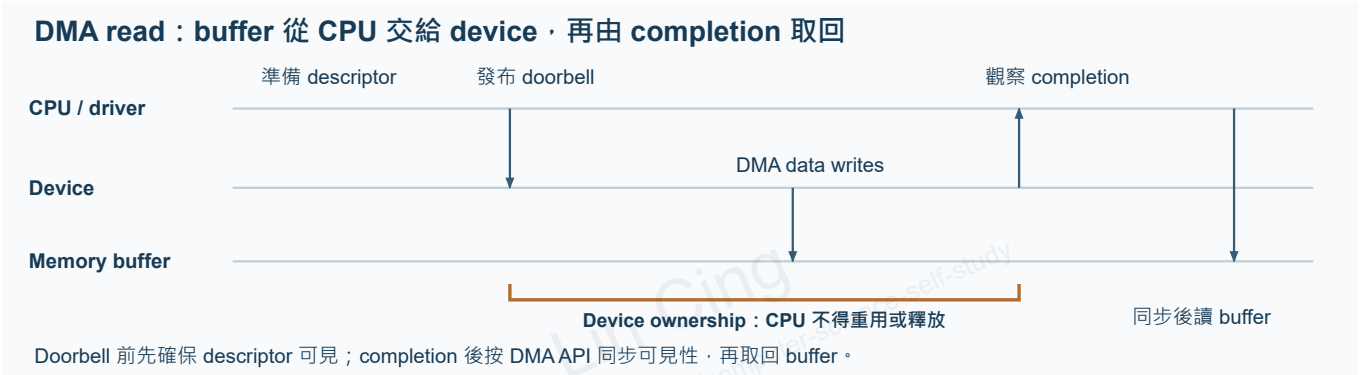
## Interrupt priority 與完成處理

Device設pending狀態，controller/CPU依enable與priority決定是否接受interrupt。進入handler後，driver讀status、確認來源、處理completion/error、按裝置規則清除或acknowledge來源，再喚醒等待工作。若一個shared interrupt有多個可能來源，不能無條件假設每次都是自己的device。

假設低priority handler關閉全部interrupt共8 µs，高priority事件在關閉後1 µs到達，則單是masking便使它至少等7 µs；「它priority較高」不能消除這段等待。允許nested interrupts可減少某些等待，但需保存獨立context、保護共享driver狀態並界定最大巢狀深度。Coalescing合併多個completion可降低每秒handler次數，也可能增加第一個completion的通知等待。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## DMA 的 buffer ownership 狀態




以device寫入memory的read request為例，CPU先配置並映射buffer，再發布descriptor。交給device後，CPU不可重用、釋放或依未完成內容作計算。Device完成data writes並發布completion；CPU觀察completion後，依DMA API同步可見性，再取回ownership。



## DMA 的 buffer ownership 狀態 (續)

Noncoherent平台需在正確時點clean/invalidate cache。若DMA已寫入新資料後，CPU才把舊dirty line盲目clean到memory，就可能覆蓋新資料。Timeout也不代表device已停止；解除mapping前須確認裝置不再發出DMA，否則會寫壞已重新配置給別人的頁面。

### DMA 節省的是 core 搬移成本

64 MiB在20 ms完成，payload頻寬為

$$64 \times 2^{20} / 0.020 = 3.3554432\ \mathrm{GB/s}.$$

CPU setup500 cycles在3 GHz需0.1667 µs。相較之下，programmed I/O每16 bytes需16搬移cycles加10 transaction cycles，共

$$(64 \times 2^{20}/16) \times 26 = 109051904\ \mathrm{cycles},$$

約36.35 ms CPU時間，尚不含等待。DMA仍消耗interconnect、memory bandwidth、IOMMU查詢與completion處理，不能把0.1667 µs誤當完整端到端時間。



## 6.7 Little 定律連結 concurrency 與 latency

穩定、無持續累積工作的系統，平均系統內requests數 $L$、完成率 $\lambda$、平均response time $W$ 滿足

$$L = \lambda W.$$

理由是觀察區間內「系統內人數對時間的面積」等於各request逗留時間之和；除以觀察長度，再讓起終邊界殘留的影響消失，即得此式。L含正在服務與排隊者；若只算queue，應使用Lq與Wq。

100,000 IOPS且平均response80 µs時，L=8。這不是保證把queue depth設8便達到該吞吐量，因為調整concurrency本身可能改變response time；它是同一穩態量測的必須一致關係。

## M/M/1 模型為何接近滿載就變慢

本頁另明定Poisson到達率λ、獨立指數service rate μ、單一server、FCFS、無限等待區，且ρ=λ/μ<1。這是分析模型，不宣稱磁碟實際service distribution一定指數。

平衡方程 $p_n\lambda = p_{n+1}\mu$ 給 $p_n = p_0\rho^n$。用機率和為1得 $p_0 = 1-\rho$。微分幾何級數可得

$$L = (1-\rho)\sum_{n\geq 0} n\rho^n = \frac{\rho}{1-\rho}, \quad W = L/\lambda = \frac{1}{\mu-\lambda}.$$

因此平均service time $1/\mu$ 不變時，queueing也能讓response暴增。ρ>=1不存在上述有限穩態平均，不能把負值或無窮級數公式硬套成正常latency。

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## 同一裝置兩個負載的排隊成本

取μ=200 IOPS，平均service5 ms。

| λ | ρ | W=1/(μ−λ) | Wq=W−5 ms | L=λW |
|---:|---:|---:|---:|---:|
| 100 IOPS | 0.5 | 10 ms | 5 ms | 1 |
| 160 IOPS | 0.8 | 25 ms | 20 ms | 4 |
| 190 IOPS | 0.95 | 100 ms | 95 ms | 19 |

從160增至190 IOPS只多18.75%完成量，平均response卻由25變100 ms。這是此模型的計算結果，不是所有實機固定倍數。只報最高IOPS會隱藏滿載附近的延遲代價；應同時固定平均/尾端latency限制。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 6.8 固定成本與持久性納入設計

總latency在無重疊串行路徑可相加；不同資源的持續吞吐上界取最小值。CPU與裝置可重疊時，不能把全部CPU時間和全部I/O時間無條件相加，也不能假設完全重疊。

Write-back cache可縮短回報時間，但若應用要求斷電後保留，就須把持久媒體、保護的cache或明確flush completion納入終點。兩個系統若一個回報「已入RAM」、另一個回報「已持久」，不能直接以latency比較相同服務。

錯誤重試需辨識命令是否已部分完成。重送write可能安全，重送「增加counter」卻可能執行兩次；controller/driver與上層交易協定共同決定恢復語意。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



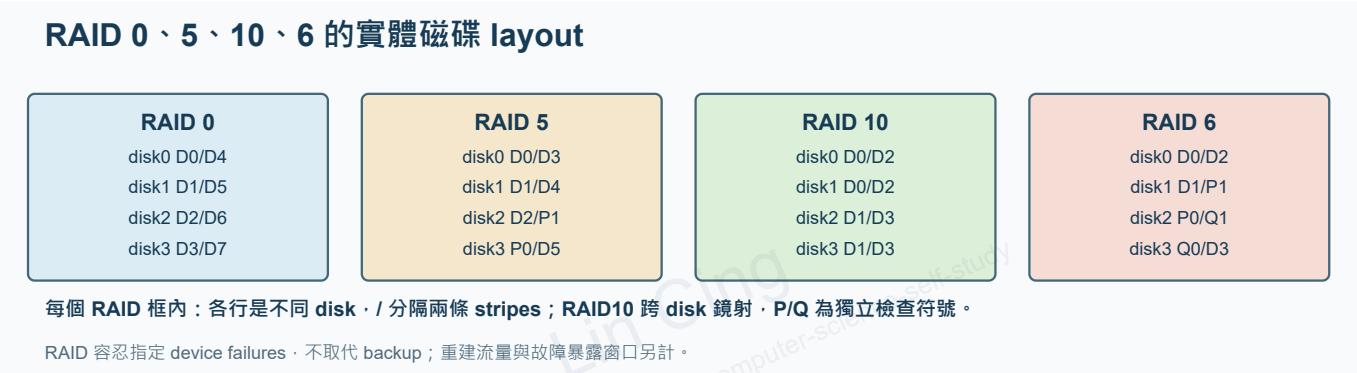
## 6.9 RAID 的容量與容錯模型

本章以n顆等容量C的disks，不含spare、metadata及保留空間。Failure指可識別的整顆disk erasure，假設survivors資料正確、controller與供電正常。

| 配置 | Usable容量 | 可恢復的整顆失效 |
|---|---:|---|
| RAID0 striping | nC | 無冗餘 |
| RAID1配對/RAID10 | nC/2，n為偶數 | 每組mirror最多失一顆 |
| RAID4/5 | (n−1)C | 任一顆 |
| RAID6 P+Q | (n−2)C | 任意兩顆，編碼及前提成立時 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.9 RAID 的容量與容錯模型 （續）




RAID3以細粒度bit/byte條帶配合同步存取，RAID4以block條帶讓小讀取分散，但集中parity造成寫入瓶頸。RAID5輪替parity位置，只消除固定單顆parity disk的瓶頸，並未消除parity更新成本。



## XOR parity 的恢復證明與數值

以bitwise XOR定義 $P = D_0 \oplus D_1 \oplus D_2$。因為 $x \oplus x = 0$、$x \oplus 0 = x$ 且可交換結合，遺失 $D_1$ 時

$$P \oplus D_0 \oplus D_2 = D_1.$$

取4-bit資料D0=1010、D1=1100、D2=0110，P=0000。遺失D1時，0000 XOR 1010 XOR 0110=1100，還原原值。

只有一個parity時，兩個未知data blocks只給一條XOR方程，因此通常不唯一。若disk回傳未被發現的錯誤值，甚至無法只由parity判定哪顆壞；「可恢復一個已知遺失」與「定位任意silent corruption」是不同保證。

## Small write 與 full-stripe write

更新一個data block時，利用

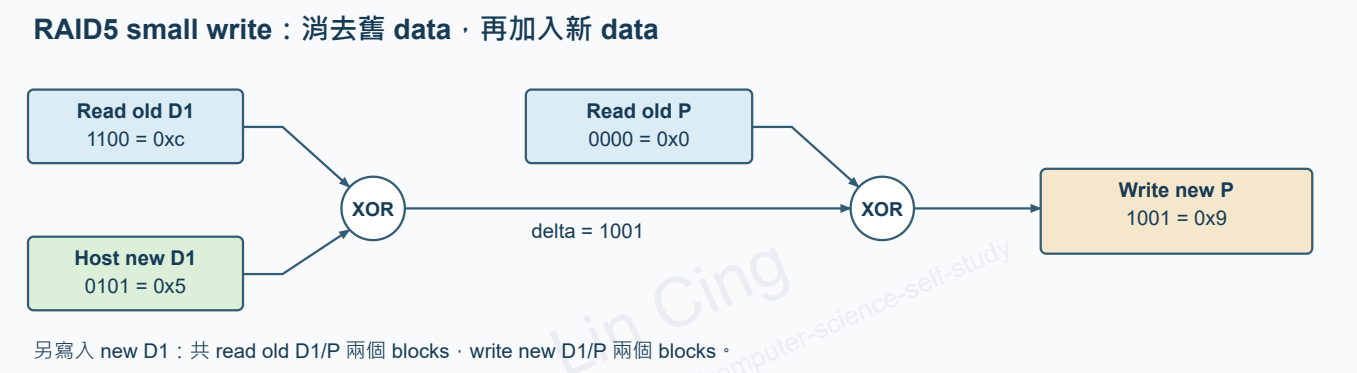
$$P' = P \oplus D_{old} \oplus D_{new}$$

可讀old data、old parity，再寫new data、new parity，共2讀2寫。若上述D1由1100改成0101，P'=0000 XOR 1100 XOR 0101=1001。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Small write 與 full-stripe write （續）




若一次覆寫整條stripe的m個data blocks，全部new data都由host提供，可以直接算new parity，不需讀old blocks，只寫m個data加1個parity。若只更新k個data，reconstruct-write可讀其餘m−k個舊data再寫k個new data與parity；需比較 $2k+2$ 與 $(m-k)+(k+1)=m+1$ 次block I/O，並考慮cache及並行瓶頸。

## RAID5 讀寫 mix 的容量上界

假設8顆disk，每顆持續100個block I/O/s；小讀只占1次disk I/O，小寫採2讀2寫，所有parity/data流量均衡，沒有cache合併或rebuild。令host中讀比例r，則每host request平均需要

$$r + 4(1 - r) = 4 - 3r$$

個disk I/O，因此host完成率 $X \le 800/(4 - 3r)$。r=0.75時，平均1.75個，X<=457.14 IOPS。

這是aggregate工作量上界，並未保證每次small write四個操作可完全平行。若某顆disk、stripe lock或controller已先飽和，實測更低。RAID4另有集中parity disk：每small write該disk要一讀一寫，寫入率先受其能力除以2限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 兩份相同 parity 為何不是 RAID6

若Q只是P的另一份copy，遺失兩個data時，兩條方程完全相同，仍無法唯一恢復。RAID6需要獨立的檢查資訊。

一種P+Q模型是在有限域GF(2^w)逐symbol計算

$$P = \sum_i D_i, \qquad Q = \sum_i a_i D_i,$$

其中加法是XOR，非零係數a_i互異。遺失Di、Dj時，扣掉已知survivors得到 $X = D_i + D_j$、$Y = a_i D_i + a_j D_j$。因 $a_i + a_j \neq 0$ 有乘法逆元素，

$$D_j = (Y + a_i X)/(a_i + a_j), \qquad D_i = X + D_j.$$

這說明兩條獨立方程如何解出兩個erasure；不是把一般整數乘除直接套進bitwise XOR。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## P+Q 的一個可重算例子

採GF(2^4)，模多項式 $x^4 + x + 1$（binary10011），symbol以4-bit多項式表示。乘2是左移；若原最高bit為1，截成4 bits後XOR0011。係數a0=1、a1=2、a2=4。

取D0=1、D1=2、D2=3。P=1 XOR 2 XOR 3=0；Q=(1・1) XOR (2・2) XOR (4・3)=1 XOR 4 XOR 12=9。

若D0、D1同時遺失，X=P XOR D2=3；Y=Q XOR(4・D2)=9 XOR 12=5。故3・D1=Y XOR X=6。域內3的逆為14，因3・14 mod10011=1；6・14=2，得D1=2、D0=3 XOR 2=1。實際RAID採用的symbol大小、係數與layout可不同，本例只證明明定編碼的恢復步驟。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Parity write hole 與一致性

Small write的data與parity是兩個實體寫入，若電源在兩者之間中斷，survivors可能各自讀取正常卻不屬於同一版本。例如old D1=1100、P=0000，先寫new D1=0101後中斷，parity仍0000，已不等於全部data的XOR。

這稱write hole。單靠parity公式無法保證更新原子性。受保護write cache、journal、完整stripe交易與恢復協定可保存重作資訊；各自需定義寫入順序、持久點及開機恢復規則。RAID也不能恢復合法指令造成的誤刪，因此不能取代獨立備份。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Rebuild 的流量分母

12顆10 TB的RAID5，明定每顆有效9 TB，controller只重建有效區域。重建每個遺失block要讀11個surviving blocks，因此讀99 TB、寫replacement9 TB。

若所有讀與寫共用aggregate1 GB/s路徑，時間下界

$$T \geq (99+9)10^{12}/10^{9} = 108000\ \mathrm{s} = 30\ \mathrm{h}.$$

若「1 GB/s」改指replacement輸出速率，則9 TB/1 GB/s=2.5小時，但surviving讀取路徑需同時支援11 GB/s。兩個分母不能混用。若全盤重建而不辨識有效區域，讀110 TB加寫10 TB，在aggregate路徑需至少33.33小時。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 重建期間的第二顆失效機率

明定n=8的RAID5已失去一顆，修復時間固定r=10小時，其餘7顆獨立、每顆固定hazardλ=10^−5/小時。剩餘disk全部活過r的機率為

$$(e^{-\lambda r})^{7} = e^{-7\lambda r},$$

所以這段時間至少再失去一顆的機率

$$1 - e^{-7\times 10^{-5}\times 10} \approx 0.000699755,$$

約0.06998%。這是已進入降級狀態後的條件機率，不是array每年資料遺失率。讀取不可校正錯誤、共同故障、修復時間分布與再次故障需另建模，不能把此值稱完整RAID可靠度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.10 Sun Fire 歷史案例與功耗邊界

原書Sun Fire x4150是2008年前後的1U server案例，列出的最大配置有兩個four-core Xeon sockets、64 GB DRAM、8顆15,000 RPM SAS disks及RAID controller。這些是原書的歷史配置，不代表當代產品。19-inch rack的1U高度為1.75 inches，42U為73.5 inches，還需留供電與散熱能力。

以明定的教學配置而非產品實測，設server idle200 W、滿載400 W、完成2000 transactions/s。滿載每transaction能耗400/2000=0.2 J。若半載仍耗300 W、只完成1000/s，則0.3 J/transaction。把兩部半載整合到一部滿載，只有在容量、尾延遲、容錯與遷移成本可接受時才可能減少總能耗。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 同一歷史配置，不同 workload 的瓶頸

以下另定義教學workload，並非原書效能量測。8個cores各持續10^9 instructions/s，每筆I/O需250,000 user加150,000 OS instructions；每筆64 KiB。CPU上界為8×10^9/400000=20000 IOPS。

假設每disk的random-read平均service4 ms，8顆合計2000 IOPS；sequential每disk持續112 MB/s，合計 $8 \times 112 \times 10^6 / 65536 = 13671.875$ IOPS。單一RAID card的payload link若2 GB/s，則 $2 \times 10^9 / 65536 = 30517.578$ IOPS。

在忽略其他限制、無disk conflict的模型，random上界min(20000,2000,30517.578)=2000，sequential上界13671.875，兩者皆disk先限制。若把裝置換成更快SSD超過20000 IOPS，CPU便可能先限制；歷史配置的圖與新模型的workload不能混成同一份產品benchmark。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.11 Network 的傳輸與等待

10 Gbit/s link、payload efficiency80%，有效頻寬8 Gbit/s=1 GB/s。傳64 MiB需

$$64 \times 2^{20}/10^{9} = 67.108864\ \text{ms}.$$

若每4 MiB為一batch，16 batches之間各需0.20 ms gap，且連線已建立，則15 gaps共3 ms，總70.108864 ms。100條streams可以重疊某些等待，卻不能把共用link的總payload能力變成100 GB/s。

Propagation（訊號傳播）、serialization（把bits送入link）、queueing與protocol setup是不同項。Bandwidth-delay product為頻寬乘RTT，表示維持某速率所需的在途資料量，而不是每筆request自動增加的固定資料量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 6.12 用反例檢查常見推論

「DMA不耗CPU」漏掉setup與completion，且資料仍占memory bandwidth。「RAID6有兩個parity blocks就安全」漏掉獨立編碼與write hole。「MTTF高就一年不會壞」把平均值誤當期限。

「IOPS相同所以效能相同」可由4 KiB與64 KiB反例推翻。「提高queue depth一定降低response time」可由M/M/1表推翻。「RAID就是backup」則無法處理誤刪及攻擊者同步改寫所有copies。

每個反例指出失效的前提，不把特定workload的結論外推成所有裝置或所有版本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 6.13 從觀測值回推限制

若吞吐不再上升而response增加，先檢查各資源利用率、queue length與per-I/O成本。Little定律可檢查三個量是否來自同一穩態，但不能單獨定位哪一段飽和。

若寫入回報很快卻在斷電後遺失，應檢查完成語意與持久路徑；若RAID重建超時，應分開計surviving讀、replacement寫與共用link。故障與效能分析都從清楚的端到端服務規格開始。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 6.14 技術演進與不變的分析方法

磁碟的機械定位、NAND的out-of-place更新與互連的共享資源不同，但都需分開容量、service rate、queueing與failure model。由bus轉向switch、由HDD轉向SSD，只改變主要成本，不消除buffer ownership、completion及持久性問題。

原書的歷史數據適合觀察設計取捨；比較新系統時必須重新取得相同workload與完成條件的數據，不把2008年的裝置峰值當作現代通用常數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料

Patterson與Hennessy，*Computer Organization and Design: The Hardware/Software Interface*，Revised Fourth Edition，6.1–6.14，印刷頁570–618（PDF實體頁597–645）。RAID small-write與layout見Fig6.13、6.14，印刷頁603–604（PDF630–631）。

本章M/M/1與GF(2^4)算例為在明定模型下展開的補充推導，不宣稱是書中原例。練習及完整解答分別在練習與詳解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 為何需要平行處理

單核頻率不能無限提高，原因包括 power density、clock distribution、instruction-level parallelism（ILP，指令層平行）有限與 memory wall。多核把工作分給多個執行單元，目標是提高 throughput 或降低 large problem 的 elapsed time。

多核處理器把多個 cores 放在同一 die；multiprocessor 可以跨 sockets；cluster 用多台 computers 與 network。三者都可稱為平行系統，但 communication latency、coherence 與 failure boundary 不同。

本章的 speedup 指同一問題或同一服務需求下的 elapsed-time 比值。沒有 workload 定義的「N 倍快」不具工程意義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Parallelism 的種類

Job-level/process-level parallelism 讓多個 independent jobs 同時執行，例如數個 web requests。這類 throughput scaling 不要求每題被拆成多份。

Parallel processing program 讓一個 job 使用多個 workers。Newton iteration、matrix multiplication 與 image processing 可能屬於此類，但程式的 dependencies 決定可平行比例。

Instruction-level parallelism 由 pipeline/superscalar 在單一 instruction stream 內找 independent operations；data-level parallelism 對大量元素做相同 operation；task-level parallelism 對不同功能或 requests 分工。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 7.1 Power-aware parallel design

若降低 supply voltage 與 frequency，單核 energy/operation 常下降，但效能也下降。把相同 power budget 分給多個較慢 cores，可能提高 aggregate performance per watt，前提是程式能用到這些 cores。

Parallelism 也提供 availability：一個 processor 失效時，replicated service 可以由其他 nodes 繼續，但這需要 redundancy、failure detection 與 recovery，不等於只要有多核就會自動可靠。

Scalable software 必須能在不同 processor counts、cache capacities 與 network topologies 上維持正確與合理效率。

「小核比較省電」只在指定電壓/頻率與 workload 下有量化意義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 7.2 Amdahl 的固定工作模型

令固定工作的單執行緒 baseline 為 $T_0$。若其中比例 $f$ 為可平行部分，$(1-f)$ 為序列部分，且 $N$ 個 workers 完美分擔平行部分、沒有額外成本：

$$T_N = T_0(1-f) + \frac{T_0 f}{N}.$$

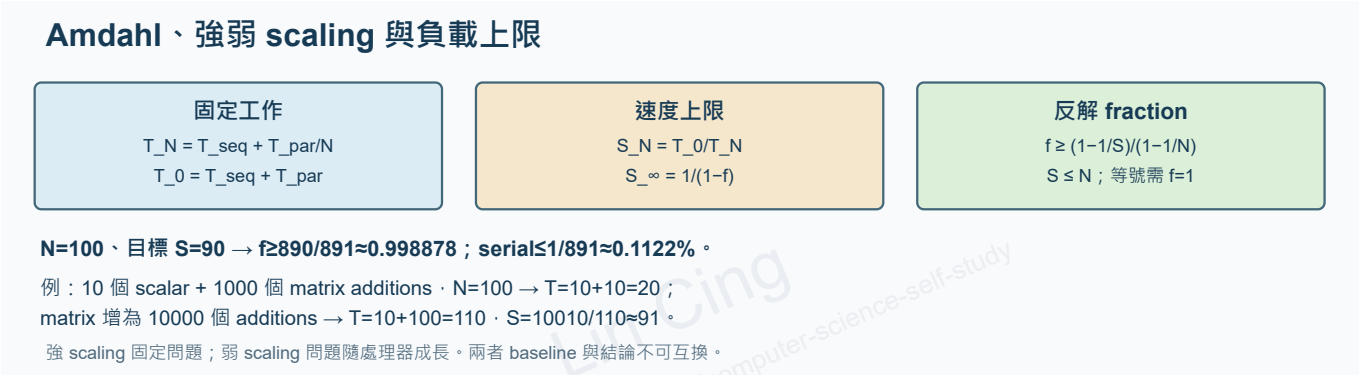
這個式子把「序列部分不縮小」直接寫進時間分解。Speedup 是 baseline time 除以新 elapsed time：

$$S_N = \frac{T_0}{T_N} = \frac{1}{1-f+f/N}.$$

Efficiency 定義為 $E_N = S_N/N$，單位不是 occupancy。若 $S_N = 4.70588, N = 8$，則 $E_N = 0.588235 = 58.8235\%$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 7.2 Amdahl 的固定工作模型 （續）




當 $N \to \infty$：

$$S_\infty = \frac{1}{1-f}.$$

這個上界只有在 workload、baseline 時間比例與成本定義固定時成立。它是 asymptote，不是某一台機器的 measured speedup。



## 7.2 Amdahl 的固定工作模型 （續）

### 從目標 speedup 反解

若已知 $N$ 與目標 $S < N$：

$$f \geq \frac{N(S-1)}{S(N-1)}.$$

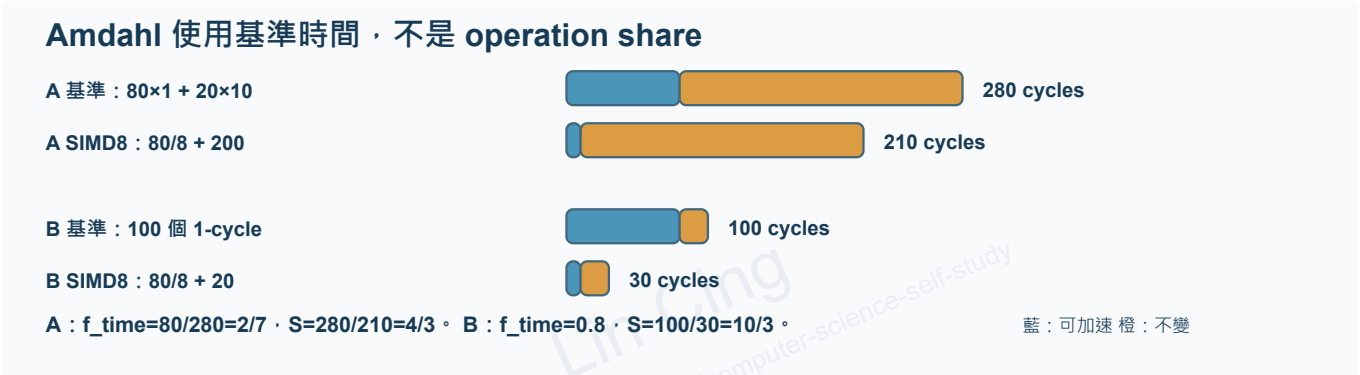
例如 $N = 100, S = 90$，必要 $f \geq 8900/8910 = 0.998878$，序列比例不得超過約 $0.1122\%$。把這個門檻誤解成「90% 可平行」會得到完全不同的答案。

模型沒有包含 synchronization、load imbalance、launch 或 shared-bandwidth contention。實際時間可以寫成

$$T_{N,\text{observed}} = T_N + T_{\text{critical}} + T_{\text{sync}} + T_{\text{contention}},$$

因此 measured speedup 通常低於 ideal Amdahl 模型。只有先驗證這些成本近似可忽略，才能把差值歸因於單一原因。

## 操作比例不等於時間比例




先看100個操作中的80個一cycle操作可向量化，另20個各花10 cycles。基準時間為$80 + 200 = 280$ cycles。8倍向量化後為$10 + 200 = 210$ cycles，故$S = 4/3$，可加速時間比例只有$80/280 = 2/7$。



## 操作比例不等於時間比例 (續)

| 模型 | 基準可加速時間 | 基準其餘時間 | 8倍加速後總時間 | 整體加速 |
|---|---|---|---|---|
| 不等成本A | 80 | 200 | 10+200=210 | 280/210=1.3333 |
| 等成本B，100個操作各1 cycle | 80 | 20 | 10+20=30 | 100/30=3.3333 |

相同80%操作比例可以對應不同時間比例。只有等成本或另外量到80%基準時間時，才可將0.8直接代入Amdahl。

## Strong scaling、weak scaling 與效率

Strong scaling 固定 problem size $W$，增加 processors $P$，問同一工作最快能多快完成。若理想時間是 $T_1/P$，則

$$S_{\text{strong}}(P) = \frac{T_1}{T_P}, \qquad E(P) = \frac{S_{\text{strong}}(P)}{P}.$$

Weak scaling 讓總 problem size 從 $W$ 成長為 $PW$，使每 processor 的工作量固定。弱擴展效率定義為

$$E_{\text{weak}}(P) = \frac{T_1(W)}{T_P(PW)}.$$

理想弱擴展保持相同完成時間，因此效率為1；此時間比不是固定問題的 speedup，也不應再除一次 $P$。若另外假設同一問題的單核成本隨工作量線性，$T_1(PW) = PT_1(W)$，才可寫

$$S_{\text{scaled}}(P) = \frac{T_1(PW)}{T_P(PW)} = \frac{PT_1(W)}{T_P(PW)} = PE_{\text{weak}}(P).$$

若缺少較大問題的單核時間或線性成本假設，不能僅從兩個不同規模的時間推出這個同規模 speedup。若資料量 $M$ 固定，strong scaling 的每處理器記憶體約 $M/P$；若 weak scaling 讓 $M$ 隨 $P$ 成長，每處理器記憶體約維持 $M$。



## Strong scaling、weak scaling 與效率 （續）

### 同一公式的兩個完整案例

假設每次 addition 成本為 $t$，序列工作為 10 次 additions。

| 問題 | $P = 10$ | $P = 100$ |
|---|---|---|
| small：10 serial + 100 parallel | $T = 10t + 100t/10 = 20t$，$S = 110/20 = 5.5$ | $T = 10t + 100t/100 = 11t$，$S = 110/11 = 10$ |
| large：10 serial + 10,000 parallel | $T = 10t + 10,000t/10 = 1010t$，$S = 10010/1010 \approx 9.91$ | $T = 10t + 10,000t/100 = 110t$，$S = 10010/110 = 91$ |

小問題固定時，100 cores 只能得到 10 倍，因序列成本占比變大。問題隨處理器成長後，100 cores 可接近 100 倍。這是 Amdahl 與弱 scaling 能夠同時成立的原因：工作與分母一起改變。

報告至少記錄 problem size、per-core work、worker count、baseline、communication bytes、memory footprint、clock/power policy 與 measured wall time。只寫「99% 效率」而沒有 growth rule，無法重現。

## Load balance 與 parallel critical path

令總 parallel work 為 $W$、最重 worker 的工作量為 $W_{\max}$、processor 數為 $P$、固定 overhead 為 $C$。其他 workers 平均分擔剩餘工作時，parallel phase 的 completion time 下界是

$$T_{\text{parallel}} = \max\left(W_{\max}, \frac{W - W_{\max}}{P - 1}\right),$$

$$T_{\text{total}} = T_{\text{parallel}} + C.$$

### 完整算例

令 $W = 10{,}000$、$C = 10$、$P = 100$。若一個 worker 分到 2%，即 200 units，其他 99 workers 共 9,800 units：

$$T = \max\left(200, \frac{9800}{99}\right) + 10 = \max(200, 98.99) + 10 = 210.$$

Sequential baseline 是 $10,000 + 10 = 10,010$：

$$S = 10{,}010/210 \approx 47.67, \qquad E = 47.67/100 \approx 47.7\%.$$



## Load balance 與 parallel critical path （續）

若重 worker 分到 5%（500 units）：

$$T = \max\left(500, \frac{9500}{99}\right) + 10 = 500 + 10 = 510,$$

$$S = 10{,}010/510 \approx 19.63, \qquad E \approx 19.6\%.$$

一般 worker 的平均工作量不能預測完成時間；最慢 worker 與 dependency chain 都可能成為 critical path。Work stealing 可動態搬移 tasks，但每次 steal 需要 queue/atomic metadata，搬移資料又增加 cache 與 network traffic。Task 太細時，coordination 成本可能超過負載改善。

## 7.3 Shared-memory multiprocessors

SMP（shared-memory multiprocessor，共享記憶體多處理器）給所有 processors 一個 common physical address space。Programs 用 shared loads/stores 溝通，硬體以 cache coherence 維持同一位置的 visible order。

UMA（uniform memory access）模型中，各 processor 到 memory 的 access time 近乎相同。NUMA（nonuniform memory access）中，local memory 比 remote memory 快；program page placement 與 data affinity 會改變效能。

NUMA 系統仍共用同一個 physical address space。NUMA 描述 memory latency 與 bandwidth 的拓樸差異，不是彼此分離的 address spaces。

Directory-based coherence 常用於較大的 SMP/NUMA，snooping 在 broadcast domain 小時較直接。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Shared-memory synchronization

同步必須同時處理三件事：

1. **Atomicity**：read-modify-write 不可被拆成其他人可觀察的中間狀態。
2. **Ordering**：哪些 memory operations 必須排在另一個 operation 前後。
3. **Visibility**：建立 ordering 後，consumer 讀到 producer 寫入的值。

Coherence 只保證同一位置的 cached copies 與 write serialization。跨不同位置的 publication 仍需要 memory consistency model 所定義的 synchronization。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Shared-memory synchronization （續）

### 共享變數 publication：atomicity、ordering 與 visibility

| Writer thread | Reader thread |
| --- | --- |
| data = 42<br>release store ready = 1 | acquire load ready<br>if value == 1: read data |

**正確關係：release 先排序 data store 與 ready store；acquire 讀到該 release 值後，才建立 synchronizes-with；reader 的後續 data load 因此看到 42。**

**獨立 relaxed 版本：data / ready 皆 atomic；ready=1 後仍可讀到 data=0。**

Plain non-atomic concurrent accesses 是 data race；acquire/release 只排序規格涵蓋的衝突存取。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Shared-memory synchronization （續）

### Publication worked example

```
int data = 0;                 // shared object
atomic_int ready = 0;

// Writer
data = 42;
atomic_store_explicit(&ready, 1, memory_order_release);

// Reader
while (!atomic_load_explicit(&ready, memory_order_acquire))
    yield();
int observed = data;
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

當 reader 的 acquire load **實際讀到** writer 的 release store 所寫入的 `1` 時，release 與 acquire 建立 synchronizes-with，再透過 program order 建立跨 thread happens-before。`data=42` 必須先於 `data` load 可見，所以 `observed=42`。

## Shared-memory synchronization （續）

作為可合法討論讀值的錯誤版本，改令 `data` 與 `ready` 都是 atomic，兩者的 store/load 全採 `memory_order_relaxed`。Writer 依序寫 data=42、ready=1；reader 讀到 ready=1 後仍可讀到 data=0，因為不同 atomic objects 之間沒有 synchronizes-with，coherence 只約束各自的 modification order。若兩者改成 plain non-atomic 且沒有其他同步，則形成 C/C++ data race、行為未定義；此時不能宣稱只會得到0或42。

同步只排除它涵蓋的衝突 accesses。若程式另有第三個 thread 在 lock 外修改同一資料，該 race 不會被修好。`atomic` 也不自動讓多個位置的 transaction 原子化。

Spinlock 以 atomic read-modify-write 嘗試取得 lock；contended spin 消耗 interconnect 與 power。Blocking mutex 將 waiting thread 交給 OS，代價是 context switch。Barrier 要所有 participating threads 到達後才釋放；tree barrier 以 $O(\log P)$ 深度換取較小 fan-out，但節點同步與喚醒成本仍存在。



## 7.4 Clusters 與 message passing

Cluster 由多台 commodity nodes 透過 network 連接，每個 node 有自己的 OS 與 physical address space。Communication 由 explicit send/receive 或 RPC/queue 完成，不使用 ordinary shared load/store。

Message passing 讓 sender/receiver ordering 較明確，但需要 serialize data、copy 到 network buffer、route、receive 與 possibly ACK。Failure isolation 較好，recovery 仍要 application framework。

MPP（massively parallel processor）類歷史上使用 specialized network；modern clusters 以 commodity switch 與 software stack 為主，取決於 latency/bandwidth/price。

Cluster 不是免費的 shared memory；distributed shared-memory layer 也建立在 message passing 之上。

## 7.5 Hardware multithreading

Hardware thread 是 CPU 可快速切換的 execution state，分開 PC、register file 與一些 control state；functional units、caches 與 memory 可以共享。它是處理 memory/instruction latency 的 microarchitecture 方法。

Fine-grained multithreading 每 cycle 輪流選 thread，ready thread 也可能被別 thread 分享 issue slots；適合隱藏長短 stalls，但單 thread latency 可能增加。

Coarse-grained multithreading 在 long stall 時才切換 thread；單 thread normal execution 較少被打斷，但可能無法隱藏短 dependency。

Simultaneous multithreading（SMT）在同 cycle 的 superscalar issue slots 中混合多 threads，對 ILP 不足有幫助；代價是 register storage、scheduler、cache pressure。

每個 core 執行兩個 threads 不代表 throughput 加倍；實際結果取決於 issue width、cache 與 workload。



## 7.6 SISD、MIMD、SIMD、SPMD 與 vector

SISD 是一個 instruction stream 作用於一個 data stream；MIMD 是多 processors 各有 independent instruction/data streams。SIMD 讓 one instruction 同時處理多個 data elements；SPMD 是 programming model，由多個 threads 執行同一 program、不同 data。

SPMD 可在 MIMD cluster 或 SIMT GPU 上實作。SPMD 不是固定硬體 classification，不能和 SIMD 當成同一層概念。

SIMD 擴充適合 regular contiguous data：array add、image filter、packed integer。Control flow 複雜或 data dependencies 不規則時，lane utilization 下降。

先列 software model 與 hardware execution model，再談 classification。

## Vector architecture

Vector ISA 使用 vector registers 與 vector length。對 $L$ 個 elements，一條 vector instruction 描述整個 vector operation；硬體以 lanes、chaining 或 pipelining 執行。

Vector code 通常用 vector length 表示 semantic，讓編譯器與硬體處理 tails 和不同 hardware widths。Multimedia SIMD 常固定 128/256 bits，software 要自行處理 padding/tail 與 narrower hardware。

令每個element做$k$ operations，硬體有$L$ lanes且每lane每cycle可完成一個element，frequency為$f$ cycles/s、有效利用率為$\eta$。此模型的throughput為

$$\mathrm{OP/s} = kLf\eta$$

量綱為operations/element × lanes × elements/(lane • cycle) × cycles/s。若每lane initiation interval是$q$ cycles/element，則再除以$q$。另一個等價描述是每條vector instruction處理$W$ elements、每cycle可啟動$r$條，則上限為$kWrf$；vector length $W$不是physical lane數，兩者不可重複相乘。

Vector 的增益來自 amortize control/fetch 與 regular lane usage，不是 lane 數必然等於 real speedup。



## 7.7 GPU 的定位

GPU 有大量較 simple cores、高 memory bandwidth 與 many warps。CPU 偏向少量 latency-sensitive threads；GPU 以 massive multithreading 隱藏 memory latency，適合 high arithmetic/data parallelism。

GPU 的效能來自 throughput，而非 single-thread latency。Host-device transfer、kernel launch、divergence、coalescing 與 occupancy 都會影響 end-to-end time。

本節只建立比較框架；SIMT warp、CUDA memory spaces 與應用 mapping 在 Appendix A 展開。

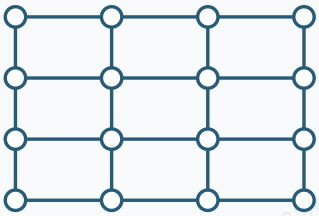
不要把「GPU core 數量較多」直接解讀為單一 scalar program 較快。

## 7.8 Multiprocessor network topologies

把 network 寫成 graph $G = (V, E)$。Node 可代表 processor-memory endpoint，link 可有 bandwidth 與 hop latency。以下都把 full-duplex physical link 計為一條 edge；若報 bisection bandwidth，必須先說每次 link 是只計單向頻寬或雙向加總。

### 7.8(a) 4×4 mesh

**4×4 mesh：24 條 local links，無 wrap**



**Mesh 指標**

horizontal = 4×(4−1) = 12

vertical = 4×(4−1) = 12

edges = 24；average degree = 48/16 = 3

corners degree 2；interior degree 4

diameter = (4−1)+(4−1) = 6

bisection links = 4

沒有第 4 欄→第 0 欄或第 4 列→第 0 列的 link，所以不是 torus。

四列各有 $4 - 1 = 3$ 條 horizontal links，合計 12；四欄各有 $4 - 1 = 3$ 條 vertical links，合計 12：



## 7.8 Multiprocessor network topologies （續）

$$|E_{\text{mesh}}| = 12 + 12 = 24.$$

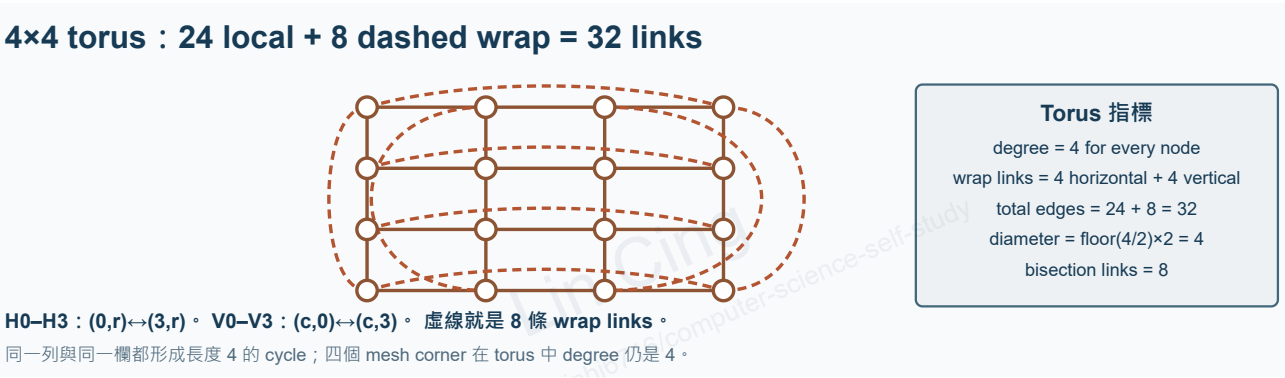
Degree 總和是 $2|E| = 48$，所以平均 degree 是 $48/16 = 3$。角落 degree=2、邊上 degree=3、內部 degree=4。任兩點最遠需水平 3 hops 加垂直 3 hops：

$$D_{\text{mesh}} = 3 + 3 = 6.$$

把四欄切成左 2 欄與右 2 欄時，跨越切面的只有 4 條 horizontal links；其他 balanced cuts 不會更少，因此 bisection links=4。

## 7.8 Multiprocessor network topologies （續）

### 7.8(b) 4×4 torus




Torus 保留 mesh 的 24 條 local links，再把每一列的右端連回左端（4 條），並把每一欄的下端連回上端（4 條）：

$$|E_{\text{torus}}| = 24 + 4 + 4 = 32.$$

每個 node 都有左右上下四條 links，所以 degree=4。任兩 node 在每一維最多繞行 $\lfloor 4/2 \rfloor = 2$ hops：



## 7.8 Multiprocessor network topologies （續）

$$D_{\text{torus}} = 2 + 2 = 4.$$

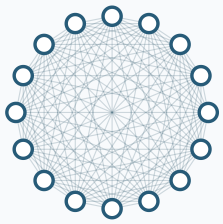
把 columns 0–1 與 columns 2–3 分開，跨切面是 4 條 local horizontal links 加上 4 條 wrap links，共 8。程式對全部 balanced cuts 取最小，得到 bisection links=8。

圖中的虛線不是裝飾：$H0$–$H3$ 是四列各自的 $(0, r) \leftrightarrow (3, r)$，$V0$–$V3$ 是四欄各自的 $(c, 0) \leftrightarrow (c, 3)$。少了這 8 條，圖就只是 mesh，degree 與 diameter 都不再符合 torus。

## 7.8 Multiprocessor network topologies （續）

### 7.8(c) K16 與 crossbar 的模型差異

**K16：16 個 endpoints 兩兩直接相連**



**K16 指標**

edges = N(N−1)/2 = 16×15/2 = 120

degree = 15

diameter = 1

bisection = (N/2)² = 8² = 64 links

單向 cut capacity = 64×link rate

**K16 的 endpoint graph 不是 central crossbar；crossbar 是另一種 switch 資源模型。**

若每條 link 是 full-duplex，雙向 cut 合計為 2×64×單向 rate；報告先固定加總慣例。

$K_{16}$ 是完全圖：

$$|E| = \frac{16(15)}{2} = 120, \quad \deg(v) = 15, \quad D = 1.$$

任一 balanced cut 有 $8 \times 8 = 64$ 條跨越 links；若 link 的單向頻寬是 $b$，單向 cut capacity 是 $64b$。



## 7.8 Multiprocessor network topologies （續）

Central crossbar 不是 $K_{16}$。$N$-port crossbar 的 endpoint 通常各接一個 switch port，內部有 $O(N^2)$ crosspoints；它的 endpoint degree、線路長度、仲裁成本與 cut bandwidth 必須用 switch 模型另算。

Total network bandwidth 是「每條 link 頻寬 × link 數」的上界；bisection bandwidth 則取所有 balanced cuts 的最小 cut bandwidth，較接近 worst-case pattern。兩者都不足以單獨預測 application latency；hotspot、routing、message size 與 flow control 都會改變結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 7.9 Multiprocessor benchmarks

Parallel benchmark 要固定程式、data set、validation、compiler flags 與 processor count。Weak scaling benchmark 可讓 problem size grow，但要記錄 growth rule。

Linpack 是 dense linear algebra 與弱 scaling 常見例；SPECrate 適合 independent job throughput；SPLASH 類 benchmark 研究 shared-memory application patterns。

Peak performance 與 measured application performance 常不同。Memory bandwidth、load imbalance、communication 與 synchronization 可能占主要時間。

對 parallel systems 要看 per-core efficiency、scaling curve、tail latency、energy 與 fault tolerance，不只 single FLOPS。



## 7.10 Roofline 的推導與數值工作點

令某 kernel 一個 phase 執行 $F$ 次 floating-point operations，跨 memory 介面實際搬移 $Q$ bytes。Arithmetic intensity 定義為

$$I = \frac{F}{Q} \quad (\text{FLOP/byte}).$$

若 compute 的 sustained peak 是 $P_C$，該 memory 介面的持續 bandwidth 是 $B$，同一 phase 的 compute 與 memory times 分別是

$$T_{\text{compute}} = \frac{F}{P_C}, \qquad T_{\text{memory}} = \frac{Q}{B} = \frac{F}{IB}.$$

若 compute 與 traffic 可以完全重疊，elapsed time 不可能小於兩者的最大值：

$$T \geq \max\left(\frac{F}{P_C}, \frac{Q}{B}\right),$$

所以 throughput 上界為

$$P \leq \frac{F}{T} \leq \min(P_C, IB).$$

## 7.10 Roofline 的推導與數值工作點 （續）

令 $IB = P_C$ 可求兩條 roof 的折點：

$$I_{\text{ridge}} = \frac{P_C}{B} \quad (\text{FLOP/byte}).$$

$I < I_{\text{ridge}}$ 是 memory-bound；$I > I_{\text{ridge}}$ 是 compute-bound。圖的上界不代表實測一定達到，optimization 仍需處理 dependencies、instruction mix、load imbalance 與 memory transaction efficiency。

### 單層工作點

取 $P_C = 100$ GFLOP/s、$B = 50$ GB/s，ridge 是

$$I_{\text{ridge}} = 100/50 = 2.$$

Kernel A：$I = 0.5$，memory roof $IB = 25$，compute roof 100：

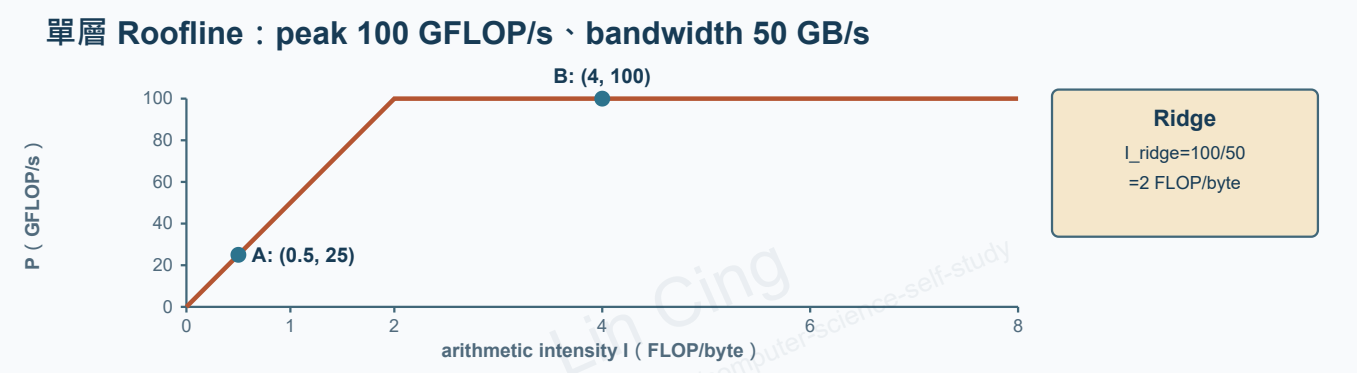
$$P_A \le \min(100, 25) = 25 \ \text{GFLOP/s}.$$

Kernel B：$I = 4$，memory roof $IB = 200$，但 compute cap 100：

$$P_B \le \min(100, 200) = 100 \ \text{GFLOP/s}.$$



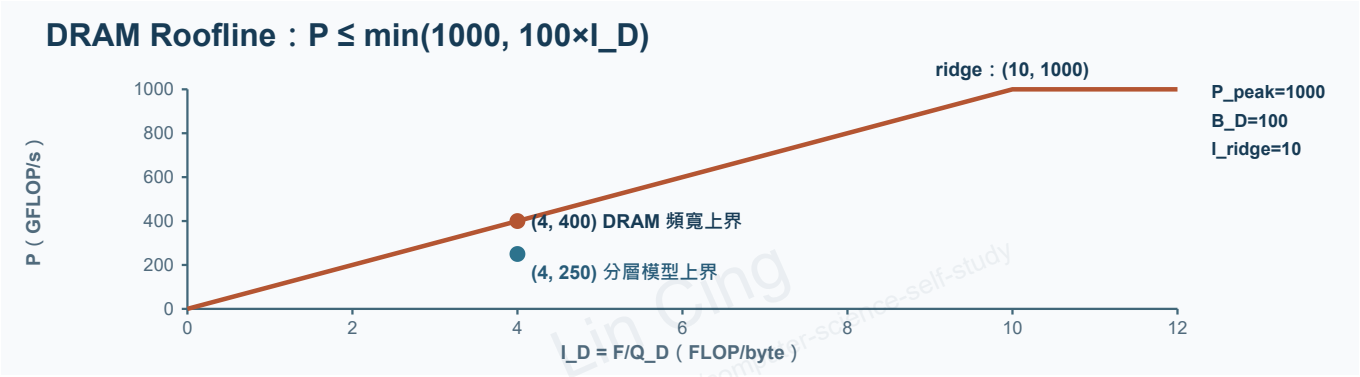
## 7.10 Roofline 的推導與數值工作點 （續）




### 階層 Roofline 的同一組數字

固定一階段 $F = 10^9$ FLOPs、$P_C = 1000$ GFLOP/s。DRAM 的 $Q_D = 0.25 \times 10^9$ bytes、$B_D = 100$ GB/s；shared memory 的 $Q_S = 4 \times 10^9$ bytes、$B_S = 1000$ GB/s。

## 7.10 Roofline 的推導與數值工作點 （續）



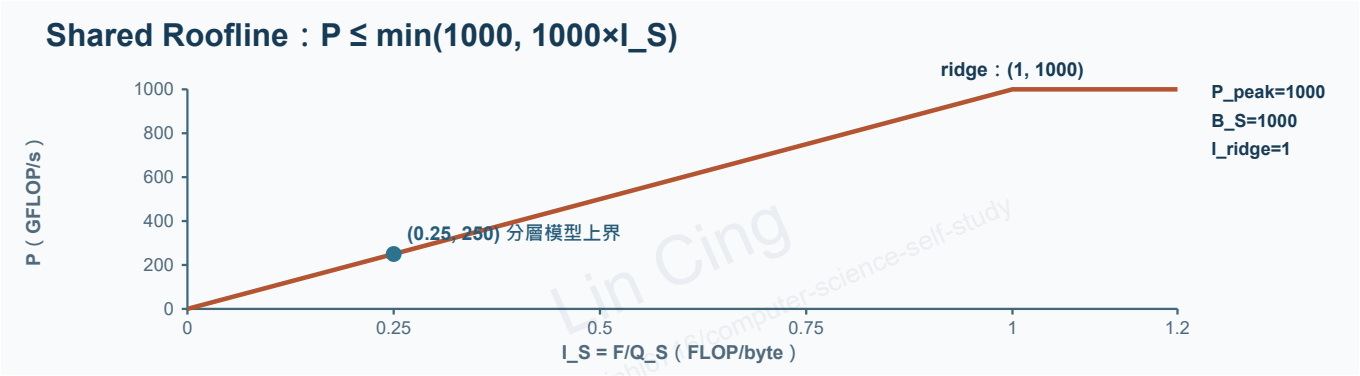

DRAM 層：

$$I_D = \frac{10^9}{0.25 \times 10^9} = 4, \qquad \min(1000, 100 \cdot 4) = 400 \text{ GFLOP/s}.$$



## 7.10 Roofline 的推導與數值工作點 （續）

shared 層：




$$I_S = \frac{10^9}{4 \times 10^9} = 0.25, \qquad \min(1000, 1000 \cdot 0.25) = 250 \text{ GFLOP/s}.$$

把三層限制合併：

$$P \le \min(1000, 400, 250) = 250 \text{ GFLOP/s}.$$

## 7.10 Roofline 的推導與數值工作點 (續)

或等價地比較時間：

$$T \geq \max\left(\frac{10^9}{10^{12}}, \frac{0.25 \times 10^9}{100 \times 10^9}, \frac{4 \times 10^9}{1000 \times 10^9}\right) = \max(0.001, 0.0025, 0.004) = 0.004 \text{ s}.$$

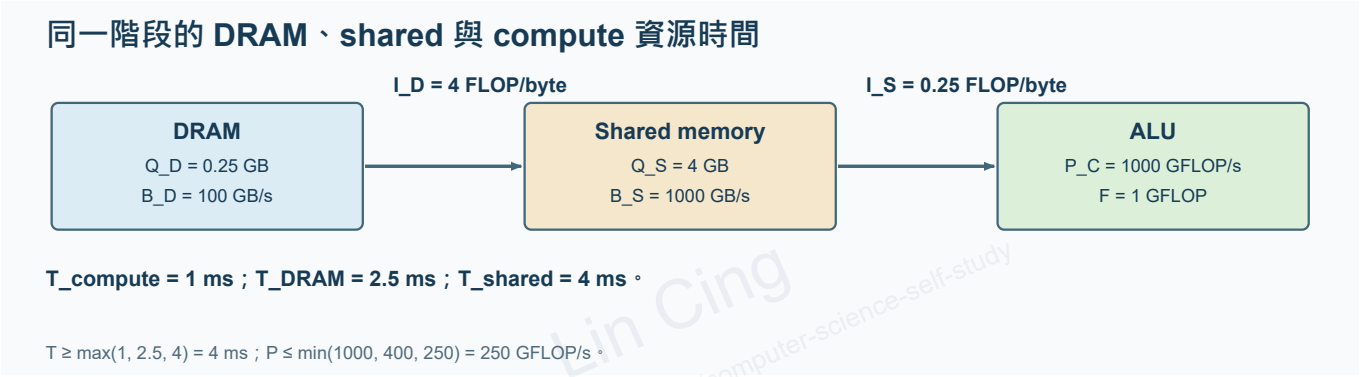
也就是 time lower bound 是 4 ms。

因此在這個明定模型中，DRAM 的工作點是 $(4, 400)$，但 shared 層把跨層上界壓到 $(4, 250)$。250 是模型上界，不是 measured performance；shared roof 本身仍以 1000 GFLOP/s 為 plateau，不能把 250 誤畫成整個 plateau。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 7.10 Roofline 的推導與數值工作點 (續)




若資料已被 cache 命中，就必須用該 cache 介面的 bytes 與 bandwidth 重算，不能把 DRAM 的 $Q_D$ 和 cache bandwidth 混用。多層 roof 的圖可以分開畫，但最終 bound 要取同一 phase 各資源的最小值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 7.11 Real multicore comparison

本章案例以 four multicore systems 與 sparse matrix、structured grid kernels 展示 roofline 的實用方式。不同 systems 的 peak FLOPS、memory bandwidth、cache/TLB 與 programming model 不同；measured point 與 roof 的距離指出 optimization headroom。

Sparse matrix 的 indirect access 與 structured grid 的 stencil access 有不同 locality，因此即使 arithmetic intensity 相近，pipeline/coalescing/compiler quality 仍造成差異。

歷史結論不能當成今日 benchmark；本章保留 metric 與 interpretation 方法。

圖上較高 peak 且較慢 measured kernel 很常見；peak 是外框，不是可交付應用效能。



## 7.12 Parallel fallacies

「Amdahl's law 不適用」常源於拿 weak scaling 去反駁 strong-scaling claim；fixed-work model 沒有規定無限 speedup forever。

「peak FLOPS 決定最快系統」忽略 memory bound、load imbalance、communication 與 software stack。

「more threads 必然更快」忽略 synchronization, oversubscription, cache thrashing 與 power。

「coherent shared memory 讓 race-free」不成立；data races 仍需要 synchronization 與 memory model。

前述80/20不等成本例否定「operation share等於time share」；單一worker分到5%的例把100核speedup壓到19.63；publication trace則指出coherence無法替代跨位置同步。這些都是明定模型的反例，不冒充實機量測。

## 7.13 Concluding remarks

Parallel hardware 提供 resources；parallel software 把它們轉成具體 speedup。Strong/weak scaling、load balance、communication、synchronization、coherence 與 benchmark 設計共同決定結果。

Shared-memory 與 message-passing 是 different address-space models。Multithreading、SIMD、GPU 與 clusters 各以不同成本隱藏 latency 或增加 throughput。

Roofline 將 compute roof 與 memory roof 結合，能指出 kernel 是 compute-bound 或 memory-bound；它只給 upper bound，實測仍需 performance counter 與 end-to-end benchmark。

若 DRAM roof 是最低上限，且工作集、資料流量與其他限制固定，提高 compute peak 不會提高該上限。可用相同完整工作階段的 DRAM bytes、持續 bandwidth 與執行時間檢查此預測；若資料重用改變，須先重算 intensity。



## 章末驗收

應能：

1. 給定 parallel fraction、worker count 與 overhead，計算 speedup/efficiency。
2. 分開 strong/weak scaling 的 problem-size 假設與 conclusion。
3. 說明 SMP、NUMA、cluster 的 address space 與 communication cost。
4. 比較 fine/coarse multithreading 與 SMT 的 stall hiding。
5. 計算 mesh 的 degree、diameter、bisection links。
6. 從 peak FLOPS、DRAM bandwidth 與 arithmetic intensity 判定 roofline bound。

先完成本章獨立練習題 PDF，再由詳解 PDF 檢查假設與推導。

## 7.14 平行架構的歷史視角

向量處理器用少量vector instructions描述大量規則運算，攤薄fetch與控制成本；shared-memory multiprocessors讓既有load/store程式模型延伸到多個processors，但需coherence及同步。Clusters以獨立節點和明確訊息傳遞換取可擴充資源與失效邊界，通信成本則直接進入演算法設計。

晶片整合增加後，multicore、SMT與GPU分別利用task、thread與data parallelism。它們沒有消除serial critical path，也沒有讓所有通信變成本地。比較歷史設計時應固定問題規模、數值驗證、記憶體容量、通信模式及power budget，才能分辨技術進步與工作負載改變。

完全連接graph與crossbar不同。K16每個endpoint連15條直接links，balanced cut有8×8=64條links；16-port crossbar通常每endpoint只連一個switch port，內部有$O(N^2)$crosspoints，其吞吐還受endpoint port rate限制，不能把兩者degree與cut-link數混成同一表。



## 參考資料

Patterson 與 Hennessy，*Computer Organization and Design: The Hardware/Software Interface*，Revised Fourth Edition，7.2、7.8–7.10，印刷頁 635–674（PDF 頁 662–701）。

Williams、Waterman 與 Patterson，〈Roofline: An Insightful Visual Performance Model for Multicore Architectures〉，*Communications of the ACM* 52(4), 2009，DOI。

分層流量定義另參照 NERSC Roofline 文件；每個層級以該介面的 bytes 計算自己的 intensity。本文數字是明定模型的代入與推導。

## 參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A.1 大量平行工作如何對應到 GPU

GPU以大量簡單執行單元與高頻寬處理許多相似工作；CPU較重視複雜控制與單一工作延遲。這個取捨不表示GPU的任一thread必定比CPU thread快，也不保證小資料量足以攤提launch與transfer成本。

Graphics與general compute共用可程式化處理器，但graphics還依賴rasterizer、texture與ROP等固定功能。Logical pipeline是API所見階段，physical SM不必每個階段各專用一批cores。

本章先固定CUDA/Tesla歷史架構的語彙；32-byte sector、32-bank及occupancy數例另有明定抽象模型，不能把它們拼成一顆不存在的GeForce 8800。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Graphics pipeline 中的插值

Vertex shader產生頂點位置與attributes，組成triangles，經clipping、rasterization後產生fragments；fragment shader計算顏色，再由depth/stencil、blending等規則更新framebuffer。Fragment不保證最後成為可見pixel。

取螢幕三角形(0,0)、(2,0)、(0,2)，三個clip w皆1。點(0.5,0.5)的barycentric weights為(1/2,1/4,1/4)，和為1且非負，因此在三角形內。若頂點顏色為純R、G、B，線性插值色為(0.5,0.25,0.25)。

若各頂點w不同，透視校正需以 $(\lambda_i/w_i)/\sum_j(\lambda_j/w_j)$ 作attribute權重，不能繼續使用相同螢幕線性權重。Texture filtering再讀取鄰近texels；ROP可能還要讀舊color/depth，故pixel traffic不只最終color的一次store。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A.2 Host、device 與端到端成本

離散GPU經I/O link與host相連，有自己的DRAM；driver管理命令、映射與completion。Integrated/shared-memory設計可能減少copy，但仍須定義可見性、權限與共享頻寬。統一位址空間不等於每個byte都無搬移成本。

明定序列執行、無copy/compute overlap：輸入64 MiB、輸出64 MiB，有效link每方向8 GB/s，launch20 µs，kernel2 ms。兩次transfer共

$$2 \times 64 \times 2^{20}/(8 \times 10^{9}) = 16.777216\ \text{ms}.$$

總時間18.797216 ms。CPU若25 ms，端到端speedup約1.330；只看kernel會誤報25/2=12.5。若資料留在device執行10次相同kernel，只付一次in/out，則總36.977216 ms；成本是否能攤提取決於真正資料相依與使用情境。



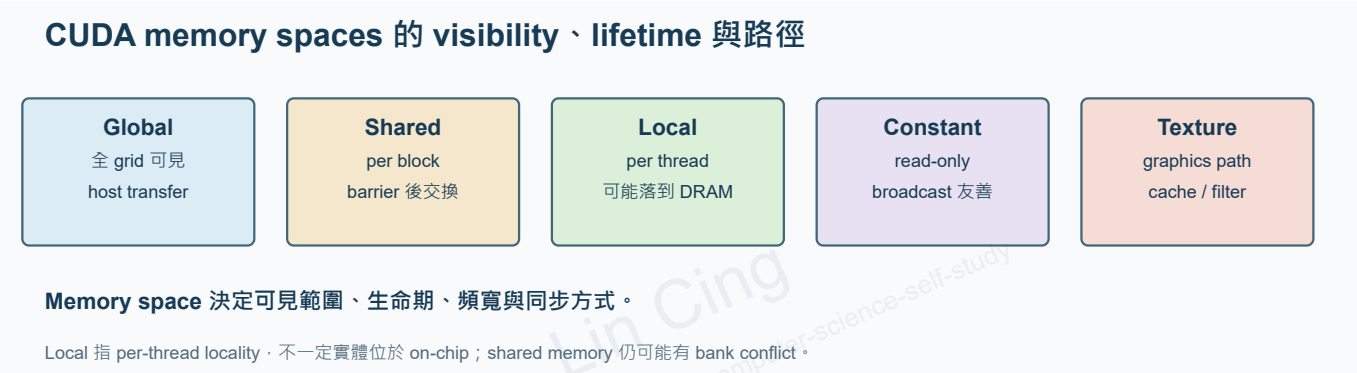
## A.3 Grid、block 與 thread 的索引

一維kernel以 $i = blockIdx.x \cdot blockDim.x + threadIdx.x$ 對應輸出。N=1000、block=256需ceil(1000/256)=4 blocks；前三個處理0–767，最後block的threads0–231處理768–999，其餘24個不得訪問越界資料。

```
i = block_id * B + thread_id
if i < N:
    y[i] = f(x[i])
```

每個有效i唯一對應(⌊i/B⌋, i mod B)，所以覆蓋所有輸出且不重複寫。這段可直接跳過無效threads，因為沒有block barrier。若後續有共同barrier，不能照抄early return，必須讓需參與的threads一致到達。

## Memory space 的 visibility 與 lifetime




Registers與local memory都是per-thread private，但local通常位於device DRAM，例如register spill或可動態索引的private array。Shared memory是per-block scratchpad，block結束後生命期結束。Global memory可跨blocks使用，但資料可見性需同步規則。



## Memory space 的 visibility 與 lifetime （續）

Constant memory是device程式讀取的常數空間；同一warp讀相同位址可利用broadcast。Texture path提供特定格式、filtering與cache語意，不等於任意load都自動有相同性能。空間名稱描述程式模型，不單獨保證實體延遲。

## Barrier 不只是等待計時器

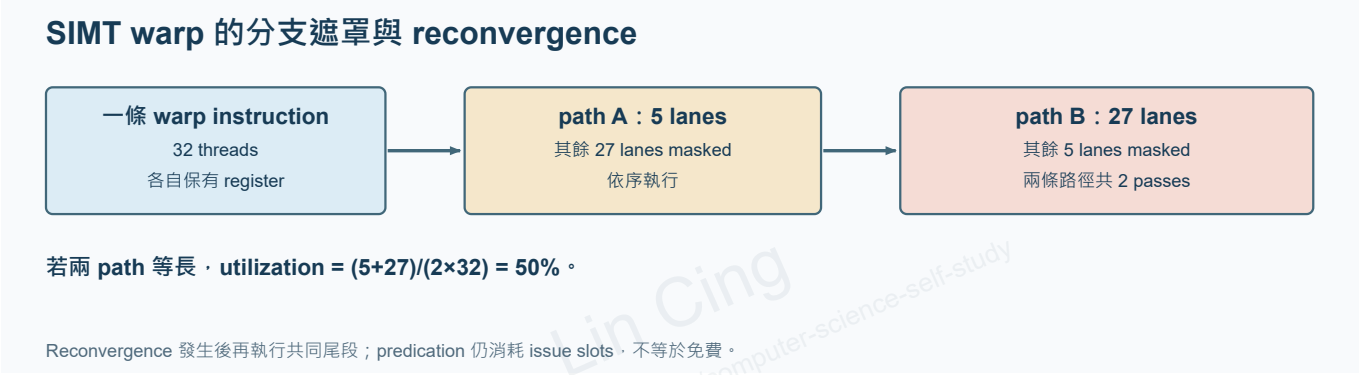
Block barrier要求參與群組按規格到達，並建立對應記憶體操作的可見性。它不能替代跨block的全域同步，也不能假設「warp內看起來同時執行」就不需規格要求的同步。

對N不是block size倍數的tile，正確形式是所有threads先以predicate載入有效值或identity，全部到barrier，只有有效輸出thread計算，然後再次全部到barrier。第二次barrier防止快thread覆寫shared tile，而慢thread仍在讀上一輪。

不同blocks的執行順序與同時resident數不保證。若resident block spin等尚未排入的block，可能占滿所有資源而永遠無法前進；多kernel phases用前一kernel完成作全域邊界，比無條件跨block忙等更容易建立正確性。



## A.4 SM、warp 與 scoreboard




Warp是共同issue的一組threads，threads保有自己的register與logical狀態。歷史Tesla例warp32 threads、8個SP cores，以4個core cycles服務一個warp的32 lanes；這是吞吐寬度，不代表所有指令的result latency都是4 cycles。

## A.4 SM、warp 與 scoreboard （續）

Scoreboard追蹤operand相依，scheduler只issue已ready的warp。某warp等待memory或相依結果時，可選另一個ready warp。Resident warps只是已配置資源的集合；若全部都在等barrier或同一memory事件，occupancy高仍可能無指令可issue。

PTX是virtual ISA，還需對目標架構編譯/JIT成實際machine code。PTX一條指令不必等於一個cycle或一條相同長度的SASS，故性能分析需分清模型層級。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Latency hiding 的有限模型

假設每4 cycles可issue一個warp instruction，每個warp只有一條長度32 cycles的dependent chain，無其他瓶頸。Round-robin選8個ready warps時，同warp相隔8×4=32 cycles，剛好可等前一結果；只有4個warps則相隔16 cycles，還需等16 cycles。

若依賴延遲增加到400 cycles，這個「每warp僅一個outstanding依賴」模型需要100個warps才能完全遮蔽，可能超過硬體容量。增加ILP、同時發出多個獨立load、改善cache與memory並行度，會改變所需warps數；不能只靠把occupancy填滿保證無stall。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Divergence 的加權 lane-slot 計算

兩個互斥paths各有a、b個active lanes，a+b=W；path長度LA、LB指令。忽略branch與reconvergence額外成本，若兩條皆需執行，

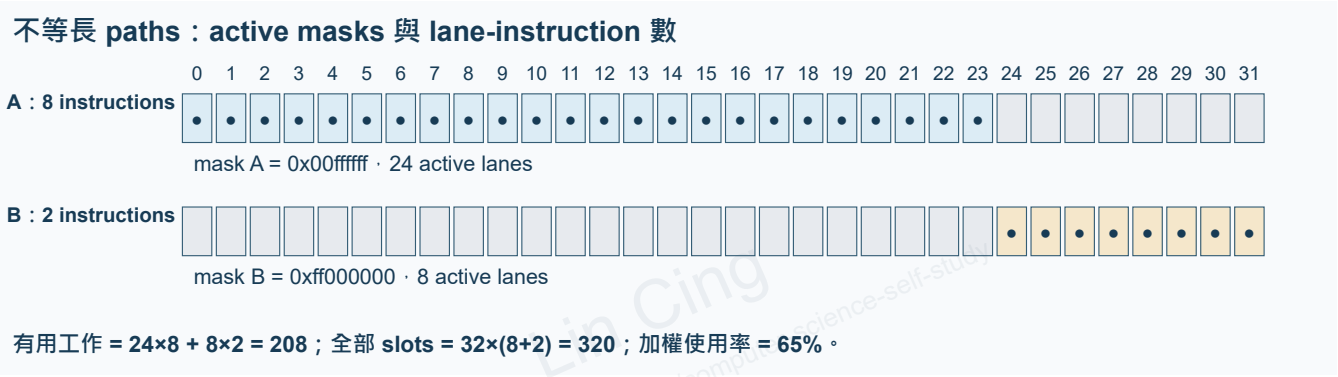
$$\eta = \frac{aL_A + bL_B}{W(L_A + L_B)}.$$

只有LA=LB時才可簡化成1/2。取W=32、a=24、b=8、LA=8、LB=2，有效lane-instructions=24×8+8×2=208，issue capacity=32×10=320，所以η=65%。這是指令lane使用率，不直接等於整個kernel speedup或DRAM效率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Divergence 的加權 lane-slot 計算 （續）




若某path完全無active lane，就不需執行該path，分母不能仍無條件加其指令數。Predication也需issue受predicate控制的指令，inactive memory操作是否形成transaction須依ISA語意，不能把false lane當成一定讀取DRAM。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Occupancy 是資源限制的整數問題

採抽象SM上限768 threads、24 warps、8 blocks、32768個32-bit registers、48 KiB shared memory，warp32。忽略allocation granularity。每block256 threads、每thread32 registers、每block16 KiB shared。

$$blocks \le \min\left(\left\lfloor \frac{768}{256} \right\rfloor, 8, \left\lfloor \frac{32768}{256 \times 32} \right\rfloor, \left\lfloor \frac{48}{16} \right\rfloor\right) = 3.$$

Active warps=3×8=24，occupancy100%。若block改128但shared仍16 KiB，shared只允許3 blocks，active warps12，occupancy50%。若shared也降8 KiB，可放6 blocks，回到24 warps。若原256-thread版register提高到64/thread，則register只允許2 blocks，occupancy16/24=2/3。

這些是resident上界，非實測利用率。Register減少若引起spill，可能增加DRAM流量，使較高occupancy反而更慢。



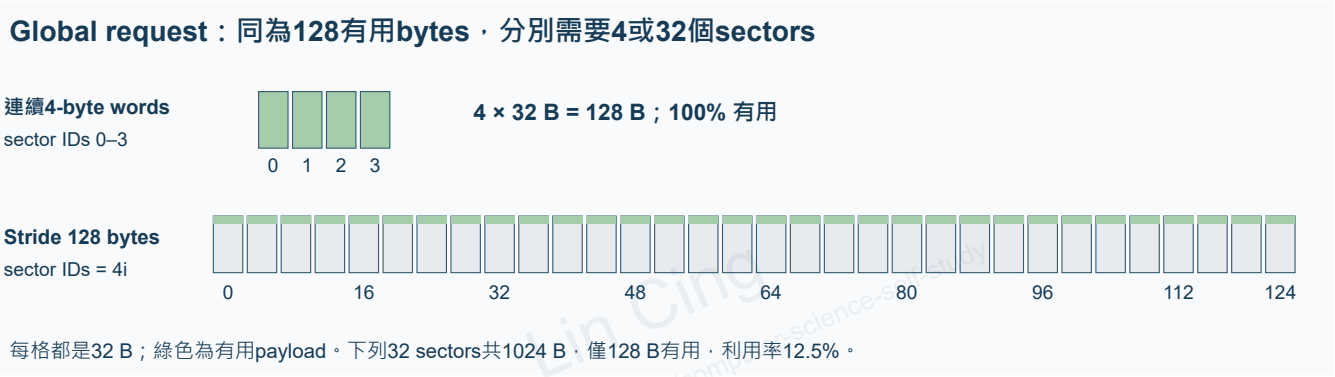
## A.5 Coalescing 的可重算交易模型

本節另採32-byte sector、32 lanes、每lane一個4-byte aligned word，只計一次warp request觸及的不同sectors，忽略cache命中與跨層合併；不是GeForce 8800的完整coalescing規則。

對byte address ai，需要sector集合 $\{\lfloor a_i/32 \rfloor\}$。連續words且base32-byte aligned時，128 bytes恰占4 sectors；base偏4 bytes時占5 sectors，讀取160 bytes但有用128，利用率80%。

Stride32 bytes或128 bytes時，每lane落在不同sector，都是32 sectors、1024 bytes，有用128 bytes，利用率12.5%；不同stride雖sector數相同，channel/row locality仍可不同。

## A.5 Coalescing 的可重算交易模型 （續）




若全部lanes讀同一word，unique requested bytes只有4，可能由一個sector供應再broadcast；不能把32次邏輯使用誤算成128個不同payload bytes。



## Shared banks、broadcast 與 padding

採32個banks，每bank word4 bytes，bank=word index mod32。同一request內不同banks可並行；同bank不同word需多次服務。本模型對同一word的多lane read可broadcast，不把它當成不同word的衝突。

32×32 row-major array，column c=0的lane r讀index=32r，32個不同words都在bank0，形成32-way conflict。改成leading dimension33，index=33r，bank=r，分散到32個banks。

## Shared banks、broadcast 與 padding （續）

Padding把shared用量由4096增至4224 bytes，可能使每SM可容納的blocks數跨過整數門檻。歷史Tesla用16-bank/half-warp服務規則，應以16個同時requests重算；不能把本節32-way結果直接標成該晶片實測。



## A.6 浮點運算的次序也是模型

IEEE binary32有24-bit有效精度。取a=1+2^−23、b=1−2^−23、c=−1，三者可表示。精確ab=1−2^−46，若先捨入binary32 product，得到1，再加c得0。

若採一次捨入的IEEE FMA，先在精確乘加層計算ab+c=−2^−46，再捨入；此值可精確表示，所以結果是−2^−46，與兩次捨入不同。歷史G80 MAD不能無條件當成這種FMA，其subnormal、近似函數與例外支援也有版本限制。

Reduction改變加法樹也可能改變結果。例如binary32的(10^8+1)+(−10^8)得到0，而(10^8−10^8)+1得到1。平行演算法的整數正確性證明若依結合律，不能直接推出浮點bitwise相同；需另外指定誤差與可重現性要求。

## A.7 GeForce 8800 的歷史結構

原書FigA.7.1的GeForce 8800例有8個TPC，每TPC兩個SM，每SM8個SP，因此8×2×8=128 SP；記憶體端有6個64-bit DRAM partitions，合計384-bit介面。SM另有shared memory、constant cache及SFU；texture與ROP負責不同資料路徑。

這個乘積是該圖的結構核對，不是今日GPU效能預測。不同世代可改變cache、warp排程、FP支援及memory transaction規則；core數相同也可能因clock、instruction mix與資料流不同而有不同throughput。SFU與ALU的peak只有在規格允許相應並行且workload真的使用時才可共同計數，不能把所有峰值直接相加。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A.8 CSR SpMV 的矩陣與 arrays

取

$$A = \begin{bmatrix} 2 & 0 & -1 & 0 \\ 0 & 3 & 0 & 4 \\ 5 & 0 & 0 & 6 \end{bmatrix}, \qquad x = (1,2,3,4)^T.$$

CSR使用Ap=[0,2,4,6]、Aj=[0,2,1,3,0,3]、Av=[2,−1,3,4,5,6]。Row i的非零項位於[Ap[i],Ap[i+1])，所以y0=2・1−1・3=−1，y1=3・2+4・4=22，y2=5・1+6・4=29。

Ap長度rows+1，最後值等於nnz；Aj皆在[0,columns)。矩陣可為非方形，不能把rows當作x的合法長度。本例x長4，輸出y長3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Row-per-thread 的正確性與不均衡

```
row = block_id * B + thread_id
if row < rows:
    total = 0
    for k = Ap[row] .. Ap[row+1]-1:
        total += Av[k] * x[Aj[k]]
    y[row] = total
```

每個row只有一個writer，loop invariant是total等於本row已處理非零項的乘積和；讀完區間即得該row點積。Empty row的區間長0，輸出0。

若同warp的31個rows各1項，另一row有64項，依masked loop的簡化模型要64輪，active lane-iterations=31+64=95，capacity=32×64=2048，利用率約4.64%。Warp-per-row可把長row的entries分給lanes再reduction，但短row可能浪費更多lanes；shared cache x也只在重用超過載入/同步成本時才有利。



## Inclusive 與 exclusive scan

對a=[3,1,7,0,4,1,6,3]，inclusive scan包含當前元素，結果[3,4,11,11,15,16,22,25]；exclusive scan只含之前元素，結果[0,3,4,11,11,15,16,22]。

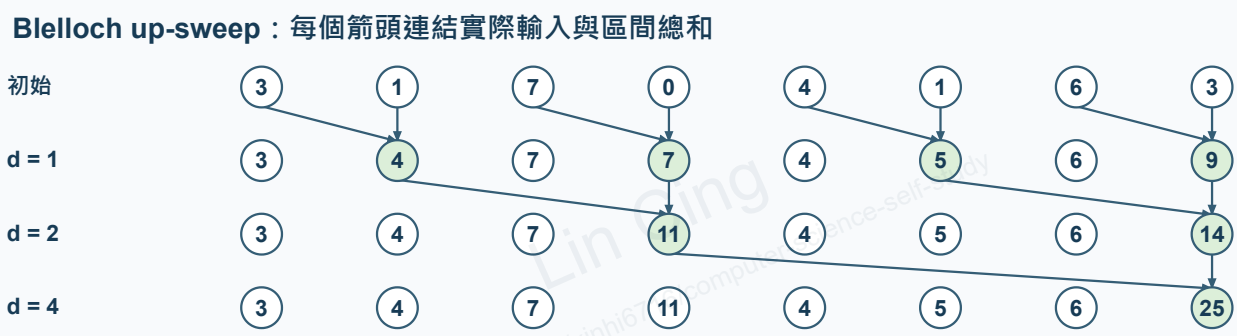
Hillis–Steele每輪從上一輪snapshot讀取i−d，d=1,2,4。實際狀態為

| d | 輪結束的array |
|---|---|
| 初始 | [3,1,7,0,4,1,6,3] |
| 1 | [3,4,8,7,4,5,7,9] |
| 2 | [3,4,11,11,12,12,11,14] |
| 4 | [3,4,11,11,15,16,22,25] |

不使用snapshot或正確同步就直接原地改寫，可能讀到本輪已更新的值。此例加法數7+6+4=17；一般work Θ(n log n)，span Θ(log n)，不能只因depth小就稱work-efficient。

## Work-efficient scan 的 up-sweep

Blelloch scan在完全二元樹先建立各區間總和。對n=8，d=1時把每對左值加到右端；d=2時合併兩對；d=4時得到整體總和。






## Work-efficient scan 的 up-sweep （續）

| 合併距離d | 更新後array |
|---|---|
| 1 | [3,4,7,7,4,5,6,9] |
| 2 | [3,4,7,11,4,5,6,14] |
| 4 | [3,4,7,11,4,5,6,25] |

每個internal node恰算兩個互不相交子區間之和，依樹高歸納得到該node的區間總和；共n−1次加法，depth log2 n。

## Down-sweep 傳遞區間之前的總和

將root總和25另存，再把root設0。若parent持有其區間之前的prefix p，left subtree總和為t，則left child獲得p，right child獲得p+t。這兩個prefix分別恰為左右子區間之前的總和。

| 步驟 | array |
|---|---|
| root設0 | [3,4,7,11,4,5,6,0] |
| d=4 | [3,4,7,0,4,5,6,11] |
| d=2 | [3,0,7,4,4,11,6,16] |
| d=1 | [0,3,4,11,11,15,16,22] |

沿樹向下歸納，leaf i取得全部先前元素的和，故輸出exclusive scan。Down-sweep每internal node一次加法，共n−1；總work2(n−1)、span2log2 n，另計swap與root初始化。對非2次方n，以identity 0補到下一個2次方；n=0直接回傳空輸出。



## 可重建的 shared-memory scan 偽碼

以下n為2次方，所有參與threads執行相同輪數；每輪互不重疊的k可平行。

```
for d = 1, 2, 4, ..., n/2:
    parallel for k = 0, 2*d, 4*d, ...:
        x[k+2*d-1] += x[k+d-1]
    block_barrier()
if thread_id == 0:
    total = x[n-1]
    x[n-1] = 0
block_barrier()
for d = n/2, ..., 2, 1:
    parallel for k = 0, 2*d, 4*d, ...:
        t = x[k+d-1]
        p = x[k+2*d-1]
        x[k+d-1] = p
        x[k+2*d-1] = p + t
    block_barrier()
```

`total`只需一個指定thread保存，再依需要發布。這個原地scan不需要double buffer；Hillis–Steele的snapshot需求與此演算法不同，不能混用其同步規則。

## 多 blocks 的 scan 與全域邊界

N=10、每block4個數，輸入[3,1,7,0,4,1,6,3,2,5]。各block補0做local exclusive scan，block sums為[11,14,7]；對block sums再做exclusive scan得offsets[0,11,25]。

各local結果加自己的offset後，完整輸出為[0,3,4,11,11,15,16,22,25,27]，總和32。Phases依序為local scan與sums、scan sums、加offset；sums很多時可遞迴分層。Kernel邊界建立跨block完成順序，不要求所有blocks同時resident。

固定block size下，各層資料量形成幾何級數，所以總work O(N)；抽象平行模型的tree span為O(log N)，實際時間還含每層launch、memory traffic與有限SM排程。



## Reduction 的不變量與數值

同一個8元素array，可用stride4、2、1，把後半加到前半。每輪前半各位置代表互不重疊子集合的和，合併保持此性質，最後x0為全部總和25。

| stride | 本輪有效partial sums |
|---:|---|
| 4 | [3+4,1+1,7+6,0+3]=[7,2,13,3] |
| 2 | [7+13,2+3]=[20,5] |
| 1 | [20+5]=[25] |

總加法4+2+1=7。多block可先輸出block sums再reduction，或用規格支援的atomic操作合併。Atomic保證單次更新不互相遺失，不保證不同blocks的浮點相加順序固定。

## Stable radix partition 的每個目的位址

採3-bit unsigned keys [7,2,5,0,3,4,1,6]。處理bit0時，定義z_i=1表示該bit為0；z=[0,1,0,1,0,1,0,1]，exclusive zero-prefix Z_i=[0,0,1,1,2,2,3,3]，總zeros Z=4。

Zero的目的位置是Z_i；one的目的位置是Z+(i−Z_i)。因此dest=[4,0,5,1,6,2,7,3]，scatter後為[2,0,4,6,7,5,3,1]。兩區內prefix都嚴格按原相對順序增加，所以partition stable、目的位置不重複，覆蓋0–7。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## LSD radix sort 的歸納與完整結果

在前一輪stable結果上處理下一個bit，實際三輪為

| 已處理bits | 結果 |
| --- | --- |
| bit0 | [2,0,4,6,7,5,3,1] |
| bit0–1 | [0,4,5,1,2,6,7,3] |
| bit0–2 | [0,1,2,3,4,5,6,7] |

歸納假設前b個低bits已排序。新partition按bit b分區，同區保留前一輪低bits順序，因此前b+1 bits有序。k-bit keys共k輪，每輪scan與scatter工作O(N)，總O(kN)，需可避免讀寫互相覆蓋的輸入/輸出storage。

這裡明定unsigned keys；signed或floating-point keys須先定義可保持所需順序的bit transform，不能直接把原始encoding當作相同unsigned順序。

## N-body 的 force 與 acceleration

對target i，softened acceleration為

$$\mathbf{a}_i = G \sum_{j \neq i} \frac{m_j (\mathbf{x}_j - \mathbf{x}_i)}{(\|\mathbf{x}_j - \mathbf{x}_i\|^2 + \varepsilon^2)^{3/2}}.$$

Target mass不在acceleration的分子，force才是m_i a_i。ε改善近距離奇異問題，卻也改變物理模型，不能只稱為免費的數值修正。

一維x=[0,1,3,4]、全部質量與G皆1、ε=0且跳過self terms，a0=1+1/9+1/16=169/144；a1=−1+1/4+1/9=−23/36；a2=23/36；a3=−169/144。總force為0，可核對成對作用的符號；有限精度與累加順序仍可造成小誤差。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## N-body tile 的資料重用與兩道 barrier

每block B個target threads，合作載入B個source bodies到shared，再各自累加對該tile的作用。偽碼如下，body包含position與mass。

```
i = block_id * B + thread_id
my = (i < N) ? body[i] : zero_body
acc = 0
for base = 0; base < N; base += B:
    j = base + thread_id
    sh[thread_id] = (j < N) ? body[j] : zero_body
    block_barrier()
    if i < N:
        for k = 0 .. min(B,N-base)-1:
            if base+k != i:
                acc += acceleration(my, sh[k])
    block_barrier()
if i < N:
    output[i] = acc
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## N-body tile 的資料重用與兩道 barrier （續）

第一道barrier確保tile全部可讀，第二道防止下一輪提早覆寫；無效target thread仍協助載入並到達barriers。每個target依序處理所有有效source一次，故和直接雙loop有相同數學項，並未減少$\Theta(N^2)$ pair工作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Tile 減少哪一層流量

明定N為B倍數、無cache重用，naive每個target另載入一次my，並讀N個source（包括self，讀入後才跳過其計算）。Source reads=$N^2$；tiled有N/B個blocks，每block讀N個sources，為$N^2/B$。

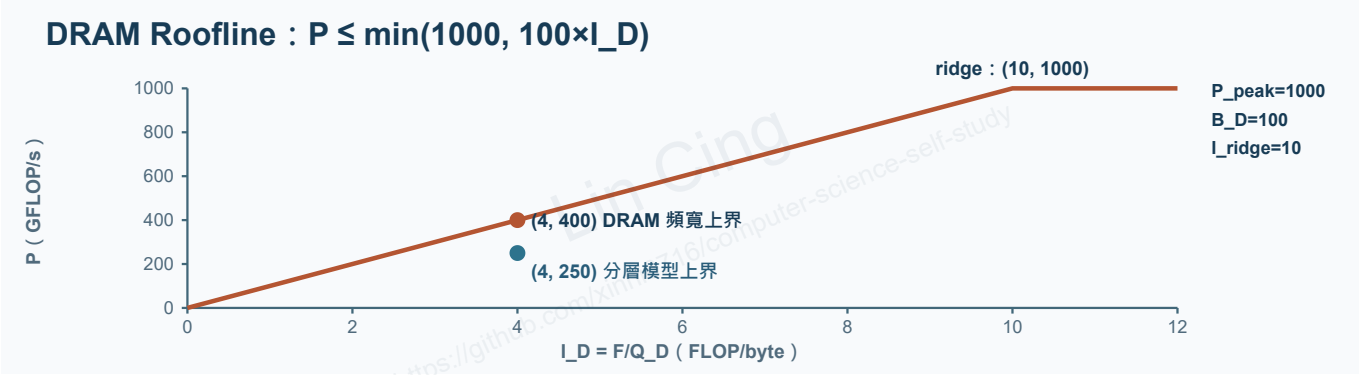
N=4、B=2時，source reads由16減到8；另加4個my loads，總position loads由20減到12，總流量只降為原本60%，不能宣稱所有流量恰減半。若compiler/cache重用self或其他sources，或body欄位寬度不同，需按實際介面bytes重算。

Shared內每個source仍可能被B個targets讀取，因此DRAM bytes降低不等於shared bytes也以相同比例降低。下一節以每個介面自己的Q計算Roofline限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 分層流量的限制不能混乘

固定同一 phase $F = 10^9$ FLOPs、$P_C = 1000$ GFLOP/s。DRAM 實際讀寫 $Q_D = 0.25 \times 10^9$ bytes，持續頻寬 $B_D = 100$ GB/s；shared memory 實際讀寫 $Q_S = 4 \times 10^9$ bytes，持續頻寬 $B_S = 1000$ GB/s。這些是明定模型輸入，不是某張 GPU 的實測規格。




DRAM 層的橫座標是

$$I_D = F/Q_D = 10^9/(0.25 \times 10^9) = 4 \text{ FLOP/byte}.$$

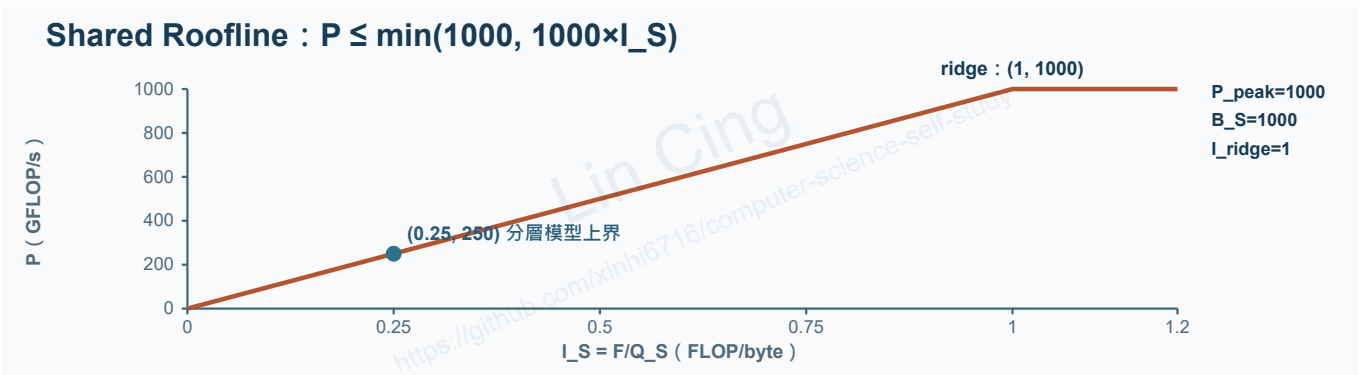


## 分層流量的限制不能混乘 （續）

其 throughput 上界為

$$\min(P_C, I_D B_D) = \min(1000, 4 \cdot 100) = 400 \text{ GFLOP/s}.$$

DRAM roof 的折點是 $P_C/B_D = 10$；$(4, 400)$ 在斜線的 bandwidth 部分。



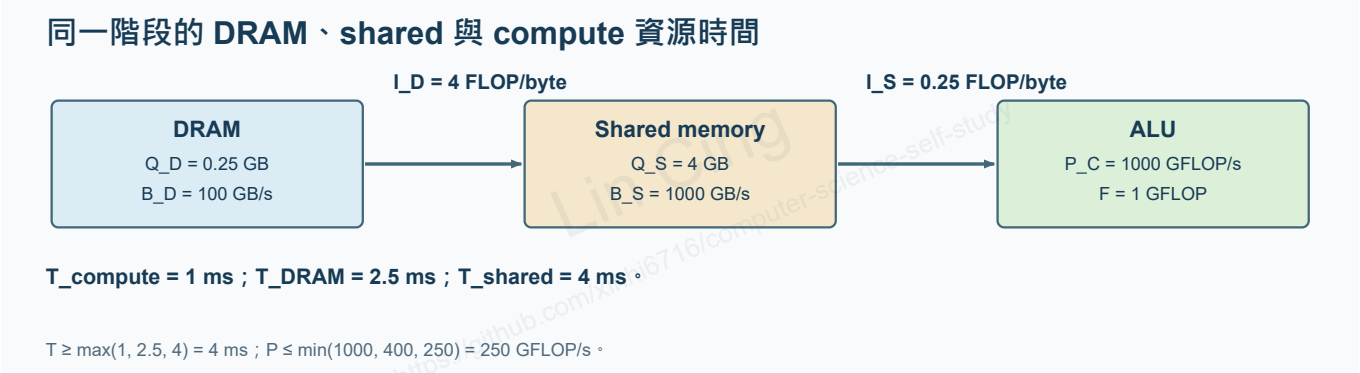

Shared 層要重新計算

$$I_S = F/Q_S = 10^9/(4 \times 10^9) = 0.25 \text{ FLOP/byte}.$$

## 分層流量的限制不能混乘 （續）

$$\min(P_C, I_S B_S) = \min(1000, 0.25 \cdot 1000) = 250 \text{ GFLOP/s}.$$

Shared roof 的折點是 1；1000 GFLOP/s 才是 plateau，$(0.25, 250)$ 位於斜線。250 不能畫成 shared roof 的水平上限。




三個 resource times 是

$$T_{\text{compute}} = 10^9/(1000 \times 10^9) = 0.001 \text{ s},$$



## 分層流量的限制不能混乘 （續）

$$T_D = 0.25 \times 10^9/(100 \times 10^9) = 0.0025 \text{ s}, \qquad T_S = 4 \times 10^9/(1000 \times 10^9) = 0.004 \text{ s}.$$

若各資源 traffic 可以重疊，

$$T \geq \max(1, 2.5, 4) = 4 \text{ ms},$$

$$P \leq F/T = 10^9/0.004 = 250 \text{ GFLOP/s}.$$

最終 $(I_D, I_S) = (4, 0.25)$ 對應的跨層上界是 250 GFLOP/s。它是 declared model bound，不是 measured throughput；實際 kernel 還可能因 bank replay、同步、occupancy 或額外 traffic 低於此值。

| 層級 | bytes | 強度 | 可持續頻寬 | 該層上限 |
|---|---|---|---|---|
| compute | 不適用 | 不適用 | $P_C = 1000$ GFLOP/s | 1000 GFLOP/s |
| DRAM | $0.25 \times 10^9$ | 4 FLOP/byte | 100 GB/s | 400 GFLOP/s |
| shared | $4 \times 10^9$ | 0.25 FLOP/byte | 1000 GB/s | 250 GFLOP/s |
| 合併 | 各層獨立 | 各層獨立 | 各層獨立 | 250 GFLOP/s |

## A.9 用明確反例檢查性能推論

高occupancy未必快，因為減register引起spill可增加memory traffic。連續access未必滿DRAM頻寬，因為outstanding requests、channel分配或依賴鏈仍可限制；shared memory也可能因bank conflicts與barriers而變慢。

單一kernel的peak不能當作應用speedup。相同FLOP數而含不同精度、近似函數、FMA計數或transfer邊界，也不可直接比較。若量測P超過由同phase之F/Q與有效頻寬得到的roof，先核對FLOP計數、cache/interface bytes、時間範圍及頻寬假設，而非立即宣稱推翻資源上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## A.10 正確性先於 mapping 選擇

CSR需保持每row完整點積與唯一writer；scan需保留prefix不變量與同步順序；radix需stable partition；N-body需覆蓋全部pair並正確管理tile生命期。以上契約成立後，才比較row-per-thread、warp-per-row、tile size或register使用量。

圖、trace與有限模型核對可定位錯誤，但一般性正確性分別依賴唯一索引、loop/tree invariant與歸納。GPU世代改變不會讓不正確的同步或越界程式因「通常同時執行」而變得合法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## A.11 歷史架構與可遷移方法

GPU由固定graphics pipeline演進為unified processors，CUDA/PTX把大量thread程式映到可伸縮的硬體。Warp寬度、banks、cache與精度支援可變，仍需分清latency、throughput、resource capacity與資料交換的語意。

本章歷史GeForce 8800的結構來自原書，sector/bank/occupancy數例則明定為獨立抽象模型。把模型的參數與推導分開，讀者才可替換成新硬體的真規格重新分析，而不是背一組過時常數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 參考資料

Patterson與Hennessy，*Computer Organization and Design: The Hardware/Software Interface*，Revised Fourth Edition，Appendix A，A.1–A.11，印刷A-3–A-77（PDF實體732–806）。Warp排程與divergence見A-28–A-30（PDF757–759）；16-bank模型見A-40（PDF769）；GeForce結構FigA.7.1見A-47（PDF776）；scan/reduction/radix與N-body見A-60–A-69（PDF789–798）。

Williams、Waterman與Patterson，〈Roofline: An Insightful Visual Performance Model for Multicore Architectures〉，*Communications of the ACM*52(4)，2009，DOI。

分層bytes定義另見NERSC Roofline文件。本文的32-bank、32-byte sectors、occupancy及計算數據是各頁明定的教學模型，未冒充同一款GPU的實測。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

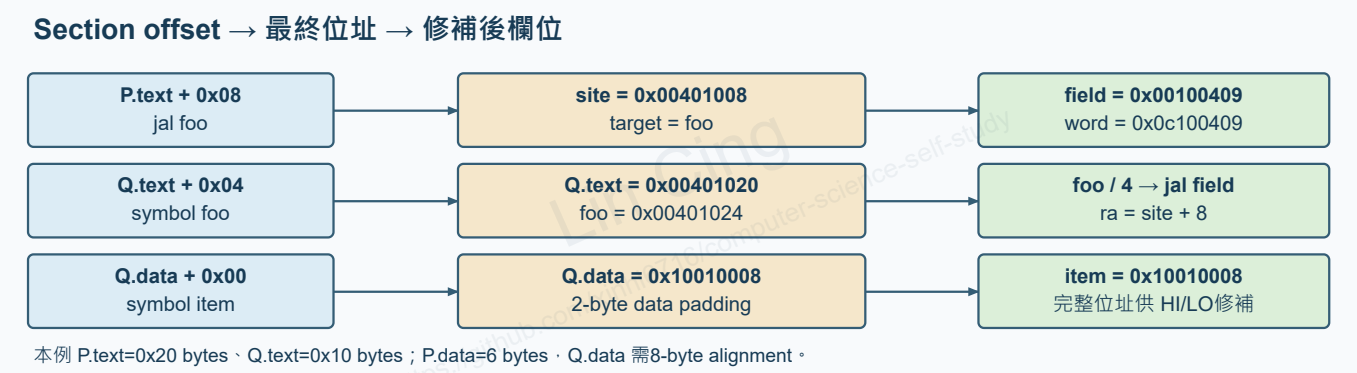
Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 原始碼到機器狀態的五個邊界

Compiler把高階程式轉成assembly或直接產生object。Assembler（組譯器）把指令、標籤與directives轉成machine code、symbols與relocations。Linker解析跨object的名稱並安排最終位址；loader建立process映射；CPU按ISA執行。




Object能被解析不代表可執行，link成功不代表呼叫慣例正確。每層有不同不變量：指令欄位合法、symbol綁定唯一、relocation位址正確、segment權限正確，以及caller/callee保存共同約定的狀態。



## 本章 MIPS 模型

採傳統pre-Release-6 MIPS32，每條machine instruction4 bytes，branch/jump有一個delay slot，`jal`寫入`$ra=PC+8`。範例以nop明示slot，不假設load delay。SPIM需選相容的delayed-branch設定；它的syscall是教學介面，不是Linux MIPS syscall ABI。

位址運算採32-bit，整數結果需要時按mod2^32解讀。一般branch immediate先sign-extend再乘4，J-format target則由PC+4高4 bits拼接26-bit field與兩個0。本文簡化object與relocation模型會明示addend，不能當作完整ELF parser。

## B.1 Assembly 每一行的不同意義

`addu $t0,$t1,$t2`是machine instruction。`.text`切換section、`.word`配置資料、`.align n`在本章assembler語法對齊到2^n bytes，這些directives不由CPU逐行執行。

`move $s0,$s1`可展開為`addu $s0,$s1,$zero`；`la $t0,label`可能展開成兩條甚至不同位址取得序列。Macro是文字/語法層展開，不建立runtime call frame。程式大小必須由展開後的指令、資料及padding計算，不能按source行數乘4。

直接用assembly適合boot、exception entry或需要精確ABI的低階邊界；效能優勢須依實際編碼與模型量測，不能由「較低階」推論一定較快。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.2 兩遍組譯的資料結構

第一遍維護每個section自己的location counter，並記錄symbol的section與offset。遇到label就綁定當前counter；遇到指令或資料就增加確切size；遇到alignment就向上取到合法邊界。

第二遍依相同展開與size規則編碼。已知同section的PC-relative reference可解出差值；仍需最終section base或外部symbol者產生relocation。兩遍若對pseudoinstruction長度的判斷不同，所有後續label都會錯；實作可先建立已展開的中間表示，避免各遍各猜一次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Location counter 的實際追蹤

本例.text offset從0開始，每條顯示的指令均已是4-byte machine instruction。

```
.text
main: addiu $t0,$zero,0
loop: addiu $t0,$t0,1
      bne   $t0,$a0,loop
      nop
done: jr    $ra
      nop
.data
msg:  .asciiz "Hi"
.align 2
value: .word 0x1234abcd
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Location counter 的實際追蹤 (續)

| 事件 | section offset之前 | 配置/對齊後 |
|---|---|---|
| main指令 | text0 | text4 |
| loop指令 | text4 | text8 |
| bne、nop | text8 | text16 |
| done: jr、nop | text16 | text24 |
| msg及NUL | data0 | data3 |
| .align 2 | data3 | data4，補1 byte |
| value | data4 | data8 |

因此symbols為main=(text,0)、loop=(text,4)、done=(text,16)、msg=(data,0)、value=(data,4)。
Section base直到link才決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Branch encoding 與 relocation invariance

若text base=0x00400000，bne在0x00400008，loop在0x00400004。Immediate為

$$\frac{target-(PC+4)}{4}=\frac{0x00400004-0x0040000c}{4}=-2=0xfffe.$$

bne opcode5、rs=$t0=8$、$rt=$a0=4，完整word為

$$(5\ll 26)|(8\ll 21)|(4\ll 16)|0xfffe=0x1504fffe.$$

整個text搬移δ時，target與PC同加δ，差值不變。因此已知同section、相對位置不變的branch通常不需為單純整段搬移而修改。若linker改寫layout或target在其他section，就要重算並檢查signed16-bit範圍及4-byte對齊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



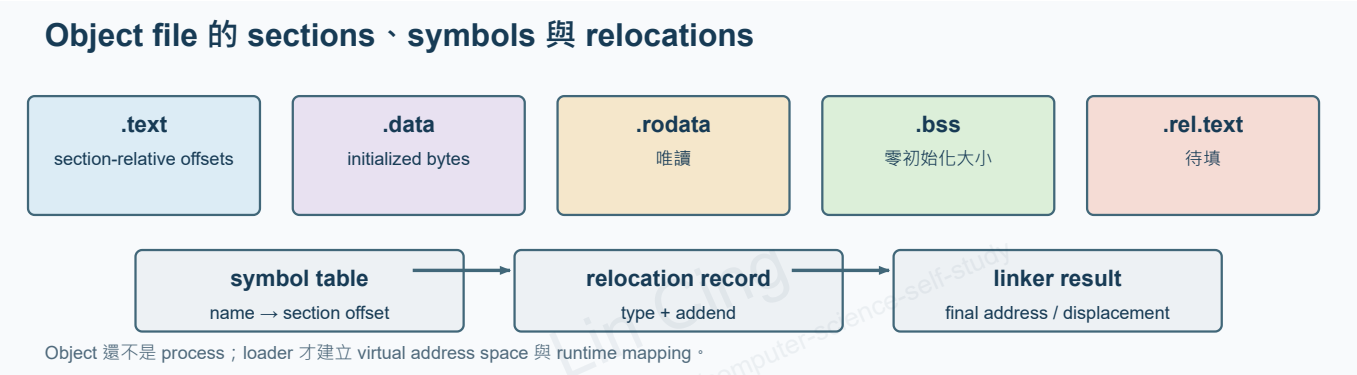
## Macro 展開也可能改變輸入

假設一個不安全的macro要把argument乘3，展開成 `addiu $t0,$zero,3; mult $arg,$t0; mflo $v0`。若呼叫的argument正是t0，且原值7，展開後第一條已把7覆寫成3，結果為9而非21。Macro作者必須明定clobbers，保證讀取原argument前不覆寫它，或使用不與arguments衝突的暫存策略。

重複展開內部label也需產生唯一名稱，否則可能重複定義或跳到另一份macro。Conditional assembly在組譯時選擇要輸出的code，不是CPU執行期的conditional branch；未輸出的分支沒有runtime成本，但配置與symbol table會隨條件改變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Object file 不是記憶體快照




`.text`保存指令，`.data`保存有初值的可寫資料，`.rodata`保存唯讀常數，`.bss`描述應配置並清零的大小，通常不需在檔案中放同量的0 bytes。Symbol除了名稱還有section、value、binding與type。

Relocation record至少要指出site、symbol、type及addend的取得方式。有的格式把addend放在record，有的編在原欄位；不能只看欄位0就猜「不需要修補」。Debug資訊也不是執行時一般load/store的資料。



## Symbol resolution 的範圍

Local symbols只在各自object有效，所以兩個object可各有名為loop的local label。兩個同名strong global definitions通常是錯誤；undefined reference必須由合法definition、library或執行期綁定機制滿足。

Weak/common與symbol visibility依格式及工具鏈規則處理，不能泛稱linker任選一個。以下例子只用唯一strong definition，使每個reference有明確target；若不唯一，就先報錯，不能靠relocation補救名稱歧義。

## B.3 合併兩個 objects 的 sections

明定P.text大小0x20、Q.text大小0x10；P.data大小6 bytes、Q.data大小4 bytes且需8-byte alignment。Text base=0x00401000，data base=0x10010000，按P、Q順序合併。

| 輸入section | 最終base | 範圍，含首不含尾 |
|---|---|---|
| P.text | 0x00401000 | [0x00401000,0x00401020) |
| Q.text | 0x00401020 | [0x00401020,0x00401030) |
| P.data | 0x10010000 | [0x10010000,0x10010006) |
| padding | 0x10010006 | 2 bytes |
| Q.data | 0x10010008 | [0x10010008,0x1001000c) |

若Q中foo=(text,4)、item=(data,0)，則最終foo=0x00401024、item=0x10010008。每個symbol的offset加的是**自己輸入section的最終base**，不是一律加整個executable base。



## 一個 jal relocation 的全部欄位

P.text offset8有`jal foo`，site=0x00401008。Foo=0x00401024且aligned，PC+4=0x0040100c的高4 bits與target相同，所以可用J-format。

$$field = (0x00401024 \gg 2)\ \&\ 0x03ffffff = 0x00100409,$$

$$word = (3 \ll 26) | field = 0x0c100409.$$

CPU重建target為 $\{(PC+4)[31:28], field, 00\} = 0x00401024$。`jal`另把return address 0x00401010寫入ra，並先執行site+4的delay-slot nop。若target跨256 MiB區域，截26 bits不能修好；需合法的veneer或間接跳躍序列，不能默默截斷。

## High/low relocation 與 sign extension

對V=0x1234abcd，`lui/ori`採零延伸low：high=0x1234、low=0xabcd，$(high \ll 16)|low = V$。

若第二條用`addiu`，low立即數sign-extend，0xabcd代表−21555。因此high必須進位：

$$H = ((V + 0x8000) \gg 16) \;\&\; 0xffff = 0x1235,$$

$$(0x1235 \ll 16) - 21555 = 0x1234abcd.$$

把0x1234直接配addiu得到0x1233abcd，差0x10000。實際MIPS HI16/LO16配對relocation還要依ABI處理addend與pair關係；本例是完整最終值V已知後的算術，不將兩個獨立截半視為所有relocation的實作。

Lin Cing https://github.com/xinhi6716/computer-science-self-study



## Static archive 的一次掃描例

明定教學linker由左至右處理檔案，object必納入；遇archive時，反覆抽出能滿足當前undefined symbols的members直到該archive不再有新需求，之後不自動重掃前面的archive。

Main.o定義main、引用f；libF.a的F.o定義f、引用g；libG.a的G.o定義g。`main.o libF.a libG.a`的未解集合依序為{f}、{g}、{}，成功。

`main.o libG.a libF.a`先在libG時只有f未解，無member可抽出；libF再抽F.o後留下g，最終失敗。這不是C語言語意而是明定archive選取規則。實際工具鏈可有group/rescan選項，不能把這個單向模型當作所有linker唯一行為。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 動態連結與 GOT 間接引用

Static link把選取的library code合併進executable。Dynamic link則在process建立時或首次需要時，把reference綁到shared object的實際位址。不同process可以把同一唯讀code頁映到不同VA，共享的是physical backing，不表示可寫global data也必須共享。

用抽象GOT slot例，code從固定可達的slot讀取function pointer。Load前slot有待修補狀態；loader找到foo=0x70001234後填入slot；之後的indirect call取該值。Lazy binding可先導向resolver再更新slot，但實際stub/register規則依ABI，不把此抽象當作MIPS或x86的完整PLT指令。

ASLR讓base改變，position-independent code使用相對位址或間接表減少text修補。仍需正確symbol版本與呼叫ABI，載入相同名稱不保證介面相容。



## B.4 Loader 的 file size 與 memory size

明定一個可寫segment：file offset0x2000、VA0x10010000、file size0x100、memory size0x180，page size0x1000。Loader從檔案映入首0x100 bytes，並把後0x80 bytes清零；不把檔案後面不相關的bytes當作bss。

Segment與section不同。Linker可把多個sections合進一個具共同權限的loadable segment；loader主要依可執行格式的load資訊建立映射。Text可設R-X，data設RW-，未映射guard page捕捉部分越界。

Runtime entry先建立執行環境並呼叫main，因此main不是process的第一條instruction。實際ELF、PIE與interpreter流程依OS，不能只依section名稱推知全部runtime權限。

## B.5 位址、byte order 與對齊

32-bit值0x12345678放在位址0x100：big-endian bytes為12、34、56、78；little-endian為78、56、34、12。Endianness改變byte位置，不改變register中數值，也不把指令的欄位bits左右顛倒。

Strict-alignment MIPS的lw要求address低2 bits為0；0x102不能直接作lw。Packet序列化要固定欄位寬度、byte order及padding；native struct的memory dump不是跨機器協定。

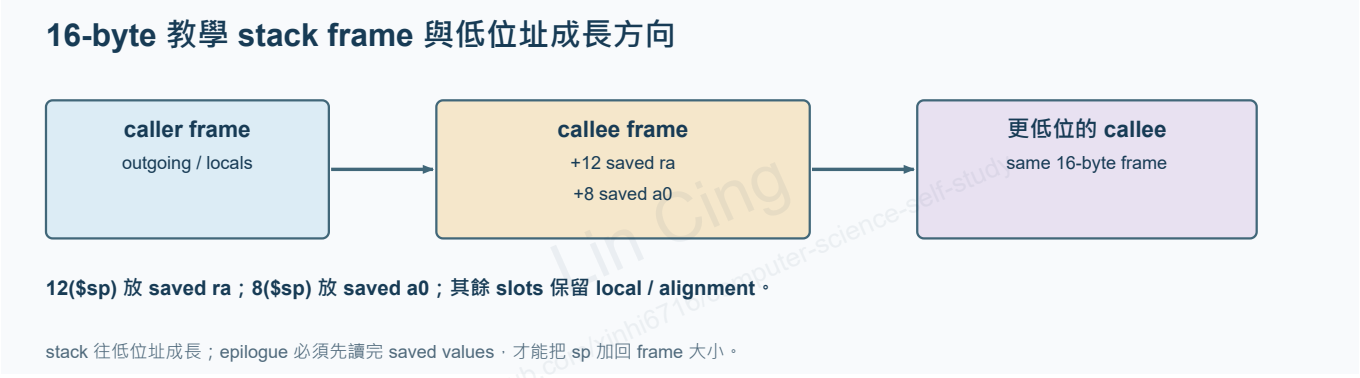
Stack與heap的成長方向、mapping與guard由ABI/OS決定，典型示意不是所有process的唯一layout。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.6 Caller 與 callee 保存責任

Caller把前四個整數arguments放a0–a3，常以v0/v1取結果。t-registers通常caller-saved，s-registers通常callee-saved；callee若改s0，return前須還原incoming s0。Caller若call後還要舊a0，需自己保存。




本章遞迴例使用16-byte簡化教學frame，不宣稱完整o32 non-leaf frame。完整ABI還可能要求outgoing argument area、特定alignment及其他保存欄位。只符合ISA、卻不符合呼叫慣例的函式，仍可能破壞其他編譯器產生的caller。

## sum_to 的完整遞迴程式

明定0<=n且和不溢出signed32-bit，sp初始aligned且空間足夠。本程式使用addu作模2^32相加，輸入界限確保同時等於數學答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## sum_to 的完整遞迴程式 (續)

```
sum_to:
  addiu $sp,$sp,-16
  sw    $ra,12($sp)
  sw    $a0,8($sp)
  beq   $a0,$zero,base
  nop
  addiu $a0,$a0,-1
  jal   sum_to
  nop
  lw    $t0,8($sp)
  addu  $v0,$v0,$t0
  j     done
  nop
base:
  addu  $v0,$zero,$zero
done:
  lw    $ra,12($sp)
  addiu $sp,$sp,16
  jr    $ra
  nop
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## sum_to 的完整遞迴程式 （續）

每條控制轉移後都有slot。遞迴返回後重新從本層frame讀n，不能假設a0仍是原值，也不能把遞迴呼叫前的t0當作callee必保留值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 遞迴的實際 frames 與返回值

以初始sp=0x1000呼叫sum_to(3)，base case也建立frame。

| n | 本層sp | saved n位址 | saved ra位址 | 返回v0 |
|---|---|---|---|---|
| 3 | 0x0ff0 | 0x0ff8 | 0x0ffc | 6 |
| 2 | 0x0fe0 | 0x0fe8 | 0x0fec | 3 |
| 1 | 0x0fd0 | 0x0fd8 | 0x0fdc | 1 |
| 0 | 0x0fc0 | 0x0fc8 | 0x0fcc | 0 |

返回由最內層依序做0、0+1=1、1+2=3、3+3=6，sp回到0x1000。最大同時4 frames，共64 bytes。各層必須先用本層sp對應的offset載回ra，再釋放frame；若先改sp，仍可用重新計算的地址讀取，但不能繼續盲用舊offset。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

### 遞迴正確性的歸納

令函式契約為：輸入n滿足範圍時，return有v0=n(n+1)/2、sp與callee-saved registers恢復。n=0時設定v0=0，epilogue取回ra/sp，符合契約。

對n>0，假設sum_to(n−1)符合契約。呼叫前把本層n與ra保存在自身frame，child的frame更低且不重疊；child返回後sp回到本層值，取回n並相加得(n−1)n/2+n=n(n+1)/2，最後還原ra/sp。n每次減1且有下界0，故終止。

此證明依賴stack容量與不溢出前提。無限遞迴、negative n或stack overflow不在契約內，不能只用已跑過n=3當作一般證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



### B.7 Exception 不受一般 call ABI 限制

同步exception由當前指令引起，如overflow、address fault或syscall；interrupt來自timer或device等外部事件。Handler可在任何程式位置打斷執行，不能只保存callee-saved registers，因為被中斷程式也可能正使用caller-saved registers。

硬體保存或提供恢復所需的PC與status，軟體保存自己會改寫的registers。Handler需判斷cause、處理/清除必要pending source，再依可恢復語意返回。未清裝置原因就return，可能立即再次中斷；nested handler也需獨立保存狀態，避免覆寫outer context。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## EPC 與 branch delay 的兩種情形

非delay-slot的可恢復fault：Cause.BD=0，EPC指向faulting instruction。若修好page mapping並重試load，通常回到該EPC；若已完成一個syscall服務，則在明定政策下由下一條繼續。

若branch在0x00400020，其delay slot的lw在0x00400024發生address fault，則BD=1、EPC=0x00400020。EPC指向branch，不是lw。單純EPC+4會直接跳到slot且遺失branch語意；handler須保留/重建branch決策與slot狀態，或採可正確重新執行的處理方式。

Precise exception要求較舊指令已完成、faulting及較年輕指令未提交不應有的effects。外部interrupt優先權不能讓較年輕同步fault越過較舊指令的程式順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.8 MMIO receiver 的 polling 例

以下是明定教學device，不宣稱所有SPIM版本固定此位址。Receiver control=0xffff0000、data=0xffff0004，control bit0=ready；讀data消耗一個byte並清ready。MMIO映射uncached，平台保證本序列所需I/O順序。

```
lui   $t0,0xffff
poll:
lw    $t1,0($t0)
andi  $t1,$t1,1
beq   $t1,$zero,poll
nop
lw    $v0,4($t0)
andi  $v0,$v0,0xff
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

若control依序回傳0、0、1，loop讀control三次，只在第三次後讀data一次。把data load提前到branch delay slot會在not-ready時也讀取，改變裝置狀態，故slot保留nop。

## B.9 SPIM syscall 的可執行語意

在SPIM教學服務中，print-int常用v0=1、a0=整數；print-string用v0=4、a0=NUL結尾字串位址；exit用v0=10。服務編號是simulator介面，不是ISA opcode或跨OS通用編號。

```
addiu $v0,$zero,1
addiu $a0,$zero,42
syscall
addiu $v0,$zero,10
syscall
```

此處syscall不在delay slot。Simulator可能直接實作服務，不等同真OS執行一份相同的handler。觀察到輸出42可核對此程式，不足以證明真實MIPS系統的exception vector或kernel ABI相同。



## B.10 指令語意的查核表

| 類別/例 | 位元運算與重要條件 |
|---|---|
| add / addu | 同樣低32-bit和；add可因signed overflow trap，addu不trap |
| addiu | immediate sign-extend，不因signed overflow trap |
| andi / ori | immediate zero-extend |
| slt / sltu | 相同比特以signed/unsigned比較，可得不同結果 |
| lb / lbu | 讀8-bit後分別sign/zero extend |
| lw / sw | base+signext immediate；strict model要求word對齊 |
| beq / bne | 比較GPR，target=PC+4+(signext imm<<2)，執行slot |
| j / jal | 拼接PC+4高bits；jal另外寫ra=PC+8 |
| jr | register target，仍有slot |

## B.10 指令語意的查核表 （續）

例如bit pattern0xffffffff，slt把它解成−1，sltu解成4294967295。Signedness由使用的operation與資料契約決定，不是bits自己帶有signed標記。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## B.11 從錯誤位址回到來源

Fault位址若落在錯誤function，先查symbol resolution與link map；若constant只差0x10000，核對low sign extension及HI/LO配對；若call返回後register錯，核對ABI與delay slot；若bytes顛倒，核對序列化及endianness。

Assembler listing可顯示source、offset與word；object dump顯示symbols/relocations；link map顯示最終section位置；execution trace顯示PC與register變化。這些證據定位不同邊界，不能用「可能compiler bug」取代具體欄位核對。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## B.12 練習的模型邊界

本章的兩遍assembler、relocation、stack及MMIO例都先固定版本與語意，再推導結果。練習需要重算變更layout、低半部負值、archive相依與異常返回的影響，不能只背一組hex答案。

完整ISA還含floating-point、coprocessor與其他system instructions。查原書reference時應分清machine instruction與pseudo，並同時讀語意、例外、對齊及delay-slot限制，不能只查opcode一欄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 參考資料

Patterson與Hennessy，*Computer Organization and Design: The Hardware/Software Interface*，Revised Fourth Edition，Appendix B。B.2–B.9印刷B-10至B-44（PDF實體815–849）；B.3.1連結圖在印刷B-19（PDF824）。B.10為指令reference，印刷B-45至B-80（PDF850–885）。

本文object layout、數值、遞迴程式與練習為明定模型的獨立例子。需要完整平台ABI時，須再查該系統規格，不能把16-byte教學frame當作完整o32 non-leaf配置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study